陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing
空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology
西北工業大學
Northwestern Polytechnical University

Python 程序设计



主　讲：　邹晓春　电子信息学院

QQ 群：　Python程序设计
808366427

第0章 课程介绍

- 课程的作用
- 课程的目的
- 课程的特点
- 课程的安排
- 授课宗旨



2025-5-14

课程的作用

- 在欧美各高等学校，Python成为Web 和 Internet开发、科学计算和统计、人工智能、图形图像处理等诸多课程的基本教学工具，成为本科生、硕士生和博士生的必须掌握的基本技能。

•一些知名大学已经采用Python来教授程序设计课程。例如卡耐基梅隆大学的编程基础、麻省理工学院的计算机科学及编程导论就使用Python语言讲授。

- 在设计研究单位和工业部门，Python已被广泛地用于研究和解决各种具体的工程问题。

- 可以预见，Python将在我国科学研究和工程应用中发挥越来越大的作用。



课程的目的和特点



本课程的目的（ Objectives of This Course ）

讲授Python语言基础入门知识，介绍Python产品的体系、Python桌面工具的使用方法，重点介绍Python的基础知识、数值计算的基本步骤以及如何使用Python语言编写整洁、高效、规范的程序。并涉及到一些具体的专业应用项目（如：2D游戏、数据可视化等）。

通过本课程的学习，了解、熟悉、掌握 Python的基本编程方法，并具有初步的利用计算机处理、解决实际问题的能力，为进一步学习后续的专业课程做好准备。

本课程的特点（ Features of This Course ）

- 交叉性课程，是计算机技术、数学理论知识以及诸多工程理论知识的综合。
- 实践性课程。
- 内容多，课时少，要求同学上课认真听讲，要充分利用上机实践消化、理解、掌握课上讲解内容。

课程的安排

- **课堂教学：共32学时；**
- **考核方式**
  - 平时（20%）（上课）
  - 期末考试（80%）
- **考试形式**
  - 随堂、开卷
- **教材**
  - **《Python编程-从入门到实践》[美] Eric Matthes著；袁国忠 译，人民邮电出版社，2016年**
- **主要参考书**
  - 《笨办法学Python》、[美]泽德 A.肖著、王巍巍 译、人民邮电出版社、2020.10
  - 《Python程序设计》[美]凯·霍斯特曼、兰斯·尼塞斯著、车万翔 等译、机械工业出版社、2021.5



授课宗旨

- 讲授Python的通用功能。
- 寓教于例，由浅入深。
- 关于科学计算，着重强调理论概念、算法和实际编程三者之间的关系。

THANK YOU!



第1章 Python编程起步



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



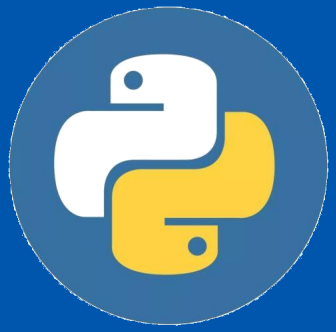
西北工業大學
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

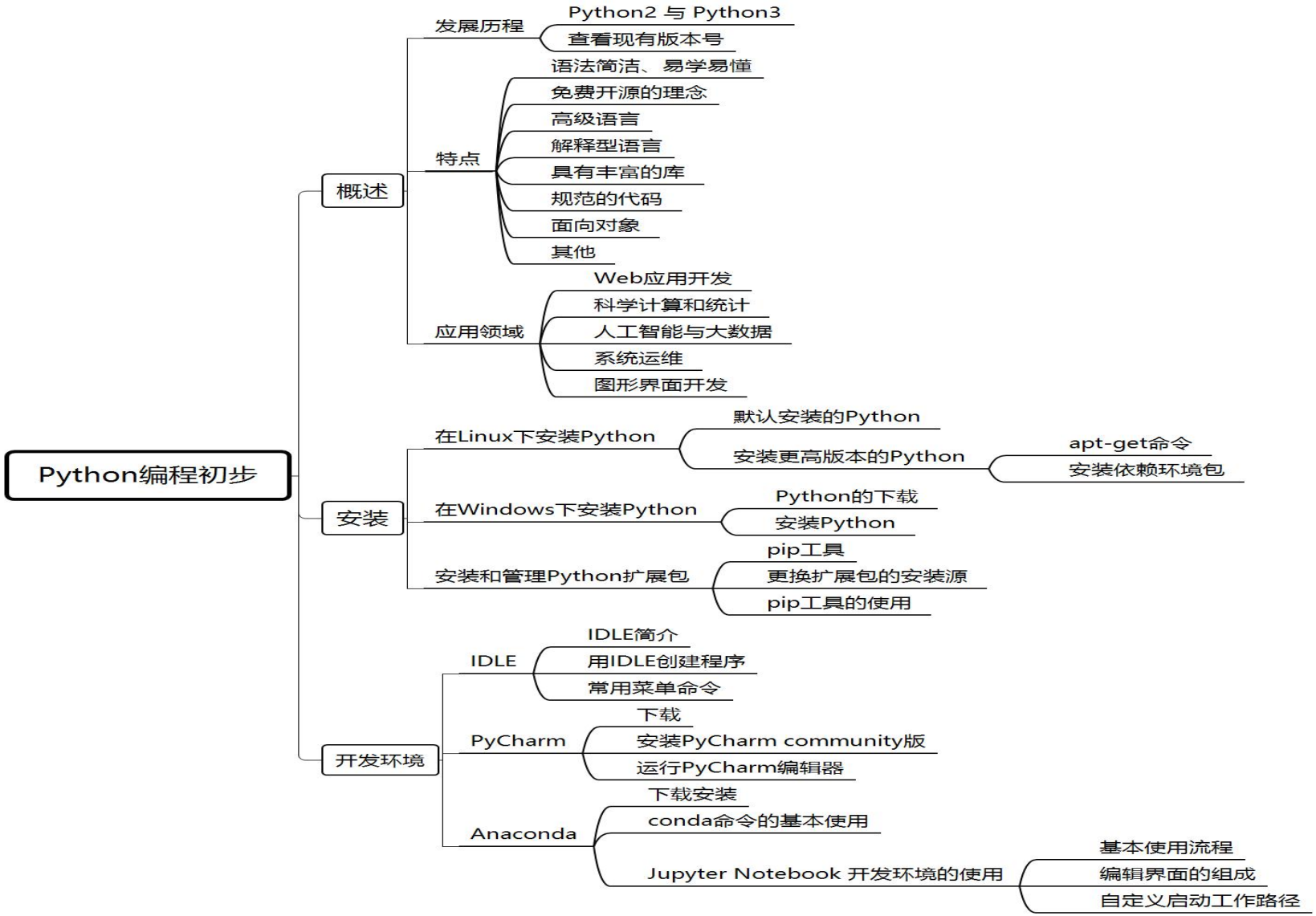
QQ 群：　Python程序设计
760581125



《Python程序设计》

第1章　Python编程起步

第1章 Python编程起步





第1章 Python编程起步



Python概述



Python的安装



Python的开发环境



第1章 Python编程起步

1.1 Python概述

•Python是一门跨平台、开源、免费的解释型高级动态编程语言，是一种通用编程语言。

- Python支持命令式编程、函数式编程，完全支持面向对象程序设计，语法简洁清晰，并且拥有大量的几乎支持所有领域应用开发的成熟扩展库。

1.1 Python概述

为什么用Python？

- 易于学习和容易理解
- 开源工具多
- Python可靠且高效
- 在人工智能领域的普及



1.1 Python概述

1.1.1 Python的发展历程

- Python 1.0 - 1994 年 1 月
- Python 2.0 - 2000 年 10 月
- Python 2.6 - 2008 年 10 月
- Python 2.7 - 2010 年 7 月
- Python 3.0 - 2008 年 12 月
- Python 3.6 - 2016 年 12 月
- Python 3.7 - 2018 年 6 月
- Python 3.8 - 2019 年 10 月



Guido van Rossum

1.1 Python概述

编程语言Python的得名就是因为Python的作者Guido van Rossum特别喜欢Monty Python飞行马戏团而以此命名的。Python语言的官方IDE编辑器叫做IDLE（成员：艾瑞克•爱都(Eric Idle)），此外还有其他程序员开发的The Eric Python IDE。此外，一款Python重构工具被取名为bicyclerepai，纪念Monty Python的Bicycle Repair Man小品。



Monty Python组合

而该团体被称为喜剧界的披头士，披头士，皇后乐队（Queen）。

1.1 Python概述

1.1.1 Python的发展历程

目前，Python官网同时发行Python 2.x和Python 3.x两个系列的版本。由于语法层面及解释器内部的诸多改进，为了不带入过多的累赘，Python 3.0在设计的时候没有考虑向下兼容。所有基于Python 2.x系列版本编写的库函数，都必须经过修改以后才能被Python 3.x系列版本使用。



Guido van Rossum

1.1 Python概述

1.1.1 Python的发展历程

Python2.X 和 Python3.X 的区别

1. 编码方式不同

Python2.x

```
>>> s = "我爱编程"
>>> s
'\xce\xd2\xb0\xae\xb1\xe0\xb3\xcc'
```

将字符串转换成GBK编码格式
GBK：国家简体中文字符集和繁体字符集，兼容ASCII，占两个字节

Python3.x

```
>>> s = "我爱编程"
>>> s
'我爱编程'
```

简体中文字符和繁体字符

1.1 Python概述

1.1.1 Python的发展历程

Python2.X 和 Python3.X 的区别

2. 语句输出不同

```
>>> print(3,4)
(3, 4)   数组形式
```

Python2.x

```
>>> print(3,4)
3 4   数字形式
```

Python3.x

1.1 Python概述

1.1.1 Python的发展历程

Python2.X 和 Python3.X 的区别

3. 除法运算“/”不同

```
>>> 1 / 2
0  整除
>>> 1.0 / 2.0
0.5 浮点除
```

Python2.x

```
>>> 1/2
0.5  浮点除
```

Python3.x

1.1 Python概述

1.1.1 Python的发展历程

Python2.X 和 Python3.X 的区别

4. 不等于运算符不同

```
>>> 1!=2
True
>>> 1<>2    不等号
True
```

Python2.x

```
>>> 1!=2
True
>>> 1<>2
  File "<stdin>", line 1
    1<>2
       ^
SyntaxError: invalid syntax
```

Python3.x

1.1 Python概述

1.1.1 Python的发展历程

Python2.X 和 Python3.X 的区别

5. import导入模块不同

Python2.x	Python3.x
默认是按照相对路径导入模块和包	import导入模块和包是默认按照绝对路径导入，如果只写相对路径，解释器会报错

1.1 Python概述

1.1.2 Python的特点

1. 语法简洁、易学易懂
2. 免费开源的理念
3. 是一种高级语言
4. 是一种解释型语言
5. 具有丰富的库
6. 规范的代码
7. 面向对象
8. 其他



1.1 Python概述

1.1.2 Python的特点

1. 语法简洁、易学易懂

Python有相对较少的关键字，结构简单，和一个明确定义的语法，学习起来更加简单。Python代码定义的更清晰，阅读一个良好的Python程序，就像在阅读一段语法要求严格的英语段落一样。

```
a = 10
print(type(a))
```

```
<class 'int'>
```

```
HP = 10
if HP > 20:
    print('此精灵宝贝的HP大于20')
else:
    print('此精灵宝贝的HP小于等于20')
```

1.1 Python概述

1.1.2 Python的特点

2. 免费开源的理念

用户可以自由地发布这个软件的拷贝，查看它的源代码，研究其代码细节或进行二次开发。具有许多针对不同项目设计的现成框架以及开源工具，拥有大量的文档、教程、问答等资料。



1.1 Python概述

1.1.2 Python的特点

3. Python是一种高级语言

低级语言泛指机器语言和汇编语言。

优点：执行速度快

缺点：

- 代码编写难度较大，可读性较差。
- 具有较高的机器相关性。

1.1 Python概述

1.1.2 Python的特点

3. Python是一种高级语言

高级语言

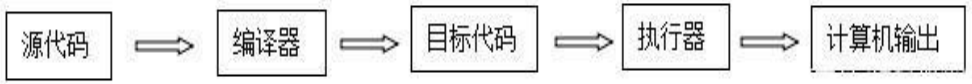
优点：

- 编写容易，代码可读性好。
- 可移植。

缺点：

- 需要先翻译成低级语言才能运行，效率相对较低。
- 对硬件的可控性相对较弱，目标代码量较大。

1.1 Python概述



1.1.2 Python的特点

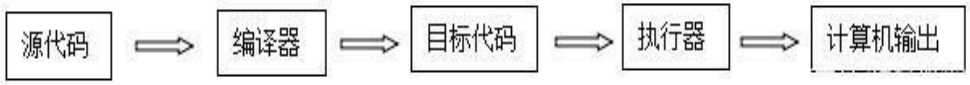
4. 是一种解释型语言

编译型语言可以从源文件转换成机器可以执行或调用的程序。这个过程通过编译器和不同的标记、选项完成。运行程序的时候，连接/转载器软件把程序从硬盘复制到内存中并且运行，无需重新编译，所以程序执行效率较高。

优点是运行速度快，代码效率高，编译后程序不可以修改，且看不到源码，保密性较好。

缺点是代码需要经过编译方可运行，可移植性差，只能在兼容的操作系统上运行。

1.1 Python概述



1.1.2 Python的特点

4. 是一种解释型语言

解释型语言不需要编译成二进制代码，可以直接从源代码运行程序。在计算机内部，Python解释器把源代码转换成字节码的中间形式，再把它翻译成计算机使用的机器语言并运行。

优点是可移植性好，只要有解释器环境，程序就可以在不同的操作系统上运行。

缺点是代码需要有专门的解释器，在程序运行时，除要给用户程序本身分配内存空间外，解释器也占用系统资源，所以其运行速度较慢。

1.1 Python概述

1.1.2 Python的特点

5. 具有丰富的库

具有一个广泛的标准库，它可以处理各种工作，包括正则表达式、文档生成、单元测试、线程、数据库、网页浏览器、电子邮件、GUI等与系统有关的操作。

除了标准库外，还有许多其他高质量的第三方库，如wxPython、Twisted和Python图像库等。

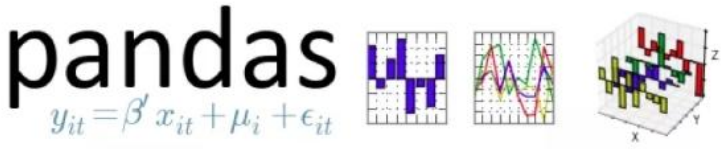
1.1 Python概述

1.1.2 Python的特点

5. 具有丰富的库















1.1 Python概述

1.1.2 Python的特点

6. 规范的代码

采用强制缩进的方式使得代码具有较佳的可读性。违反了缩进规则的程序不能通过编译，减少了程序员的自由度，有利于统一风格，使得人们在阅读代码时会更加轻松。

```
HP = 10
if HP > 20:
    print('此精灵宝贝的HP大于20')
else:
    print('此精灵宝贝的HP小于等于20')
```

1.1 Python概述

1.1.2 Python的特点

7. 面向对象

Python既支持面向过程的编程也支持面向对象的编程。

在面向过程的语言中，程序是由过程或是可重用代码的函数构建起来的。

在面向对象的语言中，程序是由数据和功能组合而成的对象构建起来的。与其他语言相比，Python以一种非常强大又简单的方式实现面向对象编程。

1.1 Python概述

1.1.2 Python的特点

7. 面向对象

- 面向过程的程序设计
  - ✓将要处理的数据保存到各种独立变量、数组等存储单元中
  - ✓通过流程控制语句，将业务逻辑写成程序代码
  - ✓为了使整个程序的结构更加清晰，引入函数的概念来简化程序设计的复杂性
- 面向对象的程序设计
  - ✓独立的变量和函数在现实世界中都是不存在的，都依附于某一事物身上，变量就是事物的特征，函数就是事物的功能（或者能力）
  - ✓解决一个问题是靠多个不同事物的相互作用来完成的

1.1 Python概述

1.1.2 Python的特点

8. 其他

- 可移植性：基于其开放源代码的特性，Python已经被移植到Linux、Windows等许多平台。

- 可扩展性：可以使用C或C++完成那部分程序，然后从你的Python程序中调用。

- 可嵌入性：还可以将Python嵌入到C/C++程序，让程序用户获得“脚本化”的能力。

1.1 Python概述

1.1.3 Python的应用领域

1. Web应用开发

2. 科学计算和统计

3. 人工智能与大数据

4. 系统运维

5. 图形界面开发



1.1 Python概述

1.1.3 Python的应用领域

1.Web应用开发









1.1 Python概述

1.1.3 Python的应用领域

2. 科学计算和统计

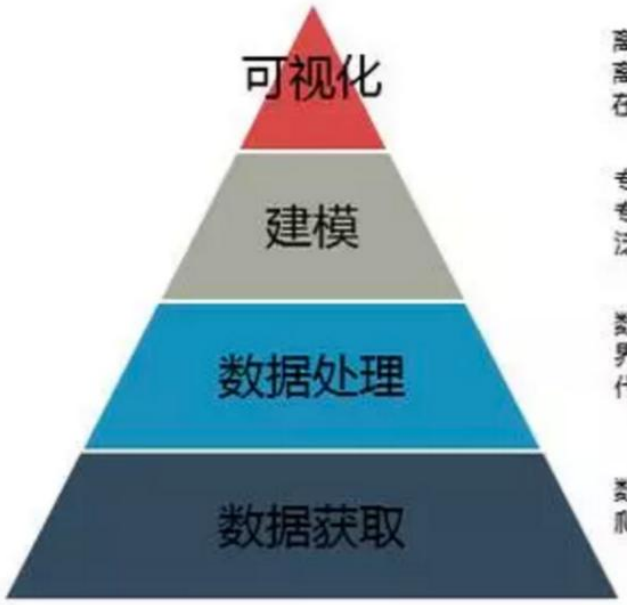


1.1 Python概述

1.1.3 Python的应用领域

3. 人工智能与大数据

python™



离线界面型：Excel，Powerpoint，PS，Tableau……
离线代码型：R，SAS，Python，D3.js，Process……
在线型：Echarts，Tagxedo……

专用界面型：SPSS……
专用代码型：R，SAS……
泛用代码型：Python

数据库：SQL，Hive……
界面型：Excel，SPSS……
代码型：VBA，Python，R，SAS……

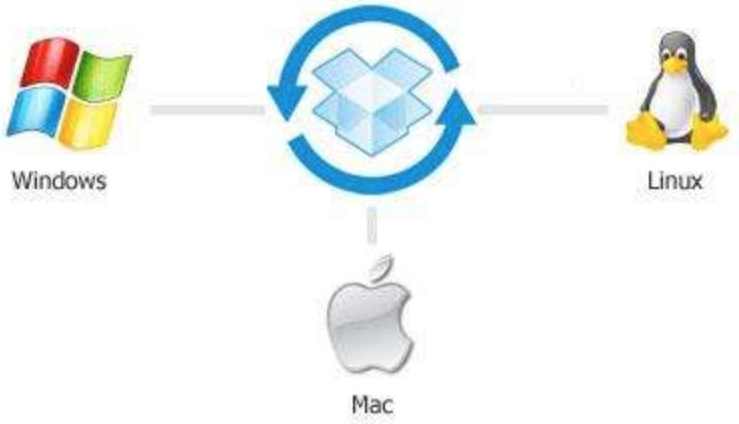
数据库：SQL，Hadoop，Hive……
爬虫：Python，Java……

1.1 Python概述

1.1.3 Python的应用领域

4. 系统运维





1.1 Python概述

1.1.3 Python的应用领域

5. 图形界面开发



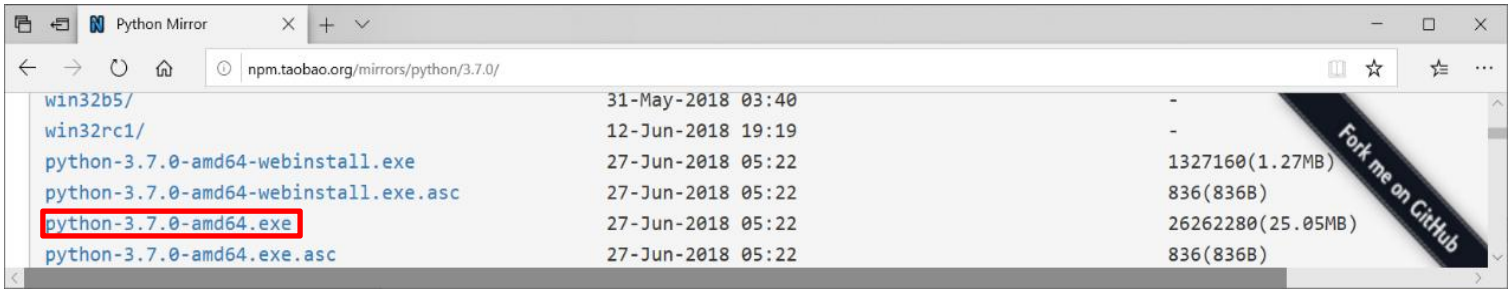




1.2 Python的安装

1.2.1 在Windows下安装Python

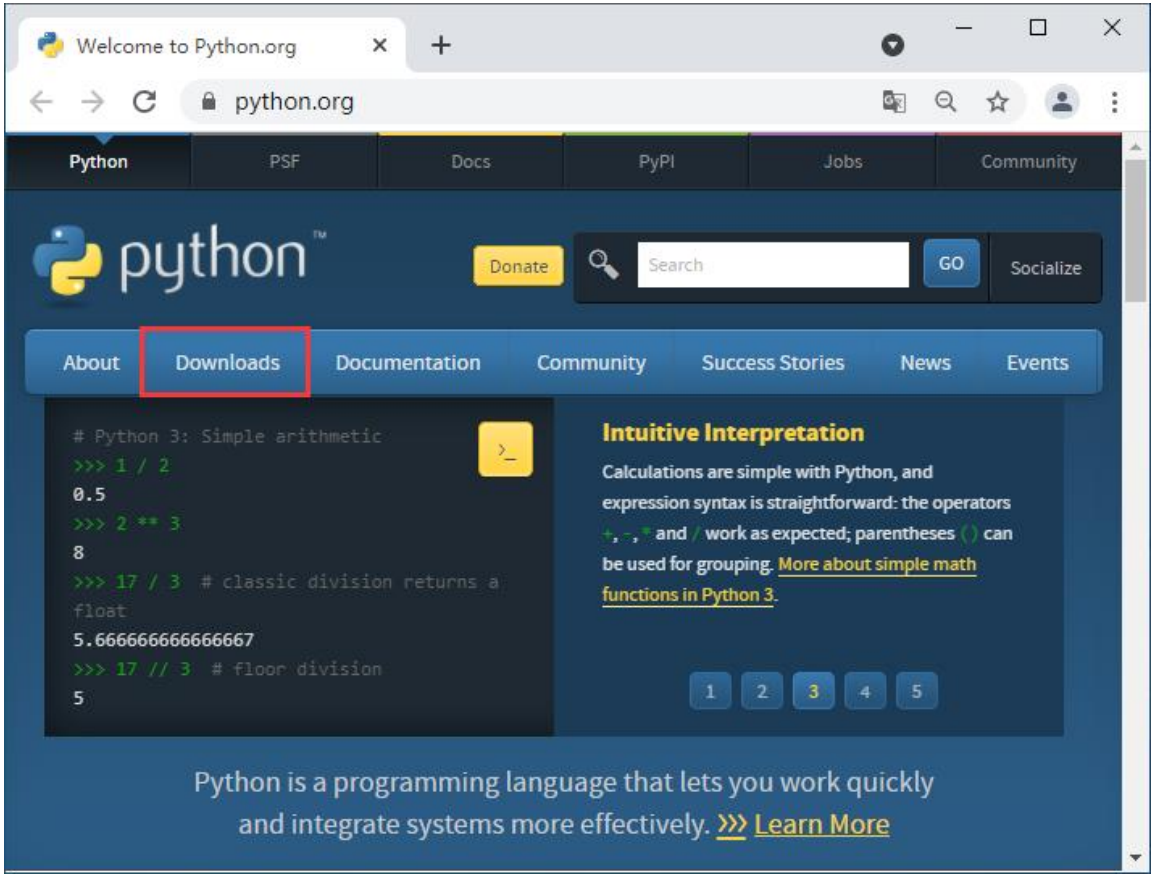
1. Python的下载



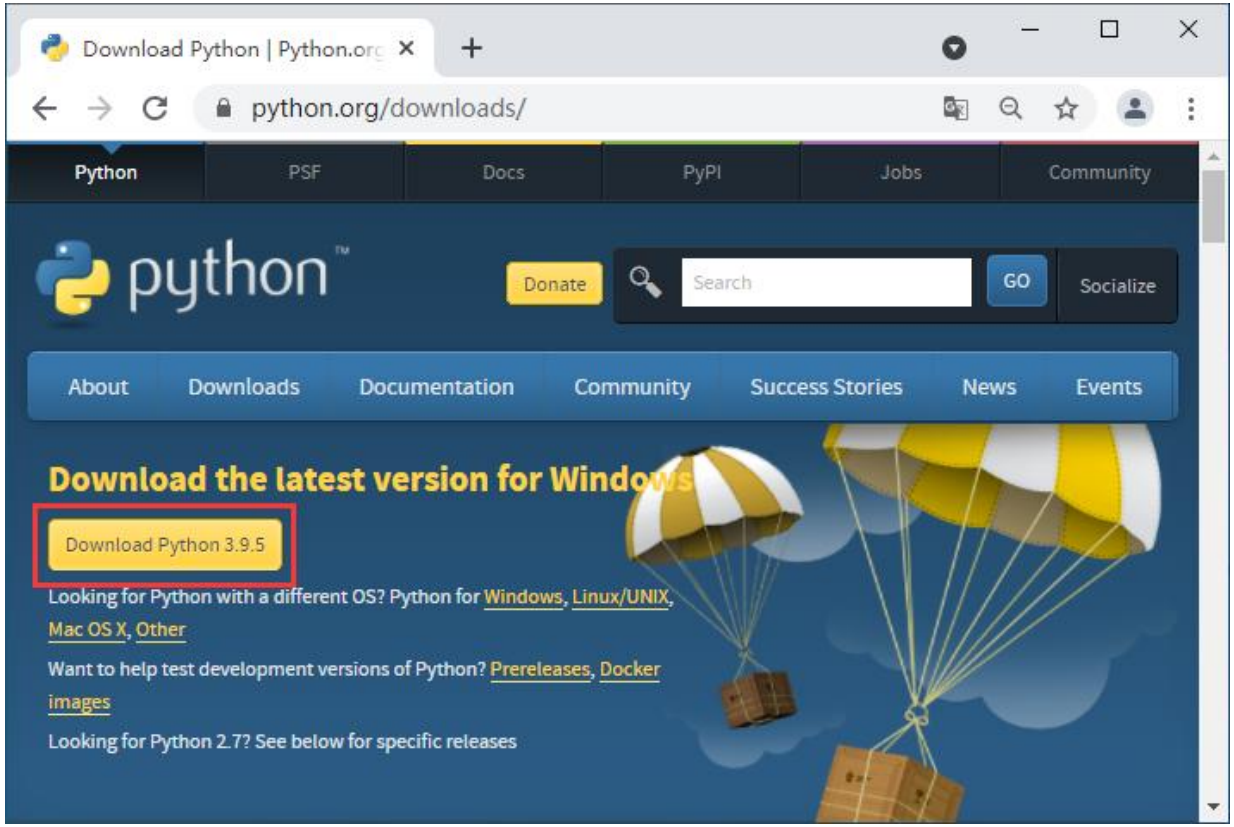


第1章 Python编程起步

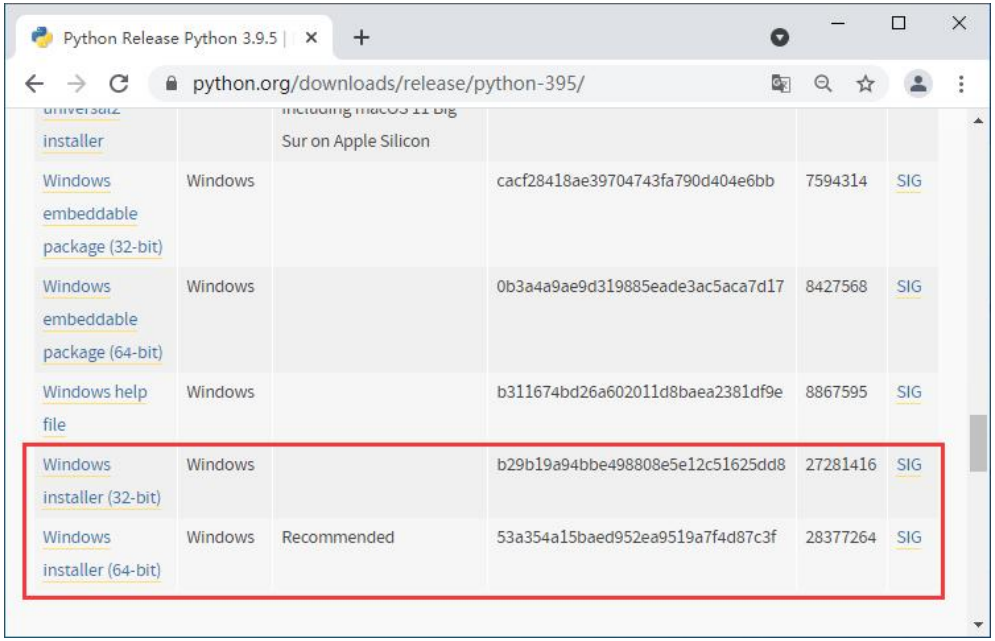
第一步：通过网址“https://www.python.org/”进入Python官方网站。



第二步：在菜单栏中，点击“Downloads”选项进入Python版本选择界面。

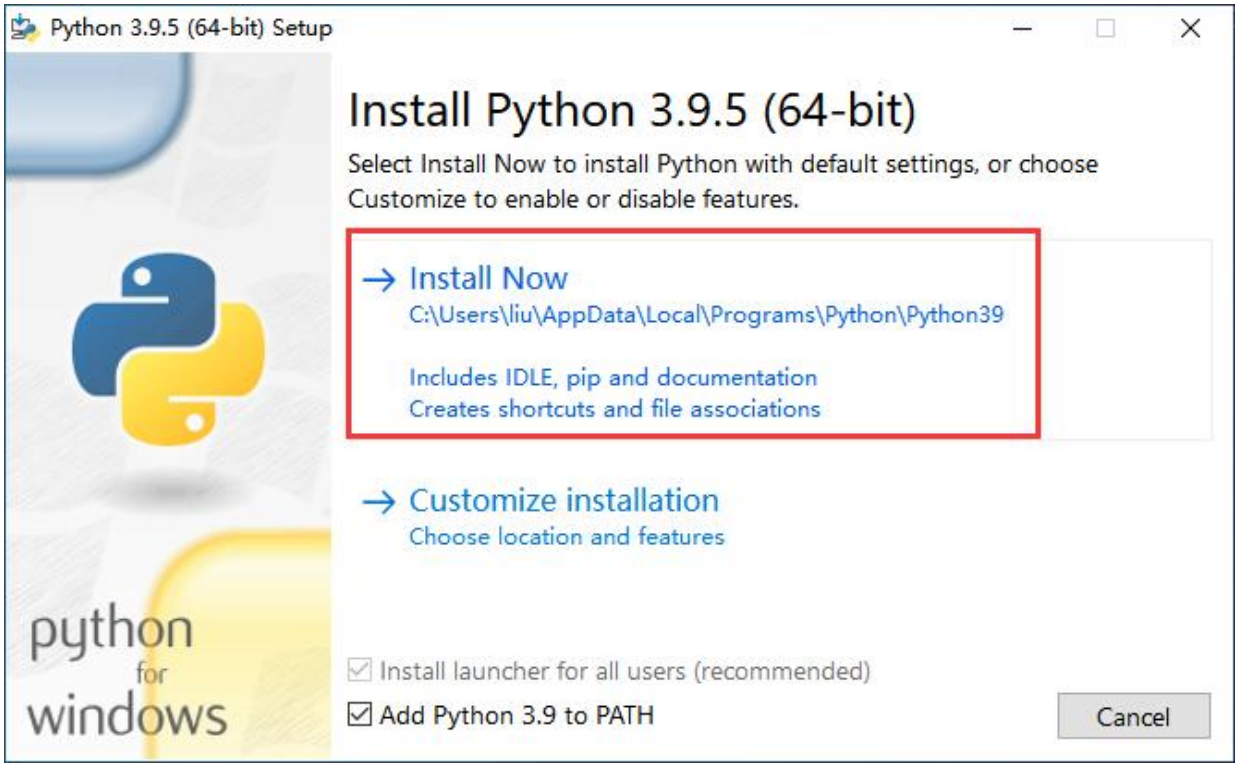


第三步：点击“Download Python 3.9.5”按钮进入Python安装文件下载界面，并滑动至页面底部。

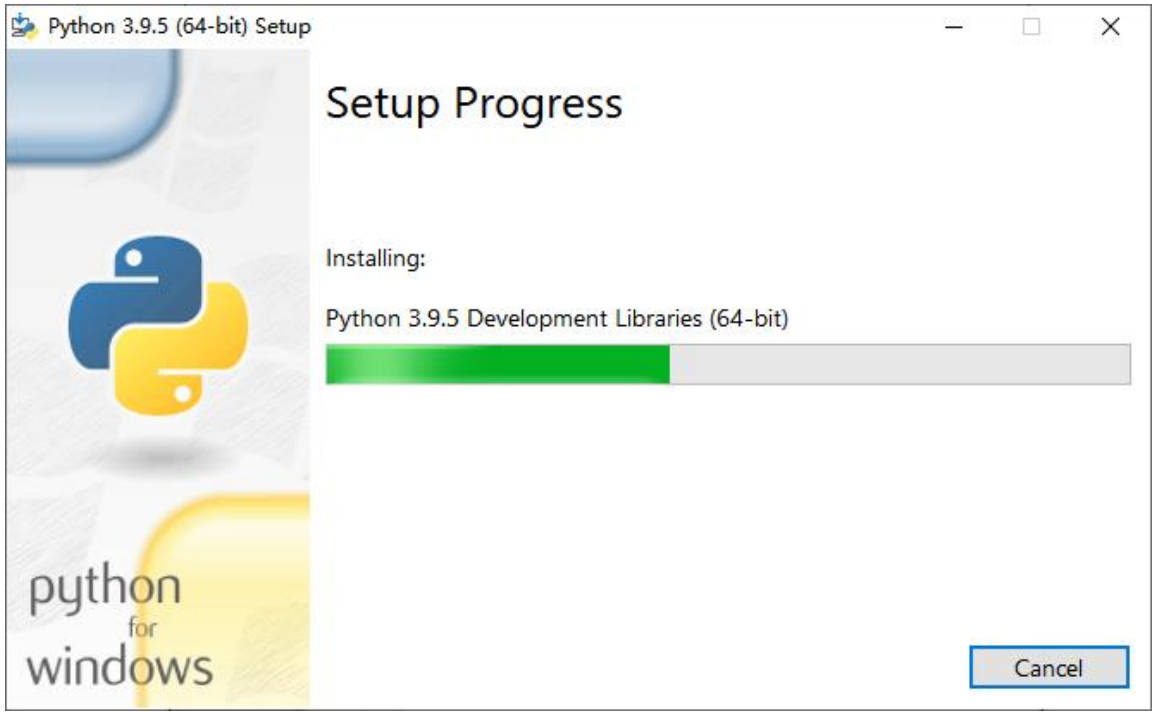


第四步：选择符合需求的Python安装文件，单击进行下载。

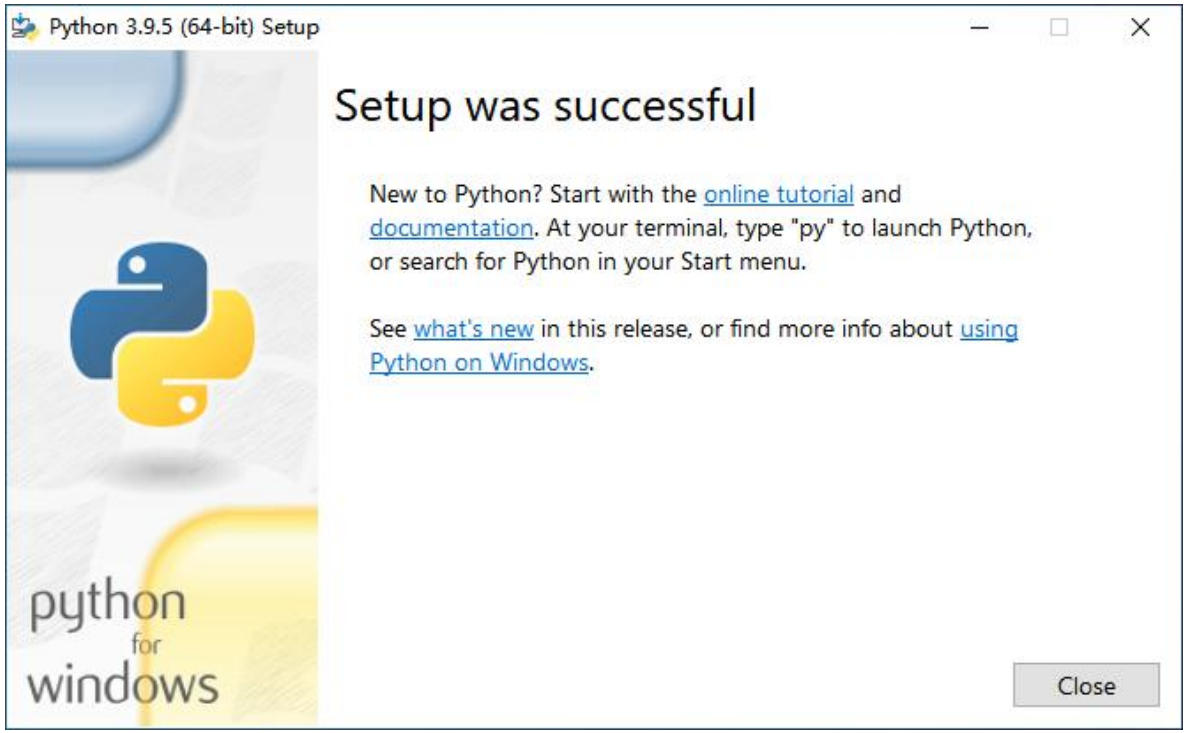
第五步：双击Python安装文件，进入安装设置界面。



第六步：选勾底部的“Add Python 3.9 to PATH”选项进行Python的环境配置，并选择“Install Now”立即安装进入Python安装界面（点击“Customize installation”将进入Python自定义安装界面，但自定义方式在安装完成后，还需对环境进行配置）



第七步：等待一段时间后，提示“Setup was successful”安装成功说明Python安装完成。

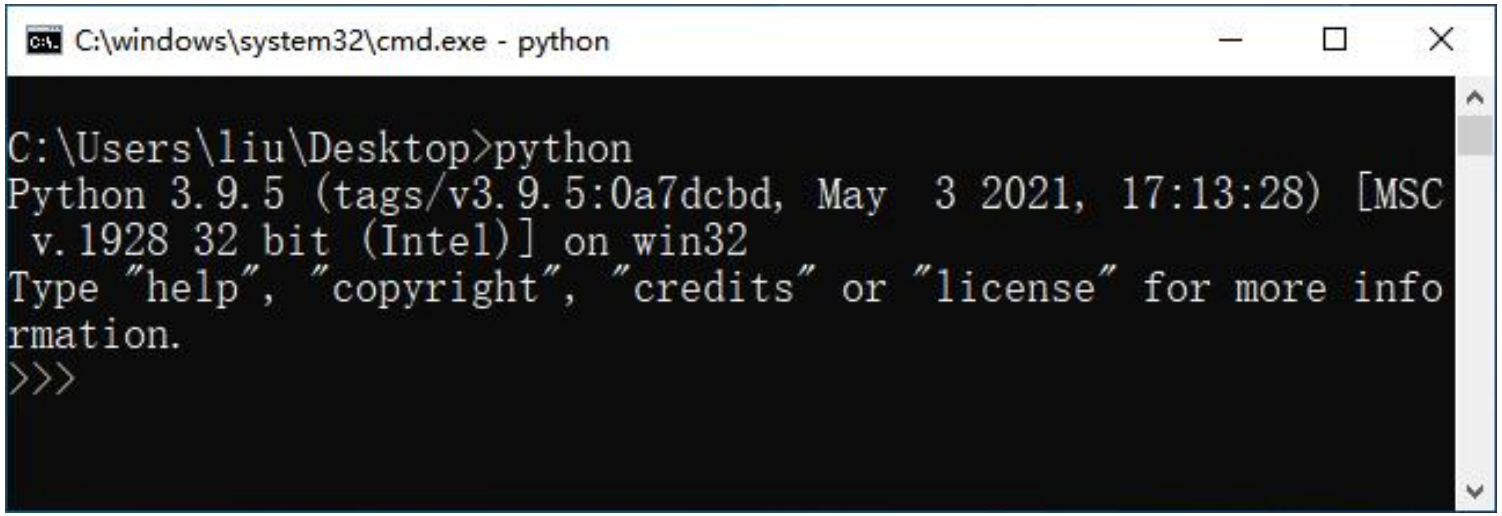


需要注意的是，如果选择自定义方式，在安装完成后，还需对环境进行配置。

第八步：打开电脑查找窗口，输入python进行验证，显示Python一系列图标，即可说明Python已在本机安装。



第九步：打开cmd命令窗口，输入python进行验证，或点击python3.x图标，如果进入Python的交互式命令行即可说明Python安装成功。



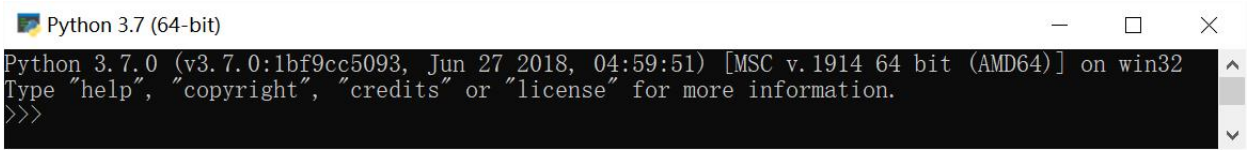
1.2 Python的安装

1.2.1 在Windows下安装Python

2. Python的安装







第1章 Python编程起步

- 常用第三方库

常用的Python内置库

名称	描述
re	正则表达式匹配
json	对JSON数据进行编解码
datetime	处理日期和时间
random	生成随机数
os	处理文件和目录
urllib	操作URL
zlib	数据打包和压缩
math	数学函数库
stat	权限操作

Python发展历程

名称	描述
Requests	HTTP库
Scrapy	爬虫工具常用的库
Matplotlib	绘制数据图的库
OpenCV	图片识别常用的库
SciPy	算法和数学工具库
BeautifulSoup/lxml	XML和HTML的解析库
NumPy	科学计算库
Pandas	数据统计、分析平台
Django	最流行的Python Web框架
Flask	轻量级Web应用程序框架
scikits-learn	机器学习库
PIL	图像处理库
jieba	中文分词工具
PyMySQL	MySQL数据库管理库
PyMongo	MongoDB数据库管理库

1.2 Python的安装

1.2.2 安装和管理Python扩展包

在Windows环境中，我们进入命令提示符窗口。

1. pip的安装与卸载

1.1 pip的安装

python中有许多有用的库，要使用这些第三方库，需要安装pip

打开cmd，输入以下命令安装pip：

```
python -m ensurepip --default-pip
```

出现如下图所示，代表pip安装成功：

```
C:\Users\Admin>python -m ensurepip --default-pip
Looking in links: c:\Users\Admin\AppData\Local\Temp\tmp7mxw5kow
Requirement already satisfied: setuptools in e:\environments\python310\lib\site-packages (63.2.0)
Processing c:\users\admin\appdata\local\temp\tmp7mxw5kow\pip-21.2.4-py3-none-any.whl
Installing collected packages: pip
Successfully installed pip-21.2.4
```

1.2 pip的卸载

1.卸载前询问一次，确认是否卸载

```
python -m pip uninstall pip
```

```
C:\Users\Admin>python -m pip uninstall pip
Found existing installation: pip 21.2.4
Uninstalling pip-21.2.4:
  Would remove:
    e:\environments\python310\lib\site-packages\pip-21.2.4.dist-info\*
    e:\environments\python310\lib\site-packages\pip\*
    e:\environments\python310\scripts\pip.exe
    e:\environments\python310\scripts\pip3.10.exe
    e:\environments\python310\scripts\pip3.exe
Proceed (Y/n)? Y
  Successfully uninstalled pip-21.2.4
```

2.直接卸载不询问

```
python -m pip uninstall -y pip
```

```
C:\Users\Admin>python -m pip uninstall -y pip
Found existing installation: pip 21.2.4
Uninstalling pip-21.2.4:
  Successfully uninstalled pip-21.2.4
```

2. pip的使用

2.1 帮助信息

帮助信息可以查看pip命令的用法，帮助命令如下：

`pip-h` 或 `pip <command> --help` ，其中 `<command>` 为可选信息，用法如下：

1.查看 `pip` 命令的用法：

```
pip -h
```

```
pip --help
```

```
C:\Users\Admin>pip -h

Usage:
  pip <command> [options]

Commands:
  install                     Install packages.
  download                    Download packages.
  uninstall                   Uninstall packages.
  freeze                      Output installed packages in requirements format.
  list                        List installed packages.
  show                        Show information about installed packages.
  check                       Verify installed packages have compatible dependencies.
  config                      Manage local and global configuration.
  search                      Search PyPI for packages.
  cache                       Inspect and manage pip's wheel cache.
  index                       Inspect information available from package indexes.
  wheel                       Build wheels from your requirements.
  hash                        Compute hashes of package archives.
  completion                  A helper command used for command completion.
  debug                       Show information useful for debugging.
  help                        Show help for commands.

General Options:
  -h, --help                  Show help.
  --debug                     Let unhandled exceptions propagate outside the main subroutine, instead of logging them
                              to stderr.
  --isolated                  Run pip in an isolated mode, ignoring environment variables and user configuration.
```

2.2 查看pip版本

```
pip --version
```

```
C:\Users\Admin>pip --version
pip 21.2.4 from E:\Environments\Python310\lib\site-packages\pip (python 3.10)
```

2.3 更新pip版本

```
python -m pip install --upgrade pip
```

```
C:\Users\Admin>python -m pip install --upgrade pip
Looking in indexes: http://mirrors.aliyun.com/pypi/simple/
Requirement already satisfied: pip in e:\environments\python310\lib\site-packages (21.2.4)
Collecting pip
  Downloading http://mirrors.aliyun.com/pypi/packages/96/2f/caec18213f6a67852f6997fb0673ae08d2e93d1b81573edb93ba4ef06970
/pip-22.1.2-py3-none-any.whl (2.1 MB)
     |████████████████████████████████| 2.1 MB 285 kB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 21.2.4
    Uninstalling pip-21.2.4:
      Successfully uninstalled pip-21.2.4
Successfully installed pip-22.1.2
```

2.4 安装库

2.4.1 安装单个库

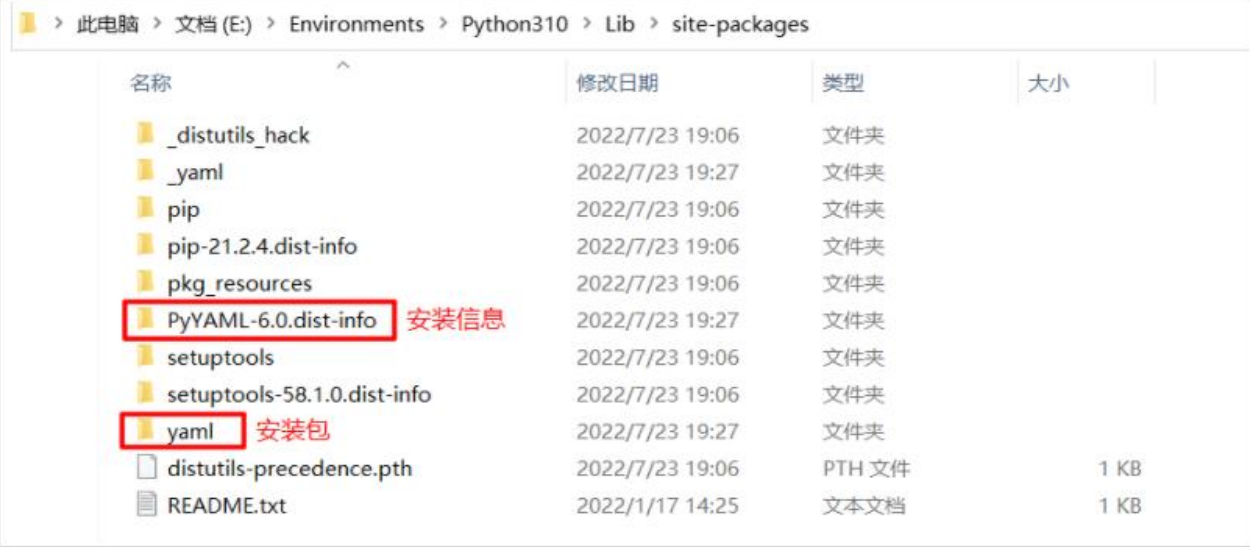
例如安装pyyaml库，打开cmd输入以下命令：

```
pip install pyyaml
```

如下图所示，代表安装成功：

```
C:\Users\Admin>pip install pyyaml
Collecting pyyaml
  Downloading PyYAML-6.0-cp310-cp310-win_amd64.whl (151 kB)
     ---------------------------------------- 151 kB 198 kB/s
Installing collected packages: pyyaml
Successfully installed pyyaml-6.0
WARNING: You are using pip version 21.2.4; however, version 22.2 is available.
You should consider upgrading via the 'E:\Environments\Python310\python.exe -m pip install --upgrade pip' command.
```

安装包会默认安装到python路径的Lib\site-packages文件夹下，如 `E:\Environments\Python310\Lib\site-packages`



2.4.2 批量安装库

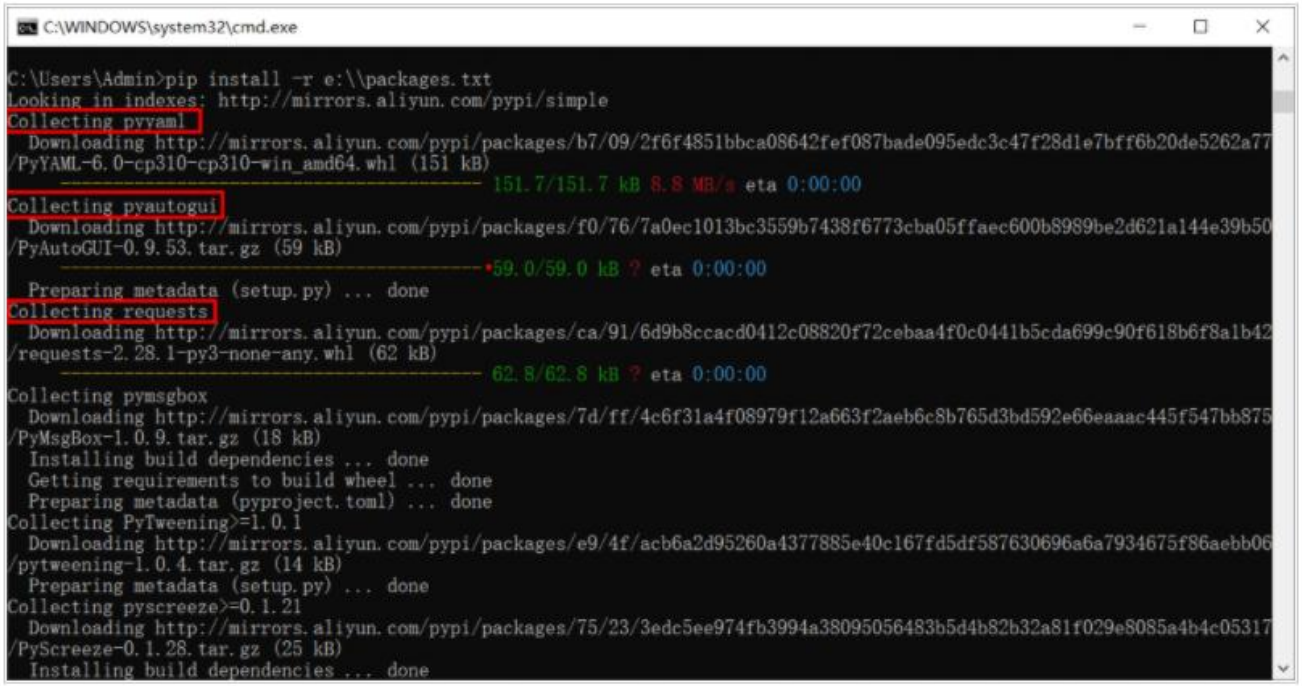
新建一个txt文件，在文件里添加库名的信息，再输入 `pip install -r 文件路径` ，即可批量安装这些库。

例如，在E盘下新建 `packages.txt` ，在txt文件中添加以下内容：

```
pyyaml
pyautogui
requests
```

再输入命令：

```
pip install -r e:\\packages.txt
```



2.5 查看库

2.5.1 查看所有已安装的库

1.查看所有已安装的库

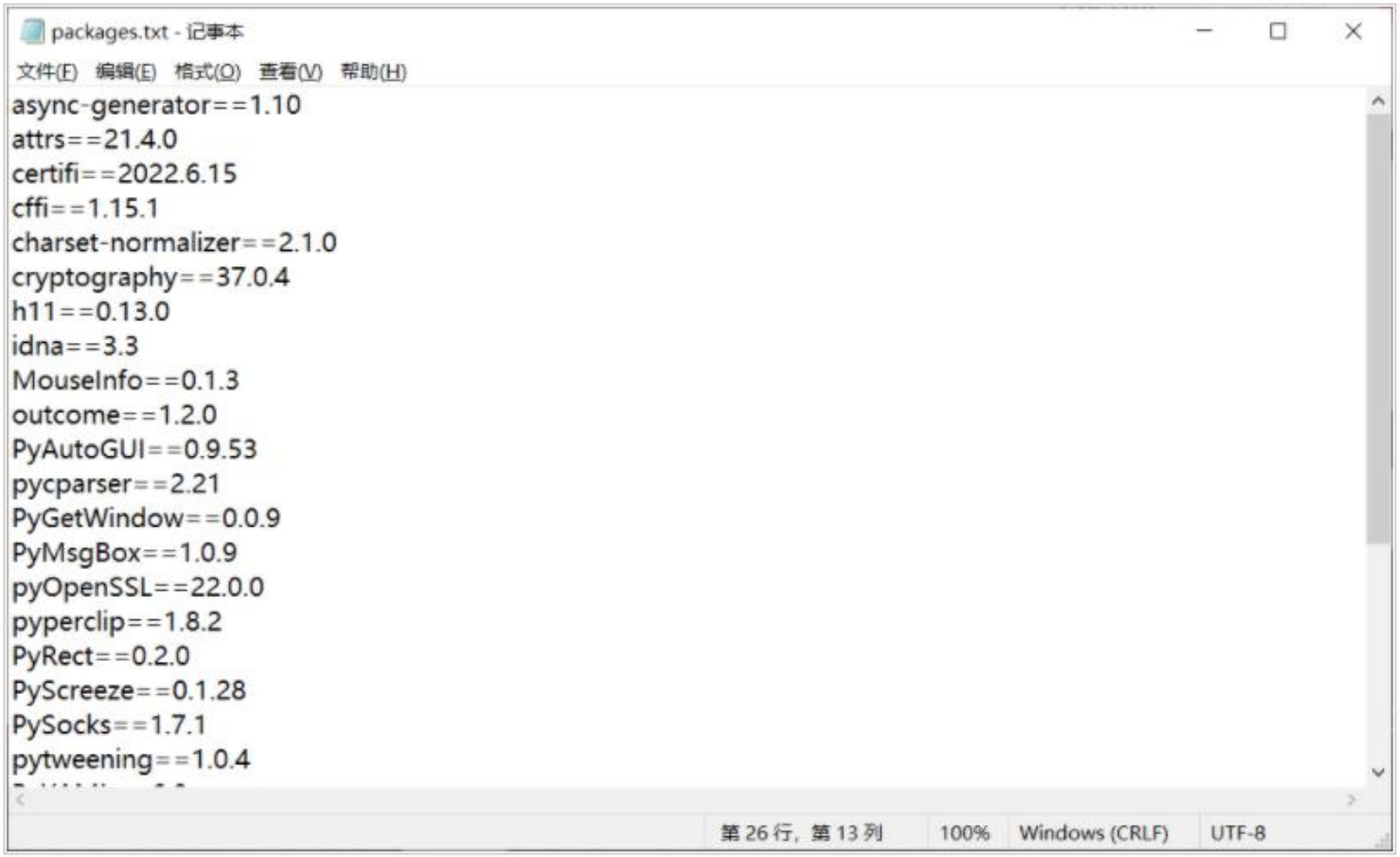
```
pip list
```

```
C:\Users\Admin>pip list
Package            Version
------------------ ---------
async-generator    1.10
attrs              21.4.0
certifi            2022.6.15
cffi               1.15.1
charset-normalizer 2.1.0
cryptography       37.0.4
h11                0.13.0
idna               3.3
MouseInfo          0.1.3
outcome            1.2.0
pip                22.1.2
PyAutoGUI          0.9.53
pycparser          2.21
PyGetWindow        0.0.9
PyMsgBox           1.0.9
pyOpenSSL          22.0.0
pyperclip          1.8.2
PyRect             0.2.0
PyScreeze          0.1.28
PySocks            1.7.1
pytweening         1.0.4
PyYAML             6.0
requests           2.28.1
selenium           4.3.0
setuptools         58.1.0
sniffio            1.2.0
sortedcontainers   2.4.0
```

2.5.2 输出已安装库的信息至文件

输出所有已安装的库的信息，到文件中

```
1 | pip freeze > E:\packages.txt
```



2.5.3 查看库的详细信息

```
1 pip show -f pyyaml
```

```
C:\Users\Admin>pip show -f pyyaml
Name: PyYAML
Version: 6.0
Summary: YAML parser and emitter for Python
Home-page: https://pyyaml.org/
Author: Kirill Simonov
Author-email: xi@resolvent.net
License: MIT
Location: e:\environments\python310\lib\site-packages
Requires:
Required-by:
Files:
  PyYAML-6.0.dist-info\INSTALLER
  PyYAML-6.0.dist-info\LICENSE
  PyYAML-6.0.dist-info\METADATA
  PyYAML-6.0.dist-info\RECORD
  PyYAML-6.0.dist-info\REQUESTED
  PyYAML-6.0.dist-info\WHEEL
  PyYAML-6.0.dist-info\top_level.txt
  _yaml\__init__.py
  _yaml\__pycache__\__init__.cpython-310.pyc
  yaml\__init__.py
  yaml\__pycache__\__init__.cpython-310.pyc
  yaml\__pycache__\composer.cpython-310.pyc
  yaml\__pycache__\constructor.cpython-310.pyc
  yaml\__pycache__\cyaml.cpython-310.pyc
  yaml\__pycache__\dumper.cpython-310.pyc
  yaml\__pycache__\emitter.cpython-310.pyc
```

2.6 升级库

1.查看需要升级的库

```
pip list -o
```

```
C:\Users\Admin>pip list -o
Package    Version Latest Type
---------- ------- ------ -----
setuptools 58.1.0  63.2.0 wheel

[notice] A new release of pip available: 22.1.2 -> 22.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

2.升级库

格式如下:

`pip install --upgrade` 库名或者 `pip install -U` 库名

例如:

```
pip install --upgrade setuptools
```

```
C:\Users\Admin>pip install --upgrade setuptools
Looking in indexes: http://mirrors.aliyun.com/pypi/simple/
Requirement already satisfied: setuptools in e:\environments\python310\lib\site-packages (58.1.0)
Collecting setuptools
  Using cached http://mirrors.aliyun.com/pypi/packages/a4/53/bfc6409447ca024558b8b19d055de94c813c3e32c0296c48a0873a161cf
5/setuptools-63.2.0-py3-none-any.whl (1.2 MB)
Installing collected packages: setuptools
  Attempting uninstall: setuptools
    Found existing installation: setuptools 58.1.0
    Uninstalling setuptools-58.1.0:
      Successfully uninstalled setuptools-58.1.0
Successfully installed setuptools-63.2.0

[notice] A new release of pip available: 22.1.2 -> 22.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

2.7 卸载库

2.7.1 卸载单个库

1.删除库前询问一次，确认是否删除

例如卸载pyyaml库，打开cmd输入以下命令：

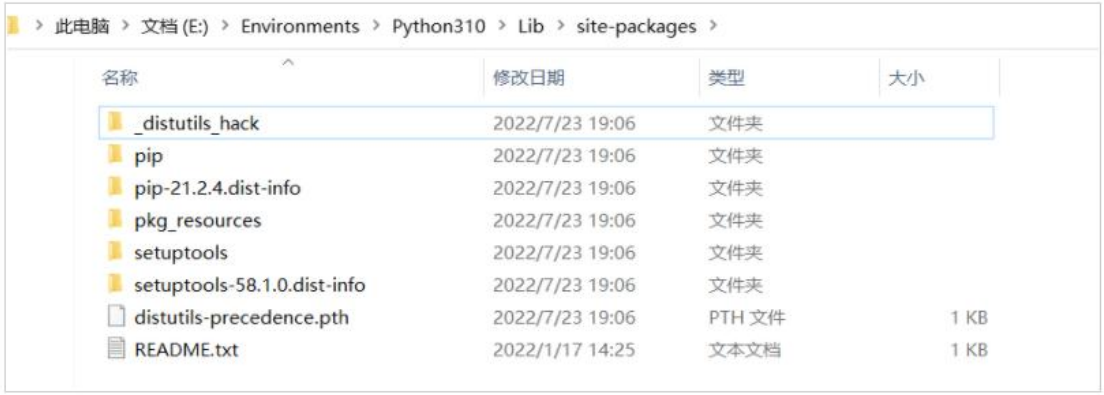
```
pip uninstall pyyaml
```

如下图所示，会出现询问是否删除包，输入Y，即删除成功。

```
C:\Users\Admin>pip uninstall pyyaml
Found existing installation: PyYAML 6.0
Uninstalling PyYAML-6.0:
  Would remove:
    e:\environments\python310\lib\site-packages\_yaml\*
    e:\environments\python310\lib\site-packages\pyyaml-6.0.dist-info\*
    e:\environments\python310\lib\site-packages\yaml\*
Proceed (Y/n)? Y
  Successfully uninstalled PyYAML-6.0

C:\Users\Admin>
```

再查看Lib\site-packages文件夹下，发现包已经删除了：



2.跳过询问，直接删除

```
pip uninstall -y pyyaml
```

```
C:\Users\Admin>pip uninstall -y pyyaml
Found existing installation: PyYAML 6.0
Uninstalling PyYAML-6.0:
  Successfully uninstalled PyYAML-6.0
```

2.7.2 批量卸载库

批量卸载库方式，同批量安装库方式。

例如，在E盘下新建 `packages.txt` ，在txt文件中添加以下内容：

```
pyyaml
pyautogui
requests
```

再输入命令：

```
pip uninstall -y -r e:\\packages.txt
```

```
C:\Users\Admin>pip uninstall -y -r e:\\packages.txt
Found existing installation: PyYAML 6.0
Uninstalling PyYAML-6.0:
  Successfully uninstalled PyYAML-6.0
Found existing installation: PyAutoGUI 0.9.53
Uninstalling PyAutoGUI-0.9.53:
  Successfully uninstalled PyAutoGUI-0.9.53
Found existing installation: requests 2.28.1
Uninstalling requests-2.28.1:
  Successfully uninstalled requests-2.28.1
```

1.2 Python的安装

1.2.2 安装和管理Python扩展包

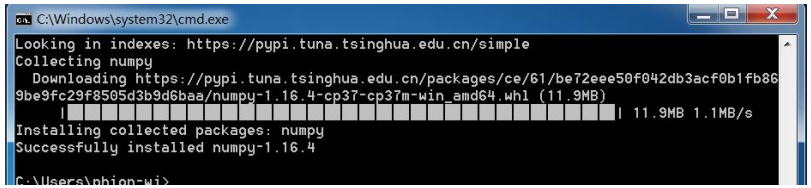
3. pip工具的使用

```
install         Install packages.
download        Download packages.
uninstall       Uninstall packages.
freeze          Output installed packages in requirements format.
list            List installed packages.
show            Show information about installed packages.
check           Verify installed packages have compatible dependencies.
config          Manage local and global configuration.
search          Search PyPI for packages.
wheel           Build wheels from your requirements.
hash            Compute hashes of package archives.
completion      A helper command used for command completion.
debug           Show information useful for debugging.
help            Show help for commands.
```

1.2 Python的安装

1.2.2 安装和管理Python扩展包

4. pip工具的使用范例

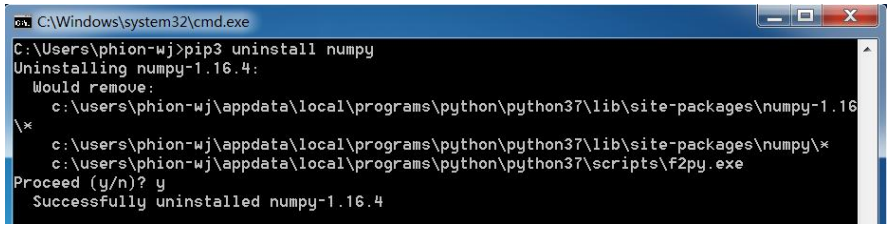


（1）安装一个扩展包：

pip3 install 扩展包名称及版本

（2）卸载一个扩展包：

pip3 uninstall 扩展包名称及版本



1.3 Python的开发环境

1、IDLE

2、Pycharm

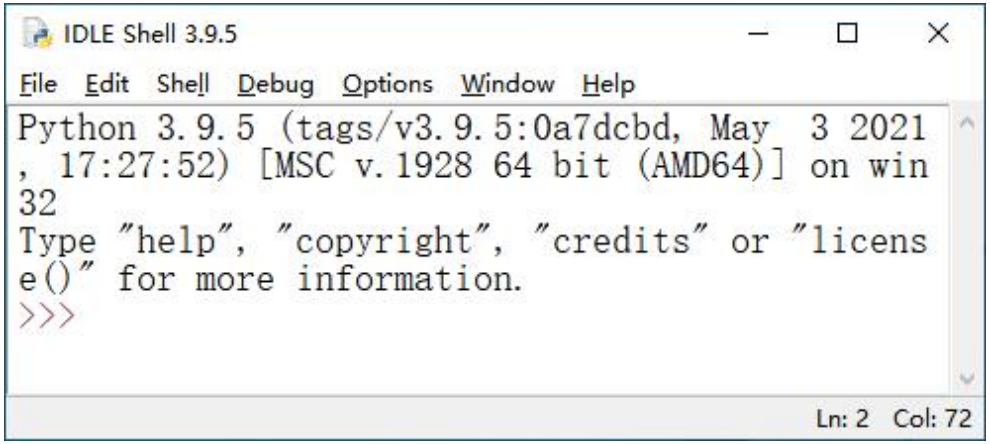
3、其他第三方开发工具

1、IDLE

集成开发环境（IDE，Integrated Development Environment ）是用于提供程序开发环境的应用程序，一般包括代码编辑器、编译器、调试器和图形用户界面等工具。

IDLE，全称为“Integrated Development and Learning Environment”，是Python的一个轻量级集成开发环境（shell编程），具有语法标记明显、段落缩进整齐、文本编辑方便、TABLE键控制和调试程序便捷等特点。另外，IDLE还是Python安装的一个可选部分，能够很方便进行Python的非商业开发，简单来说，IDLE在安装Python时会自动安装。



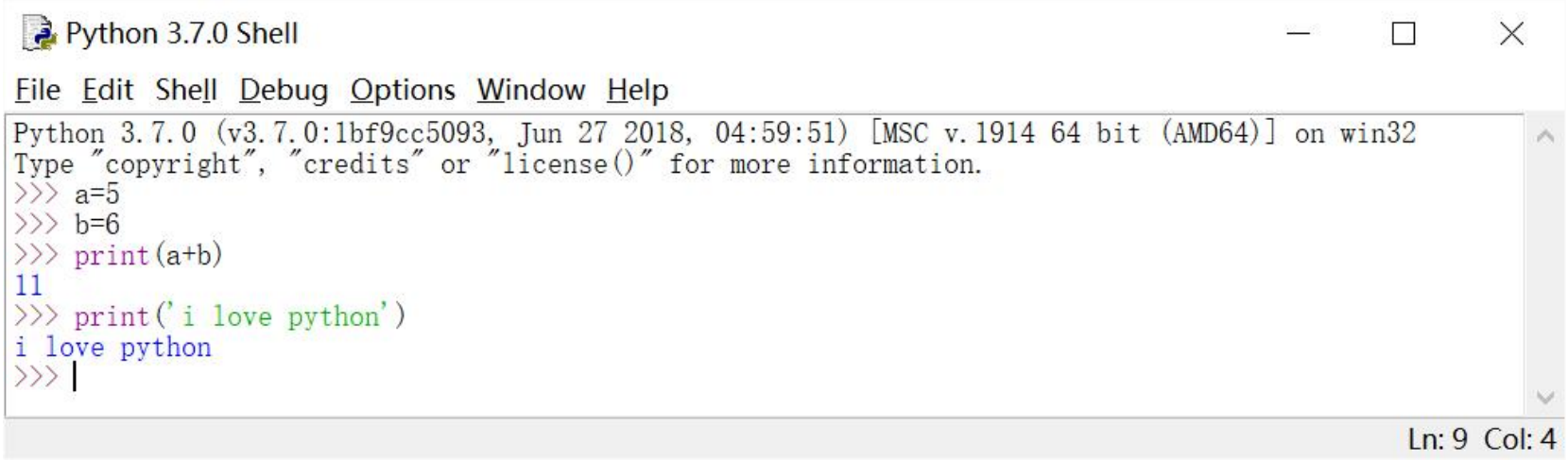


1、IDLE

IDLE

Python3.X 默认安装后，会在开始菜单中增加一个“IDLE”程序项。

IDLE的基本功能包括程序语法加亮、段落缩进、基本文本编辑、TABLE键控制和调试程序等



1、IDLE

另外，IDLE还提供了多个用于进行操作的快捷键，如撤销、复制、粘贴等，常用的快捷键。

快捷键	描述
Ctrl+z	撤销一步操作
Ctrl+Shift+z	恢复上—次的撤销操作
Ctrl+]	缩进代码块
Ctrl+[	取消代码块缩进
Ctrl+n	新建文件
F5	运行文件
Ctrl+c	复制
Ctrl+v	粘贴
Ctrl+s	保存
Alt+P	查看上一条命令
Alt+N	查看下一条命令
Ctrl+3	注释代码块
Ctrl+4	取消代码块注释
Alt+g	转到某一行
Ctrl+F6	重新启动 Python Shell

1.3 Python的开发环境

1.3.1 IDLE

2. 利用IDLE创建Python程序

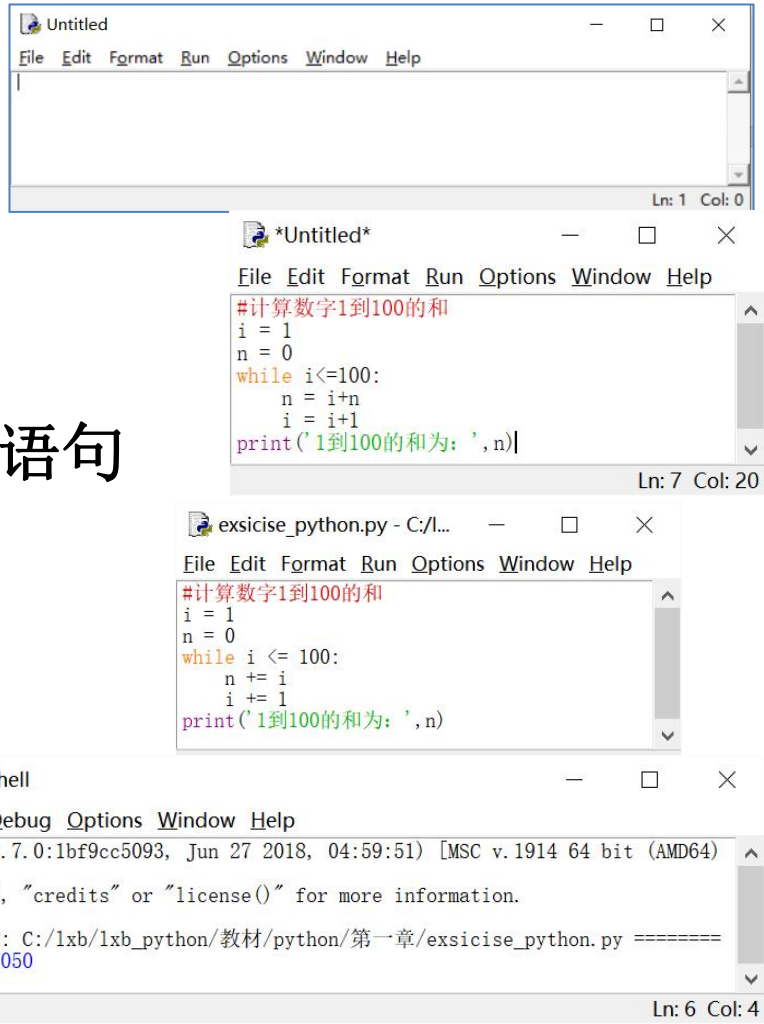
（1）新建Python程序文件

（2）在程序文件中输入程序语句

（3）保存程序文件

（4）运行程序

（5）退出IDLE环境



2、Pycharm

1.3 Python的开发环境

2 PyCharm

Pycharm简介

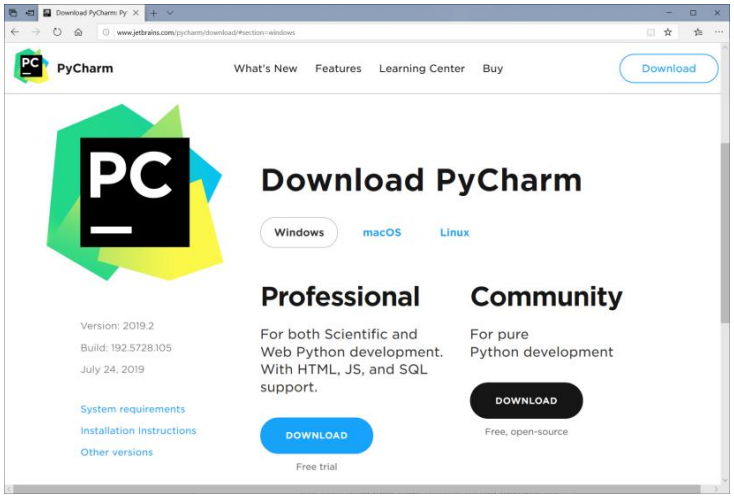
PyCharm是JetBrains开发的一款Python集成开发环境，具备了程序开发所需的多种功能，如调试、语法高亮、Project管理、代码跳转、智能提示、自动完成、单元测试、版本控制等，是开发Python最常用的一款工具。

- 编码协助
- 项目代码导航
- 代码分析
- Python重构
- 支持Django
- 集成版本控制
- 图形页面调试器
- 集成的单元测试
- 可自定义&可扩展

1.3 Python的开发环境

2 PyCharm

(1).PyCharm的下载



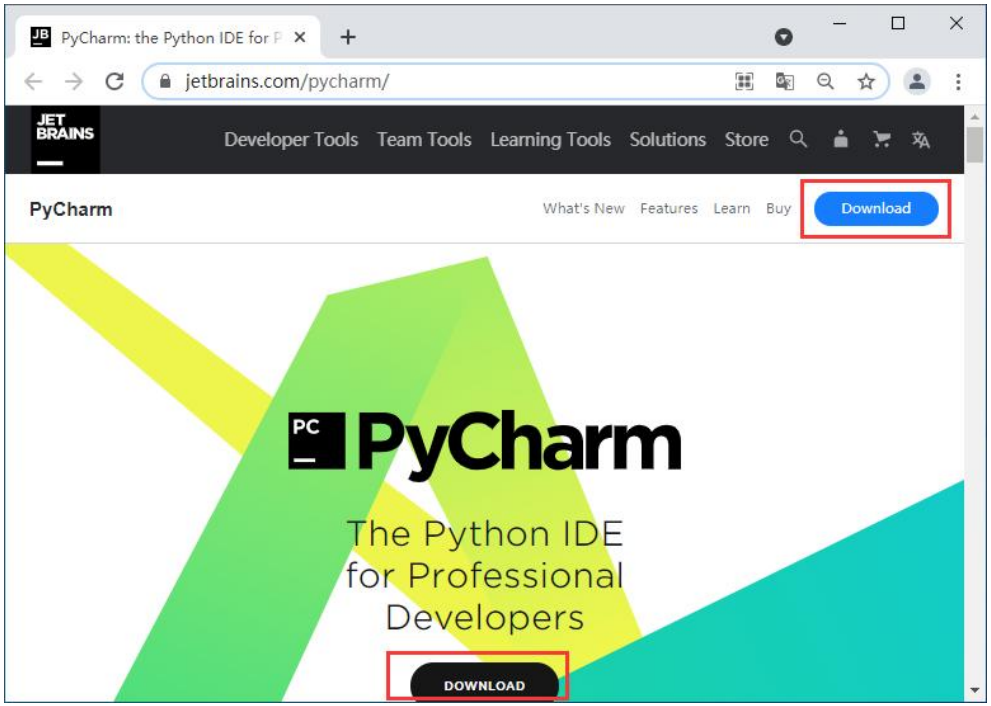
(2). 安装PyCharm community版



2、Pycharm

Pycharm安装

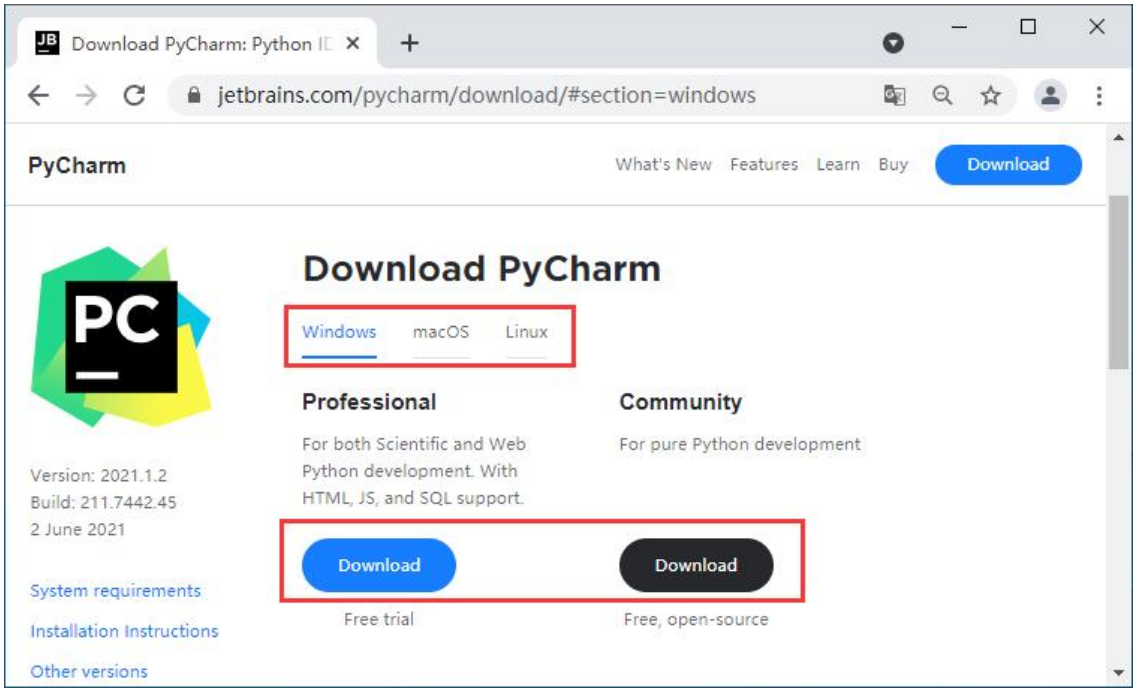
第一步：通过路径“https://www.jetbrains.com/pycharm/”进入PyCharm的官网。



2、Pycharm概述

Pycharm安装

第二步：点击图第一步中的“Download”或“DOWNLOAD”按钮进入PyCharm安装文件下载界面。

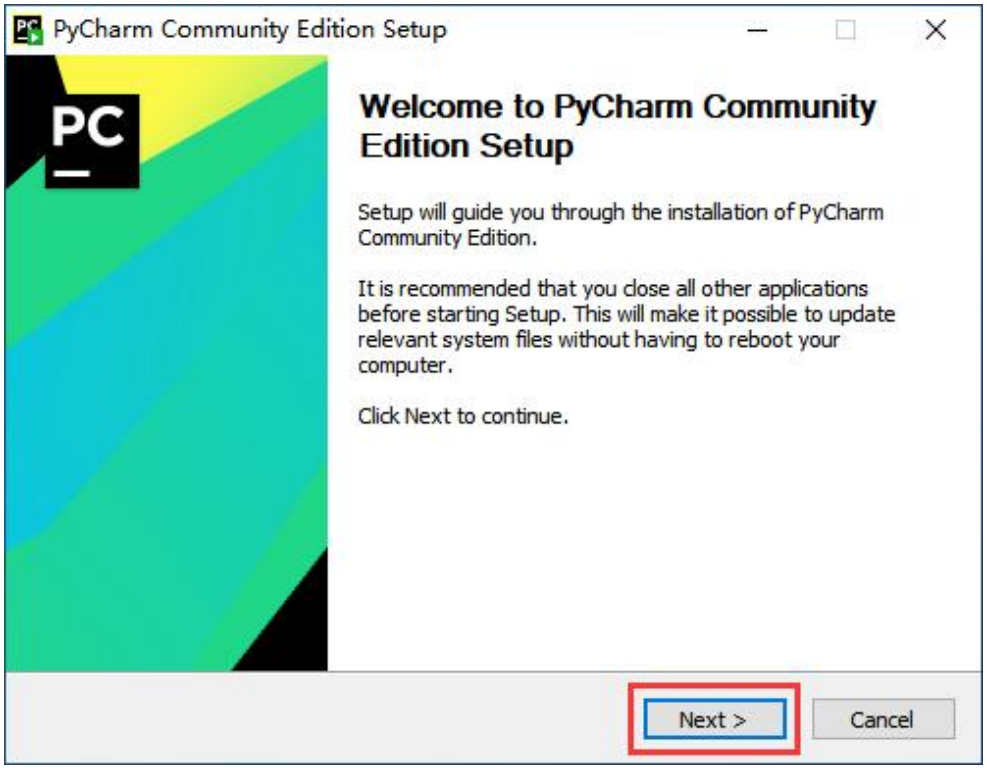


第三步：点击“Download”按钮进行PyCharm安装文件下载。其中，Professional为专业版，Community为社区版。

2、Pycharm概述

Pycharm安装

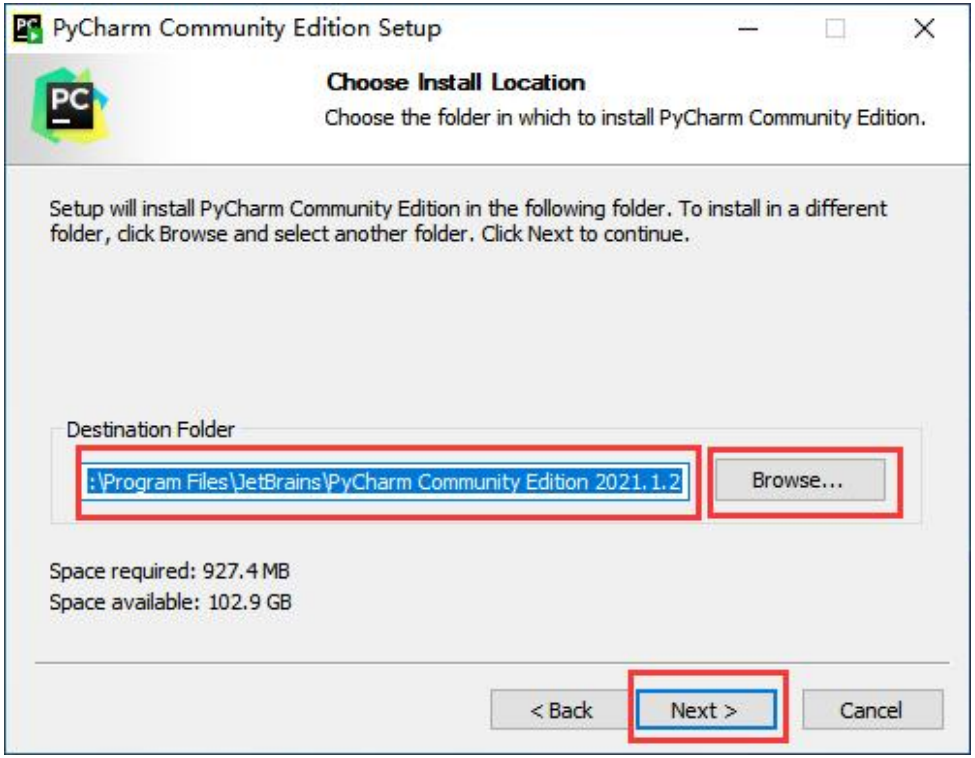
第四步：双击刚刚下载的安装文件，进入PyCharm安装首界面。



2、Pycharm概述

Pycharm安装

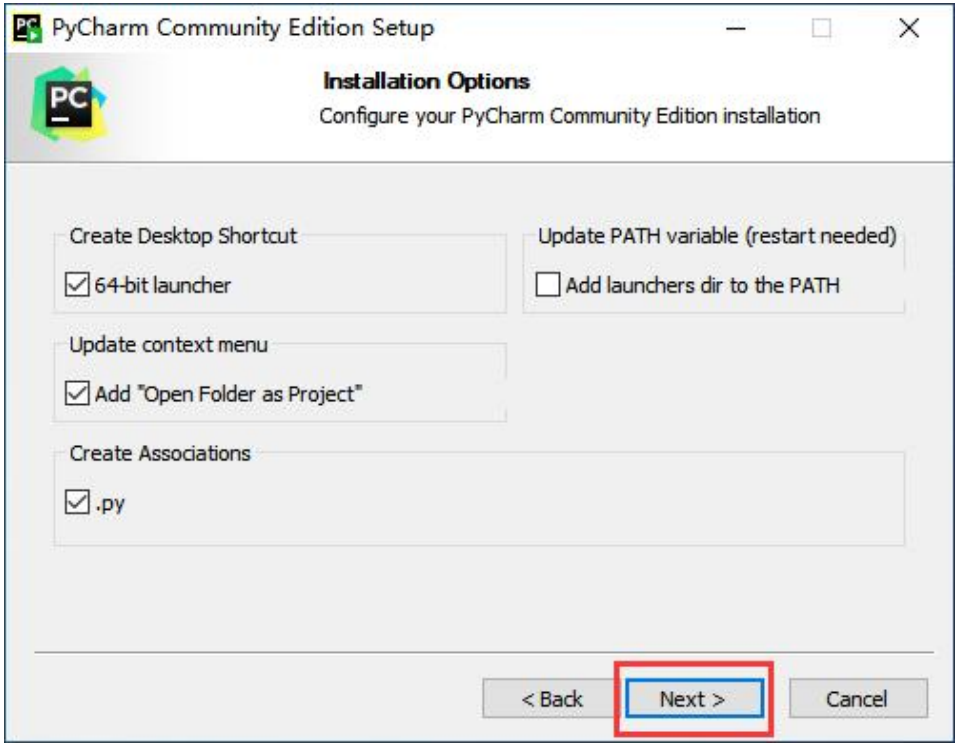
第五步：单击图中的“Next >”按钮进入安装路径选择界面。



2、Pycharm概述

Pycharm安装

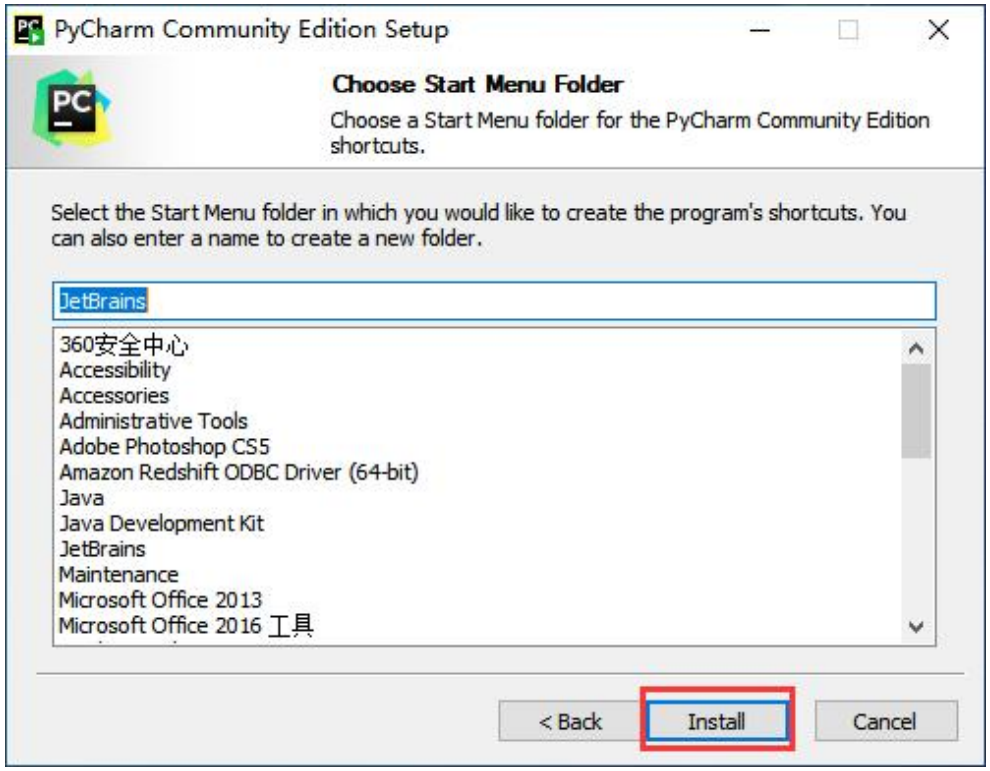
第六步：单击图中的“Browse...”按钮选择安装路径后，单击“Next >”按钮进入安装选择界面。



2、Pycharm概述

Pycharm安装

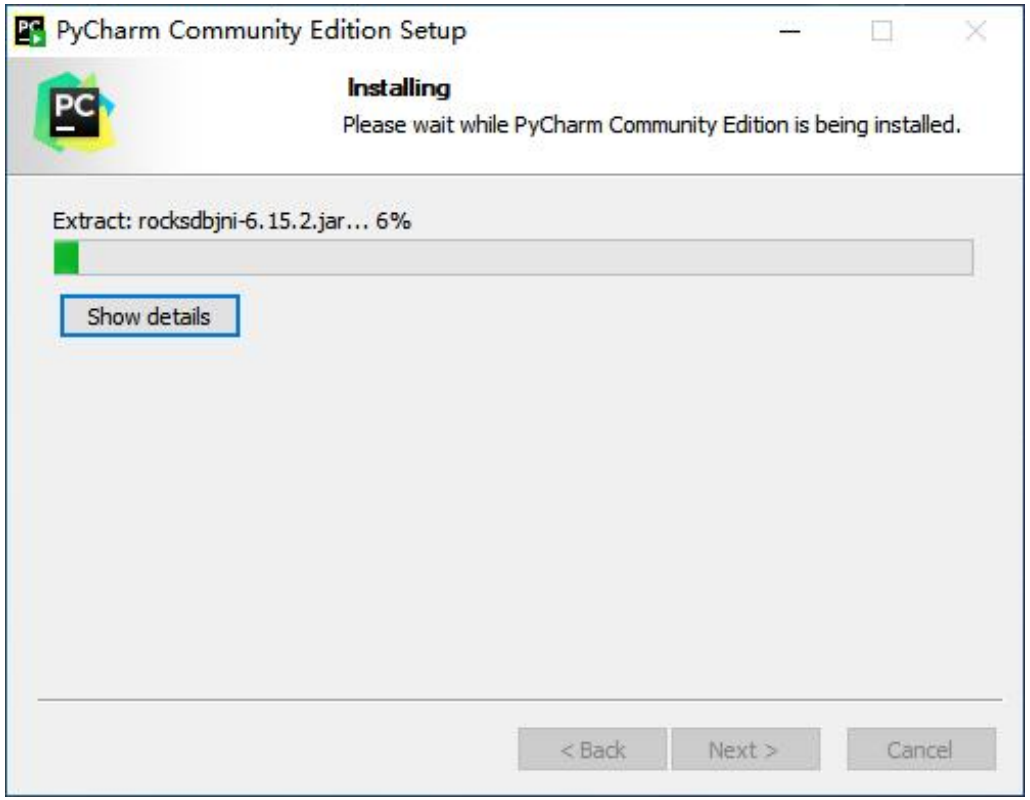
第七步：单击图中的“Next >”按钮进入安装界面。



2、Pycharm概述

Pycharm安装

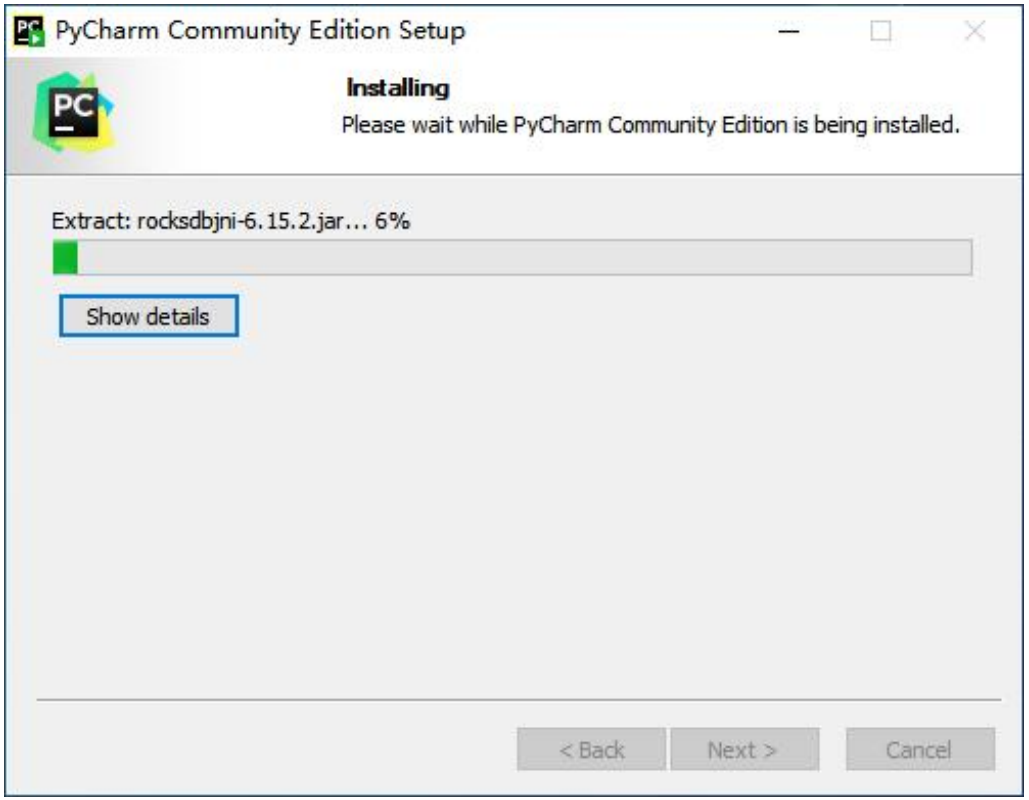
第八步：单击图中的“Install”按钮进入PyCharm的安装。



2、Pycharm概述

Pycharm安装

第八步：单击图中的“Install”按钮进入PyCharm的安装。



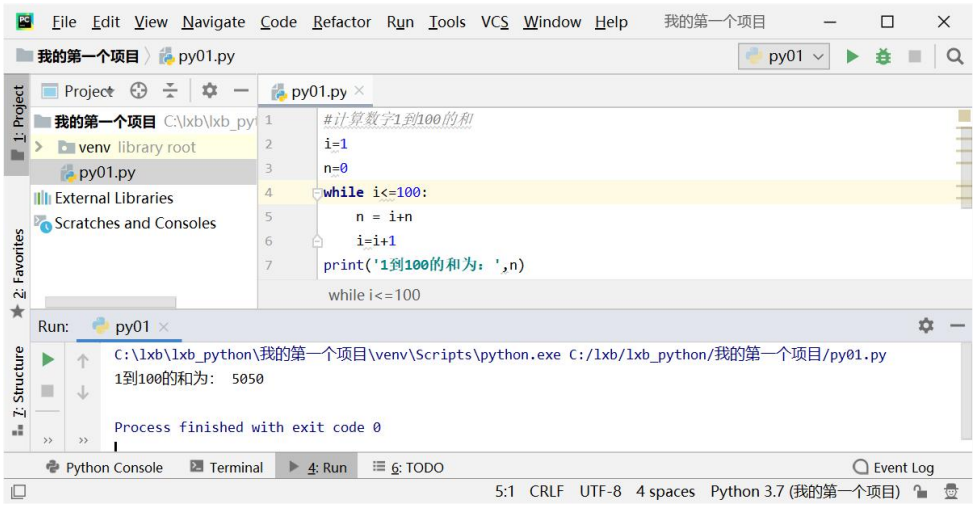
第九步：安装完成后，点击“Finish”按钮完成PyCharm安装。

1.3 Python的开发环境

1.3.2 PyCharm

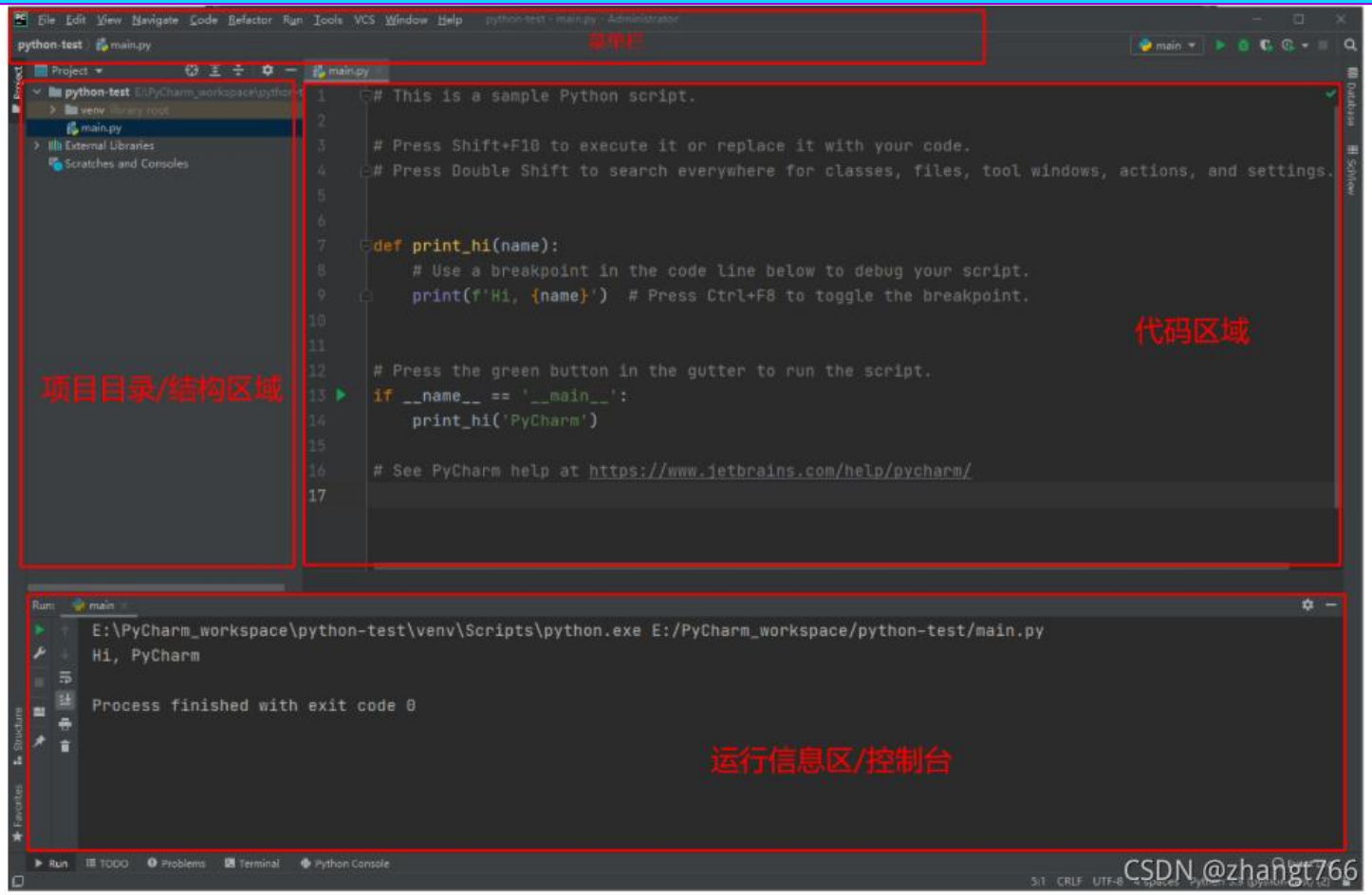
3. 运行PyCharm编辑器编辑Python程序

（1）导入配置项
（2）软件许可协议
（3）是否共享数据
（4）设置程序主题
（5）创建一个新项目
（6）输入项目名称、保存路径
（7）tip of day 窗口
（8）创建新的Python程序文件
（9）文件创建完毕
（10）运行编辑好的程序文件

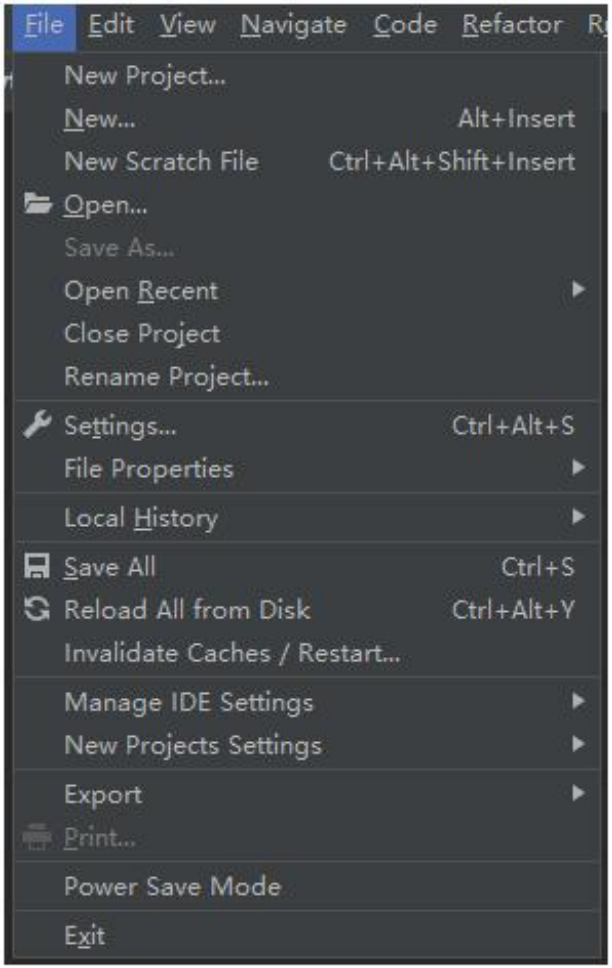


PyCharm 界面介绍

从整体界面来看PyCharm分为 菜单栏区域 / 项目结构区域 / 代码区域 / 运行信息区



一、菜单栏
1、File（文件）



`New Project`：创建新的项目

`New ...`：新建一些中间件配置，如MySQL、MongoDB、DDL等以及相关驱动

`New Scratch File`：划痕文档，也称为临时文件，可以**创建各种类型**的文件进行临时处理，在里面“打草稿”，可运行并且可调试（非常棒的一个功能，在最近的版本才有的）

`Open`：打开项目目录

`Save as`：另存为

`Close Project`：关闭项目并回答创建项目页面

`Rename Project`：给项目重命名

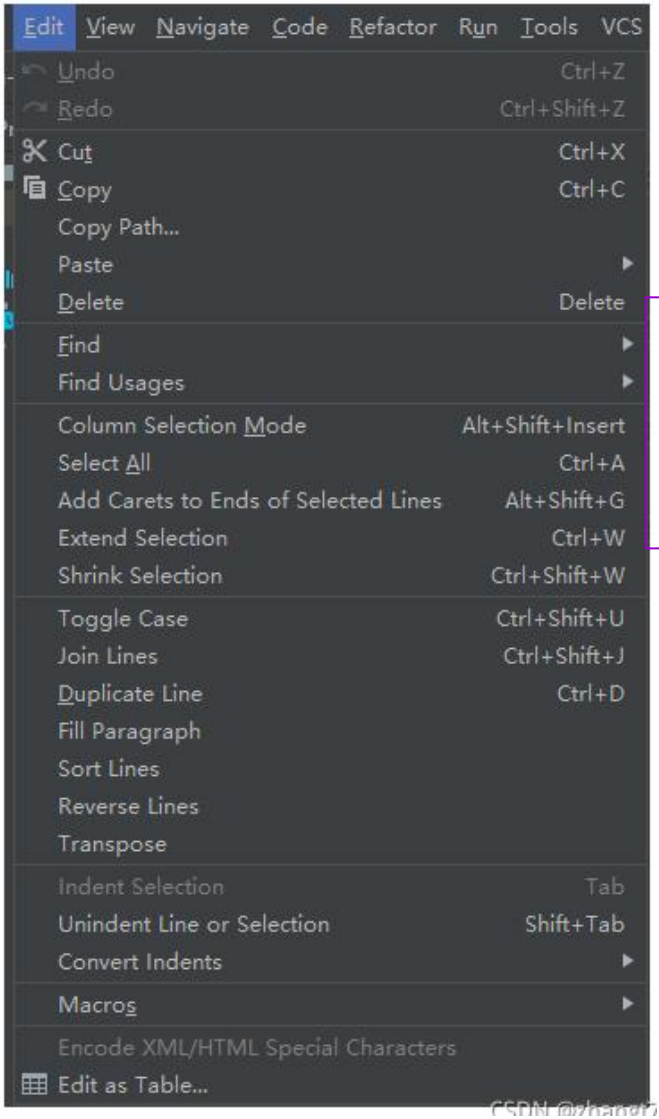
`Settings`：设置选项，重点☆☆☆☆☆

`File Properties`：文件的相关属性，包括编码等

`Invalidate Caches /Restart..`：是缓存失效，并重启

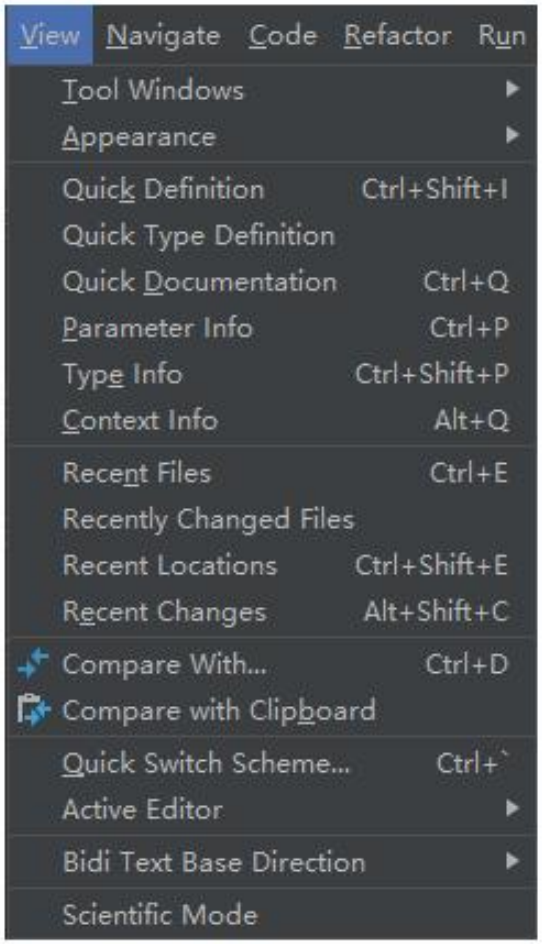
CSDN @zhangt766

2、Edit（编辑）



Find： 编辑窗口中用的最多的就是Find选项中的，例如Ctrl + F 文件内查找，Ctrl +Shift + F 项目中搜索

3、View（视图）



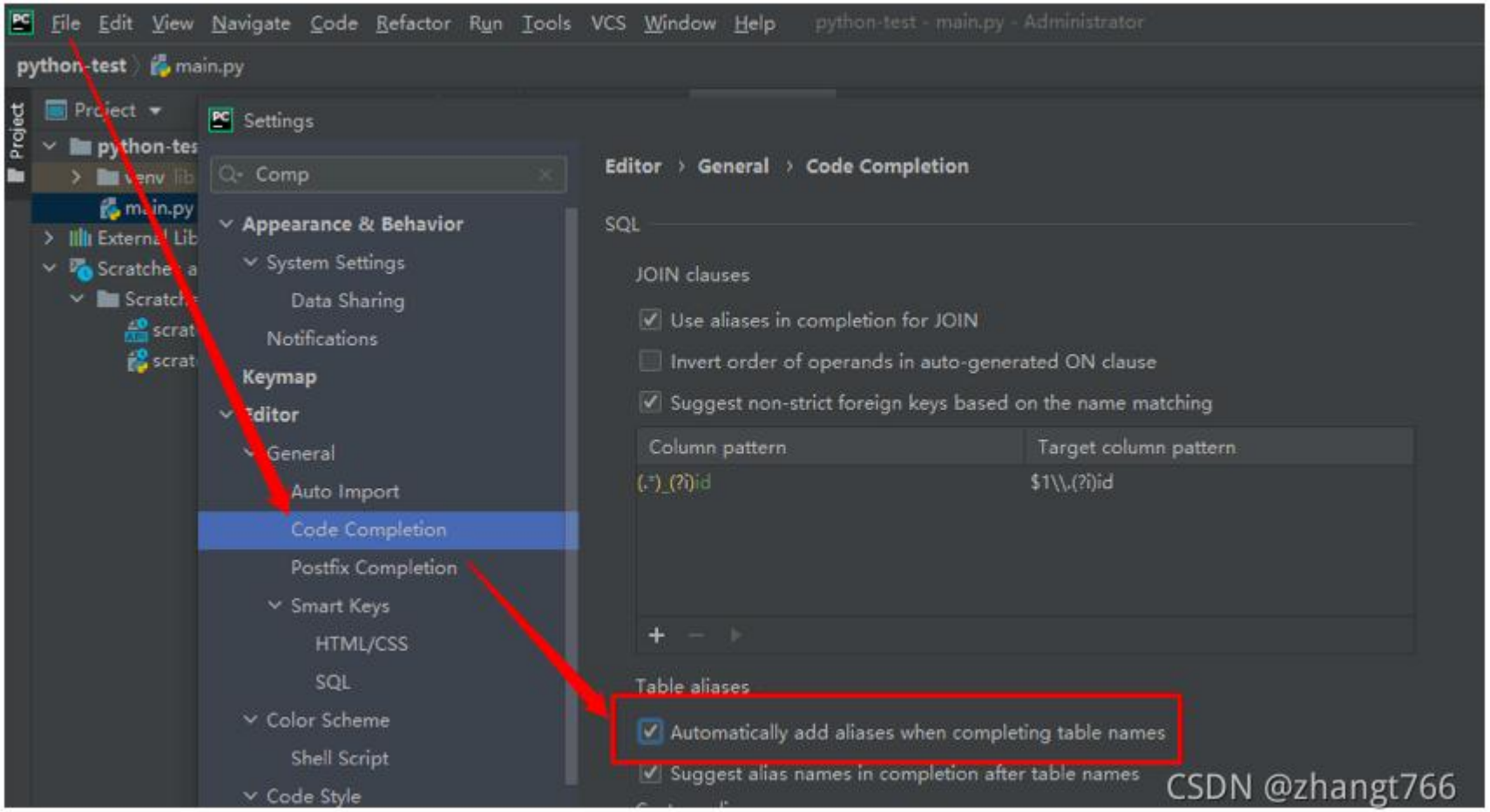
- Tool Windows： 工具窗口，如果主页面中某些窗口不小心关了，可以在这里面重新找到。
- Appearance：外观设置

4、Code（编码）

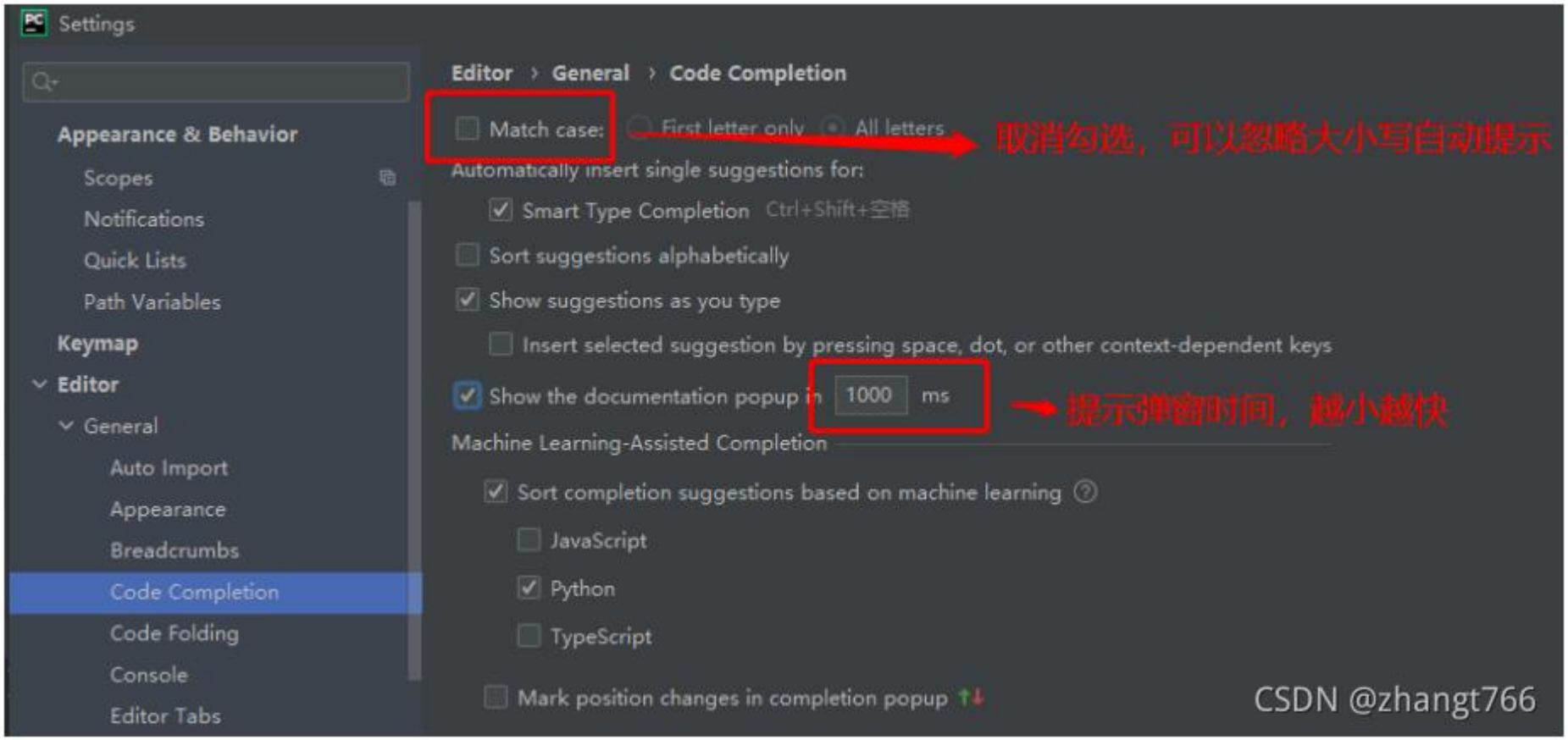

Code Refactor Run Tools VCS Window Help pyth
Override Methods... Ctrl+O
Implement Methods... Ctrl+I
Generate... Alt+Insert
Code Completion
Insert Live Template... Ctrl+J
Surround With... Ctrl+Alt+T
Unwrap/Remove... Ctrl+Shift+Delete
Folding
Comment with Line Comment Ctrl+/
Comment with Block Comment Ctrl+Shift+/
Reformat Code Ctrl+Alt+L
Reformat File... Ctrl+Alt+Shift+L
Auto-Indent Lines Ctrl+Alt+I
Optimize Imports Ctrl+Alt+O
Rearrange Code
Move Statement Down Ctrl+Shift+向下箭头
Move Statement Up Ctrl+Shift+向上箭头
Move Element Left Ctrl+Alt+Shift+向左箭头
Move Element Right Ctrl+Alt+Shift+向右箭头
Move Line Down Alt+Shift+向下箭头
Move Line Up Alt+Shift+向上箭头

第1章 Python编程起步

Code Completion: 代码补全，不过可以进行全局设置，每次敲入字母时会自动提示进行补全
设置步骤如下：File -> settings

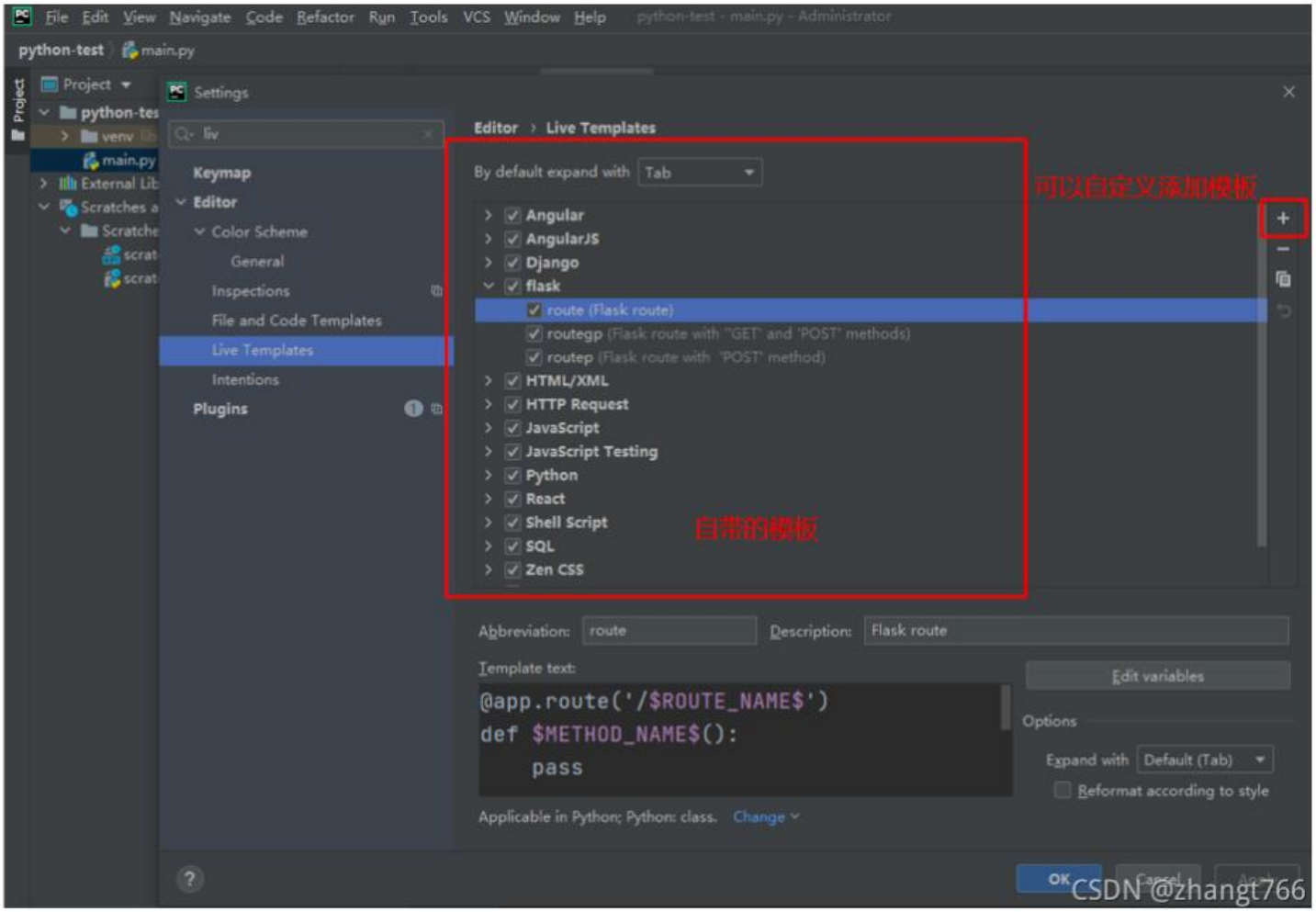


在同样的窗口，可以设置忽略大小写补全

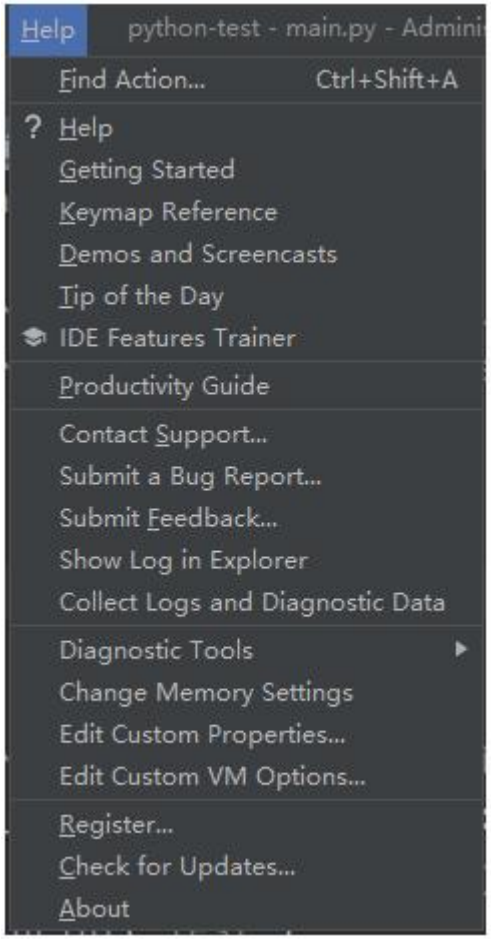


Insert Live Template: 快速插入模板。

默认模板，路径File -> Settings，也可以通过点击+号 自己添加



5. Help（帮助）



Keymap Reference: 查看快捷键清单（）



第1章 Python编程起步

PyCharm

Find any action inside the IDE	Ctrl + Shift + A

CREATE AND EDIT

Show intention actions	Alt + Enter
Basic code completion	Ctrl + Space
Smart code completion	Ctrl + Shift + Space
Type name completion	Ctrl + Alt + Space
Complete statement	Ctrl + Shift + Enter
Parameter information / context info	Ctrl + P / Alt + Q
Quick definition	Ctrl + Shift + I
Quick / external documentation	Ctrl + Q / Shift + F1
Generate code	Alt + Insert
Override / implement members	Ctrl + O / Ctrl + I
Surround with...	Ctrl + Alt + T
Comment with line comment	Ctrl + /
Extend / shrink selection	Ctrl + W / Ctrl + Shift + W
Optimize imports	Ctrl + Alt + O
Auto-indent lines	Ctrl + Alt + I
Cut / Copy / Paste	Ctrl + X / Ctrl + C / Ctrl + V
Copy document path	Ctrl + Shift + C
Paste from clipboard history	Ctrl + Shift + V
Duplicate current line or selection	Ctrl + D
Move line up / down	Ctrl + Shift + Up / Down
Delete line at caret	Ctrl + Y
Join / split line	Ctrl + Shift + J / Ctrl + Enter
Start new line	Shift + Enter
Toggle case	Ctrl + Shift + U
Expand / collapse code block	Ctrl + NumPad + / -
Expand / collapse all	Ctrl + Shift + NumPad + / -
Save all	Ctrl + S

VERSION CONTROL

VCS operations popup...	Alt + `
Commit	Ctrl + K
Update project	Ctrl + T
Recent changes	Alt + Shift + C
Revert	Ctrl + Alt + Z
Push...	Ctrl + Shift + K
Next / previous change	Ctrl + Alt + Shift + Down / Up

MASTER YOUR IDE

Find action...	Ctrl + Shift + A
Open a tool window	Alt + [0-9]
Synchronize	Ctrl + Alt + Y
Quick switch scheme...	Ctrl + `
Settings...	Ctrl + Alt + S
Jump to source / navigation bar	F4 / Alt + Home
Jump to last tool window	F12
Hide active / all tool windows	Shift + Esc / Ctrl + Shift + F12
Go to next / previous editor tab	Alt + Right / Alt + Left
Go to editor (from a tool window)	Esc
Close active tab / window	Ctrl + Shift + F4 / Ctrl + F4

FIND EVERYTHING

Search everywhere	Double Shift
Find / replace	Ctrl + F / R
Find in path / Replace in path	Ctrl + Shift + F / R
Next / previous occurence	F3 / Shift + F3
Find word at caret	Ctrl + F3
Go to class / file	Ctrl + N / Ctrl + Shift + N
Go to file member	Ctrl + F12
Go to symbol	Ctrl + Alt + Shift + N

NAVIGATE FROM SYMBOLS

Declaration	Ctrl + B
Type declaration (JavaScript only)	Ctrl + Shift + B
Super method	Ctrl + U
Implementation(s)	Ctrl + Alt + B
Find usages / Find usages in file	Alt + F7 / Ctrl + F7
Highlight usages in file	Ctrl + Shift + F7
Show usages	Ctrl + Alt + F7

REFACTOR AND CLEAN UP

Refactor this...	Ctrl + Alt + Shift + T
Copy... / Move...	F5 / F6
Safe delete...	Alt + Delete
Rename...	Shift + F6
Change signature...	Ctrl + F6
Inline...	Ctrl + Alt + N
Extract method	Ctrl + Alt + M
Introduce variable/ parameter	Ctrl + Alt + V / P
Introduce field / constant	Ctrl + Alt + F / C
Reformat code	Ctrl + Alt + L

6597

ANALYZE AND EXPLORE

Show error description	Ctrl + F1
Next / previous highlighted error	F2 / Shift + F2
Run inspection by name...	Ctrl + Alt + Shift + I
Type / call hierarchy	Ctrl + H / Ctrl + Alt + H

NAVIGATE IN CONTEXT

Select in...	Alt + F1
Recently viewed / Recent locations	Ctrl + E / Ctrl + Shift + E
Last edit location	Ctrl + Shift + Back
Navigate back / forward	Ctrl + Alt + Left / Right
Go to previous / next method	Alt + Up / Down
Go to line / column...	Ctrl + G
Go to code block end / start	Ctrl +] / [
Add to favorites	Alt + Shift + F
Toggle bookmark	F11
Toggle bookmark with mnemonic	Ctrl + F11
Go to numbered bookmark	Ctrl + [0-9]
Show bookmarks	Shift + F11

BUILD, RUN, AND DEBUG

Run context configuration	Ctrl + Shift + F10
Run / debug selected configuration	Alt + Shift + F10 / F9
Run / debug current configuration	Shift + F10 / F9
Step over / into	F8 / F7
Smart step into	Shift + F7
Step out	Shift + F8
Run to cursor / Force run to cursor	Alt + F9 / Ctrl + Alt + F9
Show execution point	Alt + F10
Evaluate expression...	Alt + F8
Stop	Ctrl + F2
Stop background processes...	Ctrl + Shift + F2
Resume program	F9
Toggle line breakpoint	Ctrl + F8
Toggle temporary line breakpoint	Ctrl + Alt + Shift + F8
Edit / view breakpoint	Ctrl + Shift + F8

jetbrains.com/pycharm
jetbrains.com/help/pycharm
blog.jetbrains.com/pycharm
@pycharm

JET BRAINS

CSDN @zhangt766

本章小结

- Python语言
- Python两个系列版本
- Python语言的特点
- Python的应用领域
- Python及扩展包的安装和管理
- 开发环境安装及使用



第2章 Python语言基础



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



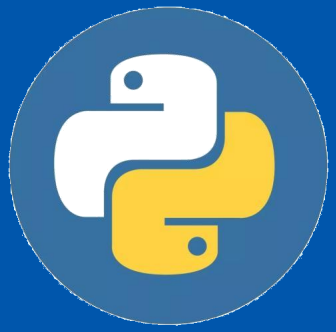
西北工业大学
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

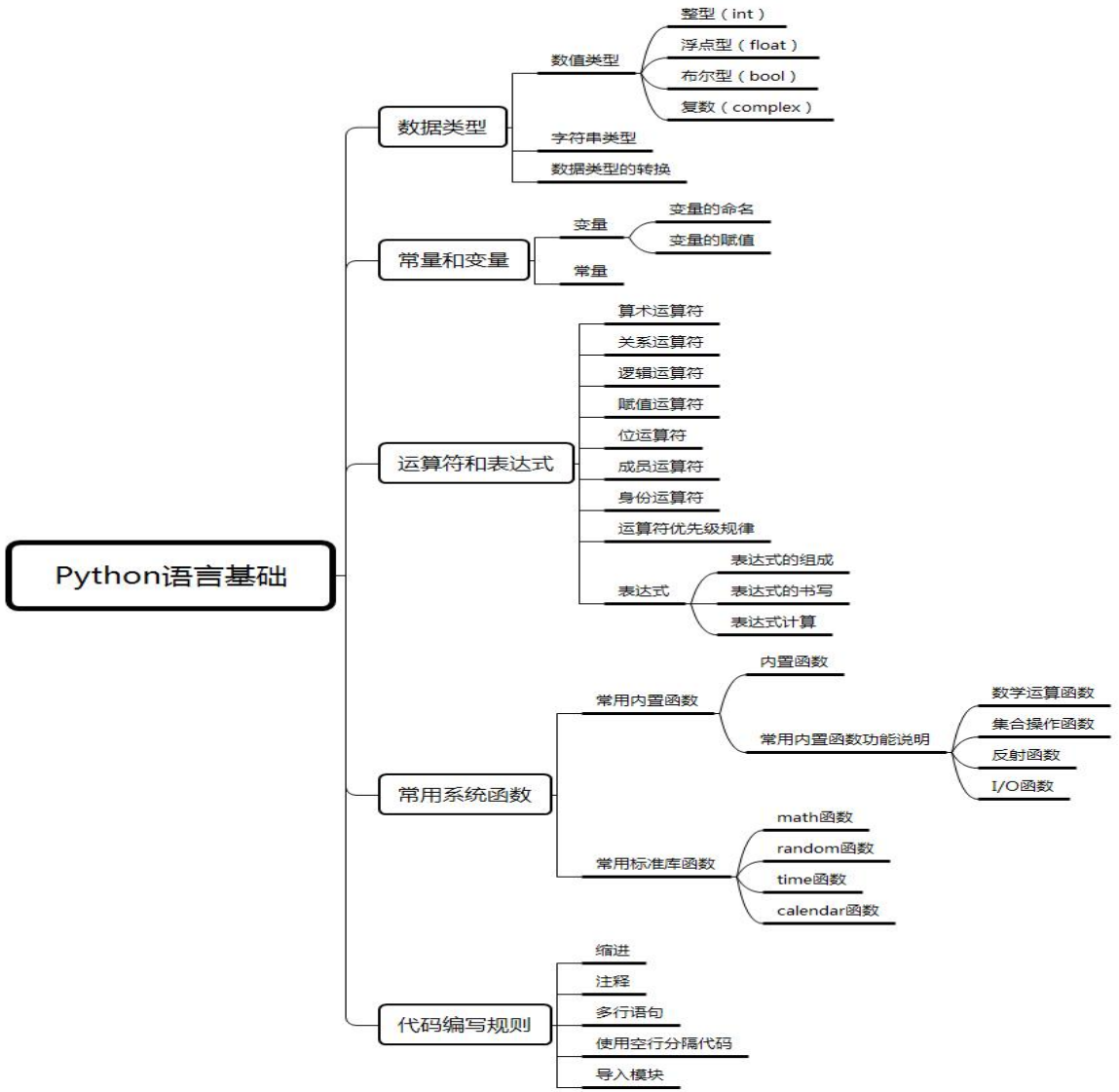
QQ 群：　Python程序设计
760581125



《Python语言程序设计》

第2章 Python语言基础

第2章 Python语言基础





第2章 Python语言基础

2.1 数据类型

2.2 常量和变量

2.3 运算符和表达式

2.4 常用系统函数

2.5 代码编写规则

2.1 数据类型

1.数值类型

数值类型包括：整型、浮点型、布尔型和复数。

（1）整型（int）

整型是带正负号的整数数据。

整型数据有4种表示形式：

- 十进制整数：由数字0~9组成；
- 二进制整数：由数字0和1组成，以0b开头；
- 八进制整数：由数字0~7组成，以0o开头
- 十六进制整数，由数字0~9以及 A~F（或 a~f）组成，以0x开头。

2.1 数据类型

（2）浮点型（float）

浮点型数据用来表示实数，由整数部分和小数部分组成。

浮点型数据有2种表示形式:

- 十进制小数形式：由数字和小数点组成。需要注意的是浮点数必须有小数点，否则作为整型处理。
- 科学计数形式：科学计数法最适宜表示很大或很小的数，以字母e（或E）表示以10为底的指数。

2.1 数据类型

（3）布尔型（bool）

布尔型数据用来描述逻辑运算的结果，True表示真，False表示假。布尔型数据在数值计算中相当于整型，用1代表True（真），用0代表False（假）。

在Python中任何为0或空的数据其布尔值均为False，其他的值为True，如数字0、0.0、None、空字符串、空元组、空序列等。

2.1 数据类型

（4）复数（complex）

复数型数据用来表示数学中的复数，由实数部分和虚数部分两部分组成，可以用a+bj或complex（a,b）表示。

a、b称为复数的实部和虚部，均为浮点数，j为虚数单位（j2=-1）。对于复数x，可以用x.real和x.imag分别获取复数的实部和虚部。

2.1 数据类型

2.字符串（string）

字符串类型指用引号括起来的一个或多个字符。单行字符串用单引号（'）或双引号（"）括起来，多行字符串用三引号（'''）括起来。

例如："I'm learning Python.“

'"Python3.7" is a string.‘

'''This is a long string,
spanning multiple lines'''

2.1 数据类型

在字符串中使用特殊字符时，需要用反斜杠（\）转义字符。

转义字符	描述	转义字符	描述
\(在行尾时)	续行符	\v	纵向制表符
\\	反斜杠符号	\t	横向制表符
\'	单引号	\r	回车
\"	双引号	\f	换页
\a	响铃	\o	八进制数代表的字符
\b	退格(Backspace)	\x	十六进制数代表的字符
\n	换行	\000	终止符，\000后的字符全部忽略

2.1 数据类型

3.数据类型的转换

函数	作用
int(x)	将 x 转换成整数类型
float(x)	将 x 转换成浮点数类型
bool(x)	将x转换为布尔型
str(x)	将 x 转换为字符串
eval(str)	计算在字符串中的有效 Python 表达式，并返回一个对象
chr(x)	将整数 x 转换为一个字符
ord(x)	将一个字符 x 转换为它对应的整数值
bin(x)	将整数x转换为二进制字符串
hex(x)	将整数 x 转换为十六进制字符串
oct(x)	将整数 x 转换为八进制的字符串

2.1 数据类型

说明:

1、int()和float()的参数可以是整数、浮点数、分数或数字字符串。

2、ord()和chr()互为反函数，ord()用来返回单个字符的ASCII码，chr()用来返回数值对应的ASCII字符。

3、bin()、oct()和hex()三个进制转换函数的参数必须为整数。

2.2 变量和常量

1. 变量

在高级语言中，采用变量来描述内存单元及其存储的数据。变量名对应该内存单元的地址编号，变量的值对应该内存单元中存储的数据，通过变量名访问变量。在程序运行过程中，变量的值是可变的。



2.2 变量和常量

（1）变量的命名

命名规则：

①变量必须以字母或下划线开头，后面跟任意数目的字母、数字或下划线。

②除了下划线外，变量名中不能有其他符号。

③不能使用系统关键字或内置函数名作为变量名。

④变量名区分大小写。

⑤变量名应尽量用有意义的单词。

⑥慎用小写字母l和大写字母O，这两个字母很容易被错看成数字1和0。

2.2 变量和常量

（1）变量的命名

注意：

Python中，以下划线开始或结束的变量名是系统定义的变量名，对解释器有特殊的意义。

① _x：以一个下划线开头的变量是一个受保护的变量，只有类对象和子类对象可以访问这些员，不能用from module import *语句导入。

② __x__:前后为双下划线的变量名，是系统定义的特殊成员。

③ __x:以双下划线开头的变量是私有类型的变量，只允许类本身进行访问，子类也不可以访问。

2.2 变量和常量

（2）变量的赋值

Python中的变量不需要事先声明变量名和类型，直接赋值就能创建各种类型的变量，并且变量的类型可以随时改变。变量的赋值方式主要有普通赋值、链式赋值、增量赋值和多元赋值。

- 普通赋值

Python赋值语句的格式如下：

<变量名>=<表达式>

该赋值语句的作用是先计算右边表达式的值，然后将左边的变量指向该值。

2.2 变量和常量

（2）变量的赋值

例如：

```
>>> a=5   #创建整型变量a
>>> type(a)   #内置函数type()用来返回变量类型
<class 'int'>
>>> a='hello'   #创建字符串变量a
>>> c    #没有赋值直接使用c，会提示出错
Traceback (most recent call last):
File "<pyshell#22>", line 1, in <module>
    c
NameError: name 'c' is not defined
```

2.2 变量和常量

（2）变量的赋值

```
>>> b=a   #创建变量b，a、b指向同一个值
>>> id(a)   #内置函数id()用来返回变量所指值的内存地址
32221920
>>> id(b)
32221920
```

说明：

Python采用基于值的内存管理方式，如果为不同变量赋相同值，这个值在内存中只有一个地址，多个变量指向同一个内存地址。

2.2 变量和常量

（2）变量的赋值

当一个变量不再使用时，Python会自动回收内存空间，也可以使用del语句删除一个或多个变量。

```
>>> del a   #使用del删除变量a
>>> del b，c   #使用del删除变量多个变量，变量间使用逗号
```

2.2 变量和常量

（2）变量的赋值

- 链式赋值

链式赋值的语句形式如下：

<变量1>=<变量2>=...=<变量n>=<表达式>

链式赋值用于为多个变量赋同一个值，该语句的作用是先计算右侧表达式的值，然后将左边的变量全部指向该值。

例如：

a=b=c=6

三个变量指向同一个数据对象6，三个变量的值均为6。

2.2 变量和常量

（2）变量的赋值

- 增强赋值

Python提供了增强赋值方式，赋值语句x+=5等价于x=x+5。

例如：

```
>>> x=10
>>> x-=1   #相当于x=x-1
>>> x*=5   #相当于x=x*5
```

说明：

增强赋值的优点是书写更简洁，执行的更快。如x+=5比x=x+5执行得快，因为后者右边还要拷贝一个x。

2.2 变量和常量

（2）变量的赋值

- 多元赋值

多元赋值的语句形式如下：

<变量1>，<变量2>，…，<变量n>=<表达式1>，<表达式2>，…，<表达式n>

赋值号两边的变量和表达式数量要一致，多元赋值首先计算赋值号右边的各表达式的值，然后依次赋值给左边的变量。

例如：

```
>>> x,y=1,2
>>> x,y=y,x   #实现两数交换
```

2.2 变量和常量

2. 常量

常量就是在程序的运行过程中，不发生改变的量。在基本数据类型中，常量按其值的类型可分为整型、浮点型、字符型、布尔型和复数型。

例如：

9、-12是整型常量，3.14是浮点型常量，'work hard'是字符串型常量。

2.3 运算符和表达式

1. 算术运算符

算术运算符用来实现数学运算。

运算符	描述	实例
+	加法	6.8+2结果8.2
-	减法	25-7结果18
*	乘法	5*6.0结果30.0
/	除法	10/4结果2.5
%	模运算符或取余运算符，返回余数	10%3结果1，10%-3结果-2
**	幂运算符	2**10结果1024
//	整除，返回商的整数部分	10//3结果3

2.3 运算符和表达式

注意：

① “+”除了用于算术加法，还可用于字符串、列表、元组的合并和拼接。

② “-”除了作为减法运算符外，还可以进行取反运算。

③ “*”除了用于算术乘法外，还可用于字符串、序列元素的重复。

④除数不能为0。

⑤ “%”运算符用于整数或实数的取余运算，如果除数是负数，那么取得的结果也一定是负数。

⑥ “//”是整除运算符，运算结果只保留整数部分，小数部分将被舍弃。

2.3 运算符和表达式

2. 关系运算符

关系运算符又叫比较运算符，用于比较关系运算符两边的值，运算结果为True或False。

运算符	描述	实例
==	等于，如果两个操作数相等，结果为True；否则为False	"b"=="B"结果False
!=	不等于，如果两个操作数不相等，结果为True；否则为 False	5!=5.0结果False
>	大于，如果左边的操作数大于右边的操作数，结果为True；否则为 False	13>2结果True
<	小于，如果左边的操作数小于右边的操作数，结果为True；否则为 False	"18"<"2"结果True
>=	大于等于，如果左边的操作数大于或等于右边的操作数，结果为True；否则为 False	"abc">="ABC"结果True
<=	小于等于，如果左边的操作数小于或等于右边的操作数，结果为True；否则为 False	-3.6<=-4.1结果False

2.3 运算符和表达式

注意：

1、使用关系运算符的前提是，操作数之间是可以比较大小的。

2、如果两个操作数均为数值型，按其实际大小进行比较。

3、如果两个操作数均为字符型，按字符的ASCII码值从左到右依次比较。

2.3 运算符和表达式

3. 逻辑运算符

逻辑运算符是对关系表达式或逻辑值进行运算，运算结果为True和False。

运算符	描述（a=True，b=False）	实例
and	与，当两个操作数均为True时，结果为True；否则为False	a and b结果False
or	或，当两个操作数之一为True时，结果为True；否则为 False	a or b结果True
not	非，当操作数为True时，结果为False；当操作数为False时结果为 True	not a结果False not b结果True

2.3 运算符和表达式

4. 赋值运算符

常用的赋值运算符

运算符	描述	实例
=	直接赋值	a=8.5
+=	加法赋值	a+=10相当于a=a+10
-=	减法赋值	a-=10相当于a=a-10
*=	乘法赋值	a*=10相当于a=a*10
/=	除法赋值	a/=10相当于a=a/10
%=	取模赋值	a%=10相当于a=a%10
**=	幂赋值	a**=10相当于a=a**10
//=	整除赋值	a//=10相当于a=a//10

2.3 运算符和表达式

注意：

①不要混淆赋值号“=”和等号“==”，两者意义完全不同。

②编程时，为了优化代码，获得更高的运行效率，推荐使用增强赋值方式。

2.3 运算符和表达式

5. 位运算符

位运算的规则是先把数字转换成二进制数，再进行运算，然后将运算结果再转换为原来的进制。

5. 位运算符

运算符	描述	实例(a=36,b=13)
&	按位与，按位将两个操作数对应的二进制数一一对应，只有对应两个数位都为1，该位的结果为1，否则为0	a&b结果为4(二进制00000100)
|	按位或，按位将两个操作数对应的二进制数一一对应，只要对应两个数位有一个为1，该位的结果为1	a|b结果为45(二进制00101101)
^	按位异或，当两个对应的二进制位相异时，结果为1	a^b结果为41(二进制00101001)
~	按位取反，将数据的每个二进制位取反，把1变为0，把0变为1	~a结果为-37(二进制10111101)
<<	左移，将数据的二进制位依次全部左移若干位，由<<符号右边的数指定移动的位数，高位丢弃，低位补0	a<<3结果为288(二进制00100000)
>>	右移，将数据的二进制位依次全部右移若干位，由>>符号右边的数指定移动的位数，高位补0	a>>3结果为4(二进制00000100)

2.3 运算符和表达式

6. 成员运算符

成员运算符用于判断一个元素是否在一个序列中，序列可以是字符串、列表、元组、集合和字典等。

运算符	描述	实例
in	用于在指定的序列中查找某个值是否存在，存在返回True，否则返回False	'a' in 'abcd' 结果为True
not in	用于在指定的序列中查找某个值是否不存在，不存在返回True，否则返回False	5 not in [7,2,5,9]结果为False

2.3 运算符和表达式

7. 身份运算符

身份运算符用于测试两个变量是否为同一个对象，如果是同一个对象，则两者具有相同的内存地址。

运算符	描述	实例
is	如果运算符两侧的变量指向相同的对象，返回True，否则返回False	x is y
is not	如果运算符两侧的变量指向相同的对象，返回False，否则返回True	x is not y

2.3 运算符和表达式

8. 运算符优先级

若一个表达式中有多个运算符，先执行优先级高的运算符，后执行优先级低的运算符，同一优先级的运算符要按照从左到右的顺序执行。

2.3 运算符和表达式

优先级	运算符	描述
1	**	幂
2	~ + -	按位取反，一元加号和减号
3	* / % //	乘、除、取模和整除
4	+ -	加法和减法
5	<< >>	左移、右移
6	&	按位与
7	^ |	按位异或和按位或
8	<= < > >=	关系运算符
9	== !=	关系运算符
10	= %= /= //= -= += *= **=	赋值运算符
11	is is not	身份运算符
12	in not in	成员运算符
13	not and or	逻辑运算符

2.3 运算符和表达式

9. 表达式

（1）表达式的组成

表达式由变量、常量、运算符、函数和圆括号按一定的规则组成。表达式的运算要遵循运算符的优先级规律，运算后得到一个确定的值。

例如：（2+x*y*z）**3/(1/x+1/y)

2.3 运算符和表达式

（2）表达式的书写

- 表达式中的“*”不能省略。

例如：b2-4ac写成Python表达式为b*b-4*a*c。

- 表达式只能使用圆括号改变运算的优先顺序，括号要成对出现。

例：将数学表达式$\frac{(2+xyz)^3}{\frac{1}{x}+\frac{1}{y}}$，写成Python表达式。

（3）表达式计算

例如：计算表达式的值。

1、8%-3*3**2+23//5-True

2、len('beijing'+'shanghai')/2+ord('c')%4

2.4 常用系统函数

Python提供了丰富的函数，分为内置函数、标准库函数和第三方库函数。

1. 常用内置函数

（1）内置函数

Python提供了一些内置函数，这类函数不需要导入任何模块即可直接使用，主要完成一些运算符无法实现的运算功能。

在命令行输入dir(__builtins__)语句，即可查看所有内置函数和内置对象。使用help（函数名）即可查看某个函数的用法。

2.4 常用系统函数

（2）常用内置函数功能说明

- 数学运算函数

函数	功能说明
abs(x)	求绝对值。返回数字x的绝对值或复数x的模
complex([real[, imag]])	创建一个复数
divmod(a,b)	分别取a/b的商和余数构成一个元组。
pow(x,y)	返回x的y次幂
round(x[,n])	对x进行四舍五入，保留n位小数

2.4 常用系统函数

- 集合操作函数

函数	功能说明
all(iterable)	元素都为真时，函数值为True；元素为空时，函数则返回False
any(iterable)	元素有一个为真时，函数值为True；元素为空时，函数则返回False
max(x)	返回序列x中的最大值
min(x)	返回序列x中的最小值
sum(x)	返回序列x中所有元素之和，所有元素必须为数字
range([start,]end[,step])	产生一个[start,end)区间内以step为步长的整数序列，默认从0开始
list(iterable)	产生一个列表
tuple(iterable)	产生一个元组

2.4 常用系统函数

- 反射函数

函数	功能说明
getattr(object,name[,defalut])	获取一个类的属性
globals()	返回一个描述当前全局符号表的字典
hasattr(object,name)	判断对象object是否包含名为name的特性
hash(object)	如果对象object为哈希表类型，返回对象object的哈希值
id(object)	返回对象的唯一标识
isinstance(object,classinfo)	判断object是否是class的实例
issubclass(class,classinfo)	判断是否是子类
locals()	返回当前的变量列表
map(function,iterable, ...)	遍历每个元素，执行function操作
memoryview(obj)	返回一个内存镜像类型的对象
next(iterator[,default])	类似于iterator.next()
object()	基类

2.4 常用系统函数

- 集合操作函数

函数	功能说明
dict()	创建数据字典
set(iterable)	创建集合
frozenset([iterable])	产生一个不可变的集合
sorted(iterable[,cmp[,key[, reverse]]])	对列表、元组、字典、集合或其他可迭代的对象进行排序，并返回新列表

说明：

①range是很常用的一个内置函数，其语法格式为range([start,]end[,step])，其中参数start默认为0，step默认为1，该函数返回[start,end)左闭右开区间以step为步长的整数。

②sorted()函数默认对序列实现升序排序，将reverse参数设置为True，可以实现降序排序。

2.4 常用系统函数

- 反射函数

函数	功能说明
property([fget[,fset[,fdel[,doc]]]])	属性访问的包装类，设置后可以通过c.x=value等来访问setter和getter
reload(module)	重新加载模块
setattr(object,name,value)	设置属性值
repr(object)	将一个对象变幻为可打印的格式
staticmethod	声明静态方法，是个注解
super(type[,object-or-type])	引用父类
type(object)	返回该object的类型
vars([object])	返回对象的变量，若无参数与dict()方法类似

2.4 常用系统函数

- I/O函数

函数	功能说明
input([prompt])	获取用户输入，输入内容作为字符串处理
print()	打印函数
open(name[,mode[,buffering]])	打开文件

说明：

①input()函数接收用户输入的数据，返回一个字符串，如果要输入数值，可以使用转换函数将字符串转换为数值。

②print()函数格式print(value1,value2,...,sep= ' ',end='\n '),其中，value为要输出的内容，无参数时将输出一个空行；sep用来设置各输出项之间的分隔符，默认为空格；end用来设置结尾字符，默认是换行符\n，可以设置为其他字符。

2.4 常用系统函数

2. 常用标准库函数

常用的标准库有数学库模块（math）、随机数模块（random）、时间模块（time）和日历模块（calendar）。

Python程序若要使用这些标准库函数，必须先使用import命令将模块导入。

2.4 常用系统函数

（1）math模块

math模块提供了许多数学运算函数。

函数	功能说明
math.e	数字常量,自然常数e
math.pi	数字常量，圆周率pi
math.ceil(x)	向上取整，返回大于或等于x的最小整数
math.floor(x)	向下取整，返回小于或等于x的最大整数
math.trunc(x)	返回x的整数部分
math.fabs(x)	返回x的绝对值
math.fmod(x)	返回x%y（取余）
math.factorial（x）	返回x的阶乘
math.gcd(x,y)	返回x、y的最大公约数
math.exp(x)	返回e的x次幂
math.log(x[,base])	返回x的以base为底的对数

2.4 常用系统函数

（2）math模块

math模块提供了许多数学运算函数。

函数	功能说明
math.log10(x)	返回x的以10为底的对数
math.pow(x,y)	返回x的y次方
math.sqrt(x)	返回x的平方根
math.degrees(x)	把x从弧度转换为角度
math.radians(x)	把x从角度转换为弧度
math.sin(x)	返回x（弧度）的三角正弦值
math.cos(x)	返回x（弧度）的三角余弦值
math.tan(x)	返回x（弧度）的三角正切值

2.4 常用系统函数

（2）random模块

随机数可以用于数学、游戏及安全领域，还经常被嵌入到算法中，以提高算法的效率和安全性。

函数	功能说明
random.choice（seq）	从序列的元素中随机挑选一个元素，如random.choice(5),从0到4中随机挑选一个整数
random.randrange（[start,]stop[,step]）	在指定范围内，按指定step递增的集合中获取一个随机数，step默认值为1，如random.randrange(3),从0到2中随机挑选一个整数
random.random（）	返回一个[0,1）范围内的实数
random.seed（[x]）	改变随机数生成器的种子seed。默认以系统时间为种子
random.shuffle（list）	将序列的所有元素随机排序
random.uniform（x,y）	随机生成一个[x,y]范围内的实数
random.randint（x,y）	随机生成一个[x,y]范围内的整数

2.4 常用系统函数

（3）time模块

Python中通常有2种表示时间的方式：

①时间戳，是从1970年1月1日00:00:00开始到现在的秒数。

②时间元组struct_time，共有9个元素，分别是tm_year（年）、tm_mon（月，1-12）、tm_mday（日，1-31）、tm_hour（时，0-23）、tm_min（分，0-59）、tm_sec（秒，0-59）、tm_wday（星期几，0-6，0表示周日）、tm_yday（一年中的第几天，1-366）、tm_isdst（是否是夏令时，默认为1夏令时）。

2.4 常用系统函数

（3）time模块

函数	功能说明
time.asctime([tupletime])	将时间元组转换为时间字符串，若时间元组省略，返回当前系统时间
time.ctime([secs])	将以秒数表示的时间转换为时间字符串
time.gmtime([secs])	接收时间戳并返回时间元组
time.localtime([secs])	接收时间戳并返回当前时间的时间元组
time.strftime(format[, t])	将时间元组格式化为指定格式的时间字符串
time.time()	返回当前时间的时间戳（1970纪元后经过的浮点秒数）

2.4 常用系统函数

4.calendar模块

此模块的函数都是与日历相关的，例如打印某月的月历。

函数	功能说明
calendar.calendar(year,w=2,l=1,c=6)	返回一个多行字符串格式的year年的年历，3个月一行，间隔距离为c。 每日宽度间隔为w字符。每行长度为21* w+18+2* c。l是每星期行数
calendar.firstweekday( )	返回当前每周起始日期的设置。默认返回 0，即星期一
calendar.isleap(year)	是闰年返回 True，否则为 False
calendar.leapdays(y1,y2)	返回在y1，y2两年之间的闰年总数
calendar.month(year,month,w=2,l=1)	返回指定年月的日历。两行标题，一周一行。每日宽度间隔为w字符。每行的长度为7*w+6。l是每星期的行数。

2.4 常用系统函数

(4) calendar模块

函数	功能说明
calendar.monthcalendar(year,month)	返回一个整数列表，每个子列表代表一个星期
calendar.monthrange(year,month)	返回两个整数。第一个是该月的星期几的日期码，第二个是该月的日期码。日从0（星期一）到6（星期日）;月从1到12
calendar.weekday(year,month,day)	返回给定日期的日期码

2.5 代码编写规则

1. 缩进

Python程序通过代码块的缩进区分代码的层次，缩进结束标志着一个代码块的结束。缩进的空格数是可变的，通常以4个空格为基本缩进单位，不要使用Tab。同一级别代码块的缩进量必须相同。

例如：

```
mark=80
if mark>=60:
    print("考试通过")
else:
    print("请继续努力！")
```

2.5 代码编写规则

2. 注释

编程时一定要养成良好的注释习惯，注释可以增强程序的可读性，也便于后期进行代码维护。Python中用#号表示单行注释，多行注释可以用三引号（'''）括起来。

3. 多行语句

如果一行语句过长，可以使用反斜杠“\”将一行语句分成多行显示。如果语句中包含 []、{} 或 ()就不需要使用反斜杠换行。

例如：

```
num =int_one+int_two+ int_three\
+int_four+int_five
```

2.5 代码编写规则

4.使用空行分隔代码

类的方法之间或函数之间用空行分隔，类和函数入口之间通常也用一个空行分隔，以突出函数的入口，增加程序的可读性。

5.导入模块

import语句应该放在文件头部，每个import语句只导入一个模块。模块导入的顺序是内置模块、第三方模块、自定义模块。

本章小结

1.Python的基本数据类型

2.变量和常量。

3.运算符和表达式

4.常用系统函数

5.代码编写规则



第3章 Python组合数据类型



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



西北工业大学
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

QQ 群：　Python程序设计
760581125



第3章 Python组合数据类型



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



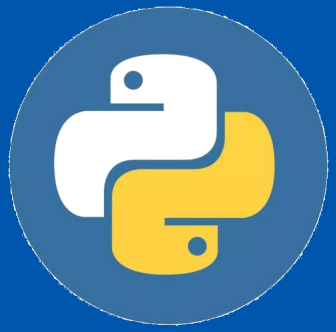
西北工业大学
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

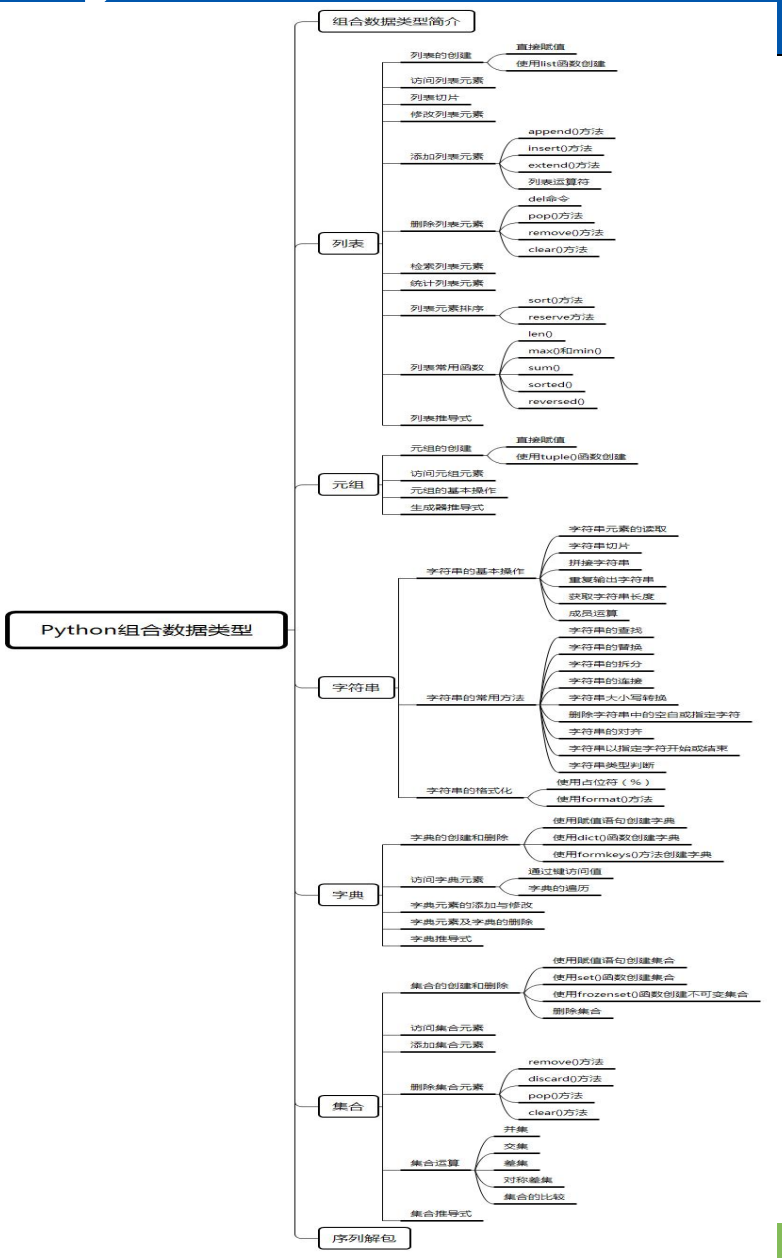
QQ 群：　Python程序设计
760581125



《Python程序设计》

第3章 Python组合数据类型

第3章 Python组合数据类型





第3章 Python组合数据类型

3.1 组合数据类型简介

3.2 列表

3.3 元组

3.4 字符串

3.3 字典

3.5 集合

3.3 序列解包



第3章 Python组合数据类型

3.1 组合数据类型简介

Python的组合数据类型有3类，分别是序列类型、映射类型和集合类型。

- 序列类型由一系列按特定顺序排列的元素组成，元素通常是相关的，可通过索引来访问。常用的序列类型有列表、元组和字符串。

3.1 组合数据类型简介

序列中的每个元素都分配一个数字，即它的索引，索引又称下标或位置。序列支持双向索引，如果使用正向索引，第一个元素的索引为0，第二个元素的索引为1，依此类推；如果使用负向索引，则最后一个元素的索引为-1，倒数第二个元素的索引为-2，依此类推。

序列	P	y	t	h	o	n		3	.	7
正向索引	0	1	2	3	4	5	6	7	8	9
反向索引	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1



第3章 Python组合数据类型

3.1 组合数据类型简介

- 映射类型是一种键值对，一个键只能对应一个值，通过键可以访问值。字典是Python中唯一的映射类型，字典中的元素是无序的。
- 集合类型与数学中的集合概念一致，是一种无序不重复的元素集。集合中的元素只能是固定数据类型，如整型、字符串、元组等，而列表、字典等可变数据类型不能作为集合中的元素。

3.2 列表

列表（List）是一个有序、可变序列。列表元素的数据类型可以不同，可以是基本类型、组合数据类型或自定义数据类型。列表的所有元素放在一对方括号“[ ]”中，相邻元素之间用逗号分隔开。

例如：

[78,83,67,89,75]

['Python',3.7, '高级语言',2018]

3.2 列表

1.列表的创建

（1）直接赋值

使用等号直接将一个列表赋值给变量即可创建一个列表。例如：

```
>>> scores=[78,83,67,89,75]
>>> scores
[78, 83, 67, 89, 75]
>>> empty=[]   #创建一个空列表
>>> empty
[]
```

3.2 列表

（2）使用list函数创建

使用list函数可以将字符串、元组、字典、集合等数据类型的数据转换成列表。

例如：

```
>>> summer=list('summer')
>>> summer
['s', 'u', 'm', 'm', 'e', 'r']
```

3.2 列表

2. 访问列表元素

使用索引可以直接访问列表元素，格式为：列表名[索引]。如果指定的索引不存在，将出现下标越界错误。

例如：

```
>>> scores=[78,83,67,89,75]
>>> scores[2]   #注意正向索引从0开始
67
>>> scores[5]   #索引5越界，抛出异常
```

3.2 列表

3. 列表切片

切片的功能非常强大，是Python序列的重要操作之一。列表切片操作可以截取列表中的任意部分，得到一个新列表。切片操作的语法格式如下：

列表名[开始索引:结束索引:步长]

其功能是获取从开始索引开始到结束索引（但不包含）为止的所有元素组成的子列表。如果省略开始索引，默认从0开始；如果省略结束索引，则截取到列表最后；如果省略步长，则步长为1。

3.2 列表

如果一个列表中的元素也是列表，那就构成了嵌套列表，也叫多维列表。嵌套列表元素也是通过索引来访问的，其格式为：

列表名[索引1][索引2]...[索引n]。

例如：

```
>>> students=[ ['001','李明','男',2000],['002','王伟','男',2001] ]
>>> students[0][1]   #用2个索引访问二维列表的元素
'李明'
```

3.2 列表

例如：

```
>>> numbers=[1,2,3,4,5,6,7,8]
>>> numbers[2:6:2]  #步长为2
[3, 5]
>>> numbers[2:6]  #步长为1，结果不包含索引为6的元素
[3, 4, 5, 6]
>>> numbers[:6]
[1, 2, 3, 4, 5, 6]
>>> numbers[2:]
[3, 4, 5, 6, 7, 8]
```

3.2 列表

注意：

（1）切片开始点的元素包含在结果之中，而结束点的元素不在切片之内。当步长为正时，开始索引应小于结束索引，从左到右提取列表元素；当步长为负时，开始索引应大于结束索引，从右到左提取列表元素。

（2）与访问列表元素不同，切片操作不会因为索引越界而抛出异常，而是提取所有列表元素或返回一个空列表。

3.2 列表

可以通过给切片赋值的方式添加、删除和修改列表元素。

例如：

```
>>> numbers=[1,5]
>>> numbers[1:1]
[]
>>> numbers[1:1]=[2,3,4]  #在列表中插入元素
>>> numbers
[1, 2, 3, 4, 5]
>>> numbers[5:]=[6]  #在列表尾部添加元素
>>> numbers
[1, 2, 3, 4, 5, 6]
```

3.2 列表

4. 修改列表元素

列表可以通过指定索引，对相应元素进行赋值修改。

例如：

```
>>> languge=['Python','C','Visual Basic','Java']
>>> languge[2]='Visual Basic.NET'
>>> languge
['Python', 'C', 'Visual Basic.NET', 'Java']
```

3.2 列表

5. 添加列表元素

添加和删除列表元素是很常用的操作，可以使用append()、extend()和insert()方法实现。

（1）append()方法

append()方法用于在列表末尾追加一个元素。方法的调用格式：

对象名.方法名（参数列表）

例如：

```
>>>languge=['Python', 'C', 'Visual Basic.NET', 'Java']
>>>languge.append('C++')
```

3.2 列表

（2） insert()方法

insert()方法用于在列表任意指定位置插入一个元素。

```
>>> languge= ['Python', 'C', 'Visual Basic.NET', 'Java', 'C++']
>>> languge.insert(2,'C#')
```

（3） extend()方法

extend()方法用于将另一个列表中的所有元素追加到当前列表的末尾。

```
>>>languge= ['Python', 'C', 'Visual Basic.NET']
>>>languge.extend([ 'Java', 'C++'])
```

3.2 列表

6. 删除列表元素

删除列表元素可以使用del命令，也可以使用pop()、remove()和clear()方法。

（1）del命令

del命令可以删除列表中指定位置的元素或整个列表。例如：

```
>>>numbers=[1,2,3,4,5]
>>>del numbers[2]   #删除索引为2的列表元素
>>> del numbers     #删除整个列表
```

3.2 列表

（4） 列表运算符

列表运算符“+”和“*”也可以实现增加列表元素的目的，这两个运算符不属于原地操作，而是返回新列表。“+”用于组合列表，“*”用于重复列表。

例如：

```
>>> numbers=[1,2,3]
>>> id(numbers)
1656232
>>> numbers=numbers+[4,5]
>>> numbers=numbers*2
```

3.2 列表

（2）pop()方法

pop()方法用于删除列表指定位置的一个元素（默认为最后一个），并返回该元素。

例如：

```
>>> numbers=[1,2,3,4,5]
>>> numbers.pop()  #默认删除列表最后一个元素
5
>>> numbers.pop(0)  #默认索引为0的列表元素
1
```

注意：pop()方法是唯一一个既修改列表又返回一个非空值的列表方法。

3.2 列表

（3）remove()方法

remove()方法用于删除列表中第一个与指定值相等的元素。如果列表中有多个与指定值相等的元素，要多次使用remove()方法。

例如：

```
>>> numbers=[1,2,3,4,5,3]
>>> numbers.remove(3)   #删除第一个值为3的列表元素
>>> numbers
[1, 2, 4, 5, 3]
```

3.2 列表

（3）remove()方法

使用remove()方法时应注意，如果要删除的元素不在列表中，则抛出异常。为了避免这个错误，最好提前判断要删除的元素是否在列表中。

例如：

```
>>> numbers=[1,2,3,4,5]
>>> 6 in numbers   #使用成员运算符in判断6不是列表元素
False
>>> 2 in numbers   #2是列表元素
True
>>> numbers.remove(2)
```

3.2 列表

（4）clear()方法

clear()方法用于清空列表。例如：

```
>>> numbers=[1,2,3,4,5]
>>> numbers.clear()  #删除列表所有元素
>>> numbers
[]
```

注意：

①clear()方法是清空列表元素，剩下空列表，而del命令会将列表变量删除，不能再访问。

②在列表的中间位置插入或删除列表元素时，会影响该位置后面所有元素的索引，应尽量避免此类操作。

3.2 列表

7. 检索列表元素

index()方法用于返回指定元素在列表中首次出现的索引，若该元素不在列表中则抛出异常。

例如：

```
>>> numbers=[1,2,3,4,5,1,2,3]
>>> numbers.index(3)   #返回列表元素3首次出现的索引
2
>>> numbers.index(8)   #数字8不是列表元素，抛出异常
```

3.2 列表

8. 统计列表元素

count()方法用于计算指定元素在列表中出现的次数。

例如：

```
>>> numbers=[1,2,3,4,5,1,2,3]
>>> numbers.count(1)    #统计数字1在列表中出现的次数
2
>>> numbers.count(6)    #数字6不在列表中为0次
0
```

3.2 列表

9. 列表元素排序

（1）sort()方法

sort()方法用于按照指定的规则对列表元素进行排序，默认的规则是比较元素的大小，按升序排序。sort()方法有两个可选参数：key和reverse。通过设置参数key的值，可以实现以字符串大小、长度等关键字进行排序。通过设置参数reverse的值为True，可以实现降序排序。

例如：

```
>>> numbers=[1,4,3,6,2,5]
>>> numbers.sort()  #默认情况下，按升序排序
```

3.2 列表

（2）reverse()方法

reverse()方法用于将列表所有元素逆序排列。例如：

```
>>> numbers=[1,2,3,4,5]
>>> numbers.reverse()
>>> numbers
[5, 4, 3, 2, 1]
```

注意：sort()和reverse()方法都是原地对列表进行排序，不生成新列表。如果不希望修改原列表，而是生成一个排序后的新列表，可以使用sorted()和reversed()函数实现。

3.2 列表

10. 列表常用函数

列表常用的函数有len()、max()、min()、sorted()和reversed()，函数的引用格式为：函数名(列表名)

（1）len()

len()函数用于返回列表中元素的个数。例如：

```
>>> numbers=[1,2,3,4,5,6]
>>> len(numbers)
6
```

3.2 列表

2.max()和min()

max()和min()函数用于返回列表中所有元素的最大值或最小值。列表元素全部为数值类型时，直接根据数值的大小比较；列表元素全部为字符串类型时，按字符的ASCII码依次进行比较；如果列表元素既有数字类型又有字符串类型时，则无法比较，抛出异常。

例如：

```
>>> numbers=[1,2,3,4,5,6]
>>> max(numbers)   #返回所有数值元素的最大值
6
>>> min(numbers)   #返回所有数值元素的最小值
1
```

3.2 列表

（3）sum()

sum()函数用于对数值型列表元素进行求和，如果存在非数值型元素，则抛出异常。

例如：

```
>>> numbers=[1,2,3,4,5,6]
>>> sum(numbers)
21
```

3.2 列表

（4）sorted()

sorted()函数用于对列表元素进行排序，默认按升序排序，返回一个新列表。通过设置参数reverse的值为True，可以实现降序排序。

例如：

```
>>> numbers=[1,4,3,6,2,5]
>>> sorted(numbers)
[1, 2, 3, 4, 5, 6]
>>> sorted(numbers,reverse=True)
[6, 5, 4, 3, 2, 1]
```

3.2 列表

（5）reversed()

reversed()函数用于将列表元素逆序排列，不排序。该方法不修改原列表，返回一个迭代对象，可以使用list()函数将其转换为列表。

例如：

```
>>> numbers=[1,4,3,6,2,5]
>>> a=reversed(numbers)
>>> a
<list_reverseiterator object at 0x00A6F7F0>
>>> list(a)   #使用list()函数将其转换为列表
[5, 2, 6, 3, 4, 1]
```

3.2 列表

11. 列表推导式

列表推导式也称为列表解析式，其代码简洁，可读性更强，且运行速度快。利用列表推导式可以简单高效地处理一个可迭代对象，生成满足特定需求的列表。

列表推导式的语法格式如下：

[表达式 for 变量 in 序列 [if 条件表达式]]

其中表达式可以是任何运算表达式，变量是序列中遍历的元素值，if条件表达式可以省略。该语句的功能是计算每个变量对应的表达式的值，生成一个新列表。

3.2 列表

列表推导式在逻辑上相当于一个循环，只是形式更加简洁。

例如：

```
>>> list1=[i**2 for i in range(1,5)]
>>> list1
[1, 4, 9, 16]
```

列表推导式的常用功能有：

①实现嵌套列表的平铺

```
>>> list1=[[1,2,3],[4,5,6],[7,8,9]]
>>> [i for a in list1 for i in a]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

3.2 列表

②过滤不符合条件的元素

在列表推导式中，可以使用If条件表达式进行筛选，只在结果列表中保留符合条件的元素。

例如：

```
>>> list1=[3,-6.7,9,10,-21,-5,3.14,0,0.0]
>>> [i for i in list1 if i<=0]   #筛选出小于等于0的元素
[-6.7, -21, -5, 0, 0.0]
>>> [i for i in range(20) if i%2==0]
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

3.2 列表

③同时遍历多个序列

```
>>> [(x,y) for x in [2,3,4] for y in [3,4,5] if x!=y]
[(2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 3), (4, 5)]
>>> [(x,y,x*y) for x in [2,3,4] for y in [3,4,5] if x<y]
[(2, 3, 6), (2, 4, 8), (2, 5, 10), (3, 4, 12), (3, 5, 15), (4, 5, 20)]
```

3.2 列表

【例3-1】已知存放于列表中的n个学生成绩，输出最高分、最低分、总分和平均分，将成绩降序排列，分别输出原始成绩和排序后的成绩。

分析：利用列表的max()、min()和sum()函数分别求得最高分、最低分和总分；利用len()函数求得学生总数，再用总分除以人数求得平均分。利用sorted()进行排序。

3.2 列表

【例3-2】输入一个月份，判断该月份属于哪个季节，并输出。

分析：创建四季的列表春季—3、4、5月，夏季—6、7、8月，秋季9、10、11月，冬季12、1、2月。利用input()输入月份，采用多分支结构实现

3.3 元组

元组（Tuple）与列表类似，也是一个有序序列。区别是元组是一个不可变序列，不能修改，也不能添加或删除元素。元组的所有元素放在一对圆括号“()”中，相邻元素之间用逗号分隔开。

下面的元组都是合法的。

(1,2,3,4,5,6)

('apple', 'orange', 'banana', 'pear')

3.3 元组

1. 元组的创建

（1）直接赋值

使用等号直接将一个元组赋值给变量即可创建一个元组。

例如：

```
>>> numbers=(1,2,3,4,5)  #创建一个元组
>>> empty=()  #创建一个空元组
>>> one=(5,)  #创建只有一个元素的元组
>>> numbers=3*(5+3)  #进行数值运算
>>> numbers=3*(5+3,)  #创建元组
```

3.3 元组

（2）使用tuple()函数创建

使用tuple()函数可以将字符串、列表、字典、集合等数据类型的数据转换成元组。

例如：

```
>>> summer=tuple('summer')
```

- `>>> numbers=tuple([1,2,3,4,5])` #将列表转换为元组
- `>>> tuple1=tuple(range(6,20,3))` #将range函数产生的序列转换为元组
- `>>> empty=tuple()` #创建一个空元组

3.3 元组

2. 访问元组元素

使用索引可以直接访问元组元素，如果指定的索引不存在，将出现下标越界错误。

例如：

```
>>> fruit=('apple','banana','orange','pear',[1,2,3,4])
>>> fruit[-1]   #注意负向索引从-1开始
>>> fruit[6]   #索引越界，抛出异常
```

- >>> food=(('apple','banana'),('bread','milk'))
- >>> food[1][1]

3.3 元组

3. 元组的基本操作

元组不能被修改，因此没有append()、extend()、insert()、remove()和pop()等能够修改序列元素的方法。除此之外，列表的运算符、函数和方法对元组同样适用。

【例3-3】元组的基本操作示例。

```
>>> numbers=(1,2,3)
>>> numbers=numbers+(4,5)  #元组的连接运算
>>> numbers*=2  #元组的重复运算
>>> numbers[:9]  #元组的切片操作
```

3.3 元组

4. 生成器推导式

生成器推导式与列表推导式形式上类似，只是生成器推导式使用圆括号，而列表推导式使用方括号。与列表推导式不同的是，生成器推导式的结果是一个生成器对象，而不是一个元组。使用生成器对象的元素时，可以将其转化为列表或元组，也可以使用生成器对象的__next__()方法或内置函数next()进行遍历，或者使用for循环遍历其中的元素。

3.3 元组

【例3-4】生成器推导式操作示例。

```
>>> tup=(x+3 for x in range(6))  #创建生成器对象
>>> tup
<generator object <genexpr> at 0x005628F0>
>>> tuple(tup)  #将生成器对象转换为元组
(3, 4, 5, 6, 7, 8)
```

3.3 元组

元组有以下适用场合：

- 元组比列表访问速度快。如果定义了一系列常量值，主要用途是对它们进行遍历，而不需要修改元素，建议使用元组。可以认为元组对不需要修改的数据进行了“写保护”，从而使得数据更安全。
- 元组作为不可变序列，与整数、字符串一样，可以作为字典的键，也可以作为集合的元素。而列表既不能作为字典的键，也不能作为集合的元素，因为列表是可变的。

3.4 字符串

字符串是一种非常重要的数据类型，字符串中的字符是按照一定顺序排列的，它是一个不可变的有序序列，支持双向索引。

1. 字符串的基本操作

（1）字符串元素的读取

字符串的索引与列表相同，使用索引可以访问字符串的元素。例如：

```
>>> s1='Python'
>>> s1[0]
>>> s1[5]
```

3.4 字符串

2.字符串切片

字符串切片操作可以截取字符串的任意部分，得到一个新的字符串，切片操作也与列表相同。

例如：

```
>>> s1[:5]   #步长为1，结果不包含索引为5的元素
>>> s1[1:]   #从第2个元素取到最后
>>> s1[::2]   #步长为2
>>> s1[-1::-1]   #将字符串反序
>>> s1[10:]   #开始索引大于字符串长度，返回空字符串
>>> s1[:10]   #结束索引大于字符串长度，提取所有元素
```

3.4 字符串

（3）拼接字符串

“+”号能将字符串拼接起来，形成新的字符串。而对于数字，“+”将做加法运算。

```
>>> "Hello"+"John"
>>>4+5
>>> 'your score is'+80
```

注意：“+”连接的对象必须类型相同，否则报错。

如果必须要实现字符串和数字的连接，可以用str()函数将数字转化为字符串类型后再连接。

3.4 字符串

（4）重复输出字符串

在数值运算中“*”表示乘法，对于字符串，“*”表示重复输出。

```
>>> 'Hello'*2
>>> '#'*10
```

（5）获取字符串长度

len()是Python的常用内置函数，用来获取字符串的长度。

```
>>> len('学习Python3.7')
```

注意：数字、英文、汉字和字符均为一个字符长度

3.4 字符串

2. 字符串的常用方法

Python提供了大量方法用于字符串查找、替换、分离、连接等操作，这里只介绍一些最常用的方法。

（1）字符串的查找

find()和rfind()方法用于查找并返回一个字符串在另一个字符串指定范围（默认为整个字符串）首次和最后一次出现的位置，找不到则返回-1。

```
>>>season='spring,summer,autumn,winter,spring,summer'
>>> season.find('spring')
>>> season.rfind('spring')
```

3.4 字符串

（1）字符串的查找

index()和rindex()方法用于返回一个字符串在另一个字符串指定范围（默认为整个字符串）首次和最后一次出现的位置，找不到则抛出异常。

```
>>> season.index('e')
```

count()方法用于统计一个字符串在另一个字符串中出现的次数，若该字符串不存在则返回0。

```
>>> season.count('summer')
```

3.4 字符串

（2）字符串的替换

replace()用来替换指定字符或字符串的所有重复出现，每次只能替换一个字符或字符串，类似于Word文本编辑器的查找和替换功能。该方法将返回一个被替换后的新字符串。

```
>>> season.replace('summer','夏天')
```

3.4 字符串

（2）字符串的替换

replace()方法无法同时替换多个不同的字符，此功能可以用maketrans()和tranlate()方法组合实现。maketrans()方法用来生成字符映射表，tranlate()按照映射表的对应关系转换并替换其中的字符。maketrans()有两个等长的字符串参数，表示第一个字符串的每个字符都用第二个字符串的对应字符替换。

```
>>> code=str.maketrans('abcdef','defghi') #创建字符映射表
>>> s.translate(code)   #按映射表进行替换
```

3.4 字符串

（3）字符串的拆分

split()方法按指定字符从左侧分隔字符串，rsplit()方法按指定字符从右侧分隔字符串，返回包含分隔结果的列表。

```
>>>season='spring,summer,autumn,winter,spring,summer'
>>> season.split(',')  #用逗号分隔字符串
>>> date='2020/5/1  '
>>> date.rsplit('/')  #用“/”分隔字符串
```

3.4 字符串

（4）字符串的连接

join()方法用来将列表中的多个字符串进行连接生成一个新的字符串，并在相邻两个字符串之间插入指定的字符。

```
>>> season=['spring', 'summer', 'autumn', 'winter', 'spring', 'summer']
>>> sep=','
>>> sep.join(season)   #使用逗号作为分隔符连接字符串
```

注意：前面介绍过用“+”也可以连接字符串，但该运算需要复制大量数据，效率很低，不适合大量长字符串的连接。

3.4 字符串

（5）字符串大小写转换

lower()方法将字符串转换为小写，upper()方法将字符串转换为大写，swapcase()方法实现大小写互换，capitalize()将字符串的首字母转换为大写，其余小写，title()将每个单词的首字母转化能为大写，均生成新的字符串。

```
>>> s='This is a book'
>>> s.lower()   #转换为小写
>>> s.upper()   #转换为大写
>>> s.capitalize()   #首字母转换为大写，其余小写
>>> s.title()   #每个单词首字母转换为大写
```

3.4 字符串

（6）删除字符串中的空白或指定字符

strip()、lstrip()和rstrip()方法分别用来删除字符串两端、左端和右端的空白字符或连续的指定字符。

```
>>> s=' name '
>>> s.lstrip()   #删除左端空白字符
>>> s.rstrip()   #删除右端空白字符
>>> s.strip()   #删除两端空白字符
>>> s='\t\nmy name\n\n'
>>> s.strip()   #删除两端空白字符
```

3.4 字符串

（7）字符串的对齐

center()、ljust()和rjust()方法通过在两端、左端和右端添加填充字符（默认为空格）的方式实现字符串居中对齐、左对齐和右对齐。zfill()方法通过在左侧添加0的方式实现字符串右对齐。

```
>>> s="I'm a Chinese!"
>>> s.center(10)
>>> s.center(20)   #居中对齐，以空格进行填充
>>> s.ljust(20,'*')   #左对齐，以字符*进行填充
```

3.4 字符串

（8）字符串以指定字符开始或结束

startswith()和endswith()方法分别用来判断字符串是否以指定字符开始和结束，通过设置参数可以限定字符串的检测范围。

```
>>> s="I'm a Chinese!"
>>> s.startswith('I')   #判断字符串以I开始
>>> s.endswith('se')
```

3.4 字符串

（9）字符串类型判断

isalnum()、isalpha()、isdigit()、isnumeric()、isdecimal()、isspace()、isupper()和islower()方法分别用于判断字符串是否全为字母或数字、是否全为字母，是否全为数字（0~9）、是否全为数字（支持罗马数字）、是否全为十进制数字、是否为空白字符、是否全为大写和是否全为小写。

```
>>> '12cd'.isalnum()  #判断字母或数字
>>> '12cd'.isalpha()  #判断全为字母
```

3.4 字符串

3.字符串的格式化

字符串的格式化通常结合print()函数使用，以输出指定格式的字符。字符串的格式化有两种方法，占位符（%）和format()方法，占位符方式在Python2.x中使用广泛，Python3.x中format()方法使用更广泛。

（1）使用占位符（%）

格式字符串和格式字符构成一个格式字符串，%之前为格式字符串，之后为需要格式化的内容。格式字符串的形式如下：

<格式字符串>%(<值1>,<值2>,..., <值n>,)

3.4 字符串

3.字符串的格式化

（1）使用占位符（%）

格式字符	说明
%c	字符及其ASCII码
%s	字符串
%%	百分号标记
%d	十进制整数
%o	八进制整数
%x	十六进制整数（用小写字母）
%X	十六进制整数（用大写字母）
%e	指数（以e为底）
%E	指数（以E为底）
%f、%F	浮点数
%g	指数（e）或浮点数（根据显示长度）
%G	指数（E）或浮点数（根据显示长度）

3.4 字符串

（2）使用format()方法

与占位符（%）相比，format()方法拥有更多的功能，操作起来更加灵活方便，可读性更强。该方法将字符串看成一个模板，通过传入的参数进行格式化，使用大括号{}作为特殊字符代替%。

```
>>> print('{} {}'.format('Hello','world'))  #不指定位置，按默认顺序格式化
>>> print('{0} {1}'.format('Hello','world')) #设置位置参数
>>> print('{0} {1} {0}'.format('Hello','world')) #单个参数多次输出
```

3.4 字符串

【例3-5】给定一段英文，输入要查找的单词，统计其在文本中出现的次数并输出；再输入要替换的单词，将查找的单词全部替换，并输出替换后的结果。

3.5 字典

- 字典（Dict）是无序可变序列。字典中的每个元素是一个键值对，包含键（Key）和值（Value）两部分，键和值是相对应的，表示一种映射关系。
- 定义字典时，每个元素的键和值用冒号分隔，相邻元素之间用逗号分隔，所有元素放在一对大括号“{}”中。
- 字典中元素的键可以是任意不可变类型，如整数、实数、复数、字符串和元组等，不能使用列表、集合、字典等可变类型。
- 字典的键是唯一的，而值可以不唯一。

下面的字典都是合法的：

{'语文':89.5, '数学':95, '英语':90, '物理':79, '化学': 78}

3.5 字典

1. 字典的创建和删除

（1）使用赋值语句创建字典

使用赋值运算符“=”将一个字典赋值给一个变量即可创建一个字典变量。

例如：

```
>>> scores={'语文':89.5, '数学':95, '英语':90, '物理':79}
>>> empty_dict={}   #创建一个空字典
```

3.5 字典

（2）使用dict()函数创建字典

使用dict()函数可以将键值对形式的列表或元组创建为字典。因为每个列表或元组将被当成键值对处理，所以这些列表或元组都只能包含两个元素。

- 将键值对形式的列表创建为字典。

```
>>> scores=dict([['语文',89.5],['数学',95],['英语',90],['物理',79]])
```

- 将键值对形式的元组创建为字典。例如：

```
>>> scores=dict((('语文',89.5),('数学',95),('英语',90),('物理',79)))
```

3.5 字典

- 通过指定关键字参数创建字典。

```
>>> scores=dict(语文=89.5,数学=95,英语=90,物理=79)
>>> numbers=dict(one=1,two=2,three=3)
```

- 将多个序列生成字典

使用zip()函数，可以将多个序列作为参数，返回由元组构成的列表。

```
>>> keys=['数学','语文','英语', '物理']
>>> values=[89.5,95,90,79]
>>> kv=zip(keys,values)
```

3.5 字典

（3）使用formkeys()方法创建字典

fromkeys() 方法用于创建一个新字典。其语法格式为：

dict.fromkeys(seq[,value])

其中，参数seq表示字典的键值序列；参数value为可选项，是字典所有键对应的初始值。如果指定值，则所有键具有这个相同的值；如果不指定值，则所有键为空值None。

```
>>> scores={}
>>> scores.fromkeys(['数学','语文','英语', '物理'],0)  #创建值为0的字典
```

3.5 字典

2. 访问字典元素

（1）通过键访问值

字典的键值对是一种映射关系，通过键就可以访问值。字典元素的访问与列表类似，只不过在中括号“[]”里添加的是键而不是索引，如果键不存在会抛出异常。

例如：

```
>>> scores={'语文': 89.5, '数学': 95, '英语': 90, '物理': 79}
>>> scores['数学']  #指定的键存在，返回对应的值
>>> scores['高等数学']  #指定的键不存在，抛出异常
```

3.5 字典

2. 访问字典元素

（1）通过键访问值

字典对象还提供了一个get()方法用来返回指定键对应的值。如果键存在时，返回该键对应的值；如果键不存在时也不会出错，而是返回特定的值。

例如：

```
>>> scores.get('数学')  #指定的键存在，返回对应的值
95
>>> scores.get('高等数学','键不存在')  #指定的键不存在，返回指定的值
'键不存在'
```

3.5 字典

（2）字典的遍历

- 遍历字典的键

结合for循环语句，对字典对象进行遍历时，默认遍历字典的键。

例如：

```
>>> scores={'语文': 89.5, '数学': 95, '英语': 90, '物理': 79}
>>> for item in scores:
...     print(item,end=' ')
```

字典对象也提供了遍历键的keys()方法

```
>>> scores.keys()
```

3.5 字典

（2）字典的遍历

- 遍历字典的值

使用字典对象的values()方法，可以遍历字典元素的值。

```
>>> scores.values()
dict_values([89.5, 95, 90, 79])
```

- 遍历字典元素

使用字典对象的items()方法，可以遍历字典元素。

```
>>> scores.items()
dict_items([('语文', 89.5), ('数学', 95), ('英语', 90), ('物理', 79)])
```

3.5 字典

3. 字典元素的添加与修改

字典创建以后，如果要添加或修改字典元素，可以通过赋值语句实现。形式如下：

dict[key]=value

与字典的访问不同，如果指定的键存在，则修改该键对应的值；如果指定的键不存在，则添加一个新的数组元素。

例如：

```
>>> scores={'语文': 89.5, '数学': 95, '英语': 90, '物理': 79}
>>> scores['语文']=85
>>> scores['化学']=68
```

3.5 字典

字典对象还提供了update()方法，可以将另一个字典的元素一次性全部添加到当前字典。如果两个字典中存在相同的键，则以另一个字典中的值为准对当前字典进行更新。

例如：

```
>>> scores.update({'语文':89.5,'历史':92})  #修改语文的值，添加字典元素'历史':92
>>> scores
{'语文': 89.5, '数学': 95, '英语': 90, '物理': 79, '化学': 68, '生物': 80, '历史': 92}
```

3.5 字典

4. 字典元素及字典的删除

使用del命令可以删除字典中的一个元素，也可以删除整个字典。

例如：

```
>>> scores={'语文': 89.5, '数学': 95, '英语': 90, '物理': 79}
>>> del scores['英语']   #删除指定的一个字典元素
>>> del scores   #删除整个字典
```

3.5 字典

字典对象的pop()方法用于弹出指定的键对应的值，并删除这个字典元素，如果字典为空，则抛出异常；popitem()方法用于弹出字典的一个元素，并将其删除；clear()方法用于清空字典中的所有元素。

例如：

```
>>> scores={'语文': 89.5, '数学': 95, '英语': 90, '物理': 79}
>>> scores.pop('语文')  #弹出指定键对应的值，并删除该元素
89.5
>>> scores.popitem()  #弹出并删除一个字典元素
('物理', 79)
```

3.5 字典

5.字典推导式

从形式上看，字典推导式与列表推导式类似，只是把中括号“[]”改为大括号“{}”。因为字典的一个元素包含key和value两部分，所以表达式也是由两部分组成，其形式如下：

[表达式1：表达式2 for 变量 in 序列 [if 条件表达式]]

其中，表达式1和表达式2分别表示key和value。

例如：

```
>>> fruit=['apple','banana','orange','pear','strawberry']
>>> fruit_dict={k:len(k) for k in fruit}
```

3.5 字典

【例3-6】输入一串字符，统计每个字符出现的次数。

分析：定义一个空字典对象，存放字符和出现次数键值对。遍历字符串中的每一个字符，如果首次出现，则添加到字典，将出现次数置为1；否则该字符的出现次数加1。

【例3-7】用户输入一个人的姓名后，输出其电话号码，如果输入的姓名不在字典中，则输出字符串“Not Found！”。

分析：首先定义一个字典存储人名及其电话号码，通过input()函数输入要查找的人名，使用双分支结构判断，如果人名在字典中则输出相应电话号码，否则输出Not Found！。

3.6 集合

集合（set）是一个无序可变序列，其元素放在一对大括号“{}”中，元素之间用逗号分隔。同一个集合中的元素都是唯一的，不允许重复。集合元素类型只能是数字、字符串、元组等不可变类型，不能是列表、字典、集合等可变类型。

- 集合的基本功能是进行成员测试和删除重复元素。
- 下面的集合都是合法的：

{1, 98, 4, 3.8,120}

{('apple', 5), ('orange', 4.3), ('banana', 4, 5)}

3.6 集合

1. 集合的创建和删除

（1）使用赋值语句创建集合

使用赋值运算符“=”将一个集合赋值给一个变量即可创建一个集合变量。

例如：

```
>>> scores={89.5, 95, 90, 79}   #创建一个集合对象
>>> type(scores)   #查看对象类型
```

3.6 集合

（2）使用set()函数创建集合

使用set()函数可以将字符串、列表、元组、range对象等可迭代对象转换成集合。如果原来的数据存在重复元素，则在转换为集合后只保留一个。

例如：

```
>>> set1=set('college')  #利用字符串创建一个集合
>>> set1
{'e', 'c', 'l', 'o', 'g'}  #自动去掉重复元素
```

注意：创建一个空集合必须用set()创建而不是{}，因为{}是用来创建一个空字典。

3.6 集合

（3）使用frozenset()函数创建不可变集合

使用set()函数创建的集合是可变的，可以添加或删除元素；使用frozenset()函数创建的集合是不可变的，不能进行添加或删除元素的操作。frozenset()函数创建的集合可以作为字典的键，也可以作为其他集合的元素，而set()函数创建的集合不可以。

例如：

```
>>> numbers_set=frozenset('one')  #创建一个不可变集合
```

3.6 集合

2. 访问集合元素

由于集合本身是无序的，也没有任何的键与集合元素对应，所以无法取指定的某个元素，只能使用in、not in或循环遍历来访问或判断集合元素。

例如：

```
>>> set4={'语文', '数学', '英语', '物理'}
>>> '语文' in set4   #使用成员运算符in判断语文是集合元素
True
>>> for i in set4:   #使用for循环遍历集合元素
...     print(i,end=' ')
```

3.6 集合

3. 添加集合元素

使用集合对象的add()方法可以向集合中添加一个元素，如果该元素已存在则忽略该操作；使用update()方法可以向集合中添加另外一个集合中的多个元素，并自动去除重复元素。

例如：

```
>>> set4={'语文', '数学', '英语', '物理'}
>>> set4.add('化学')   #使用add()方法添加一个元素
>>> set4.add('英语')   #元素存在忽略该操作，不会抛出异常
```

3.6 集合

4. 删除集合元素

（1）remove()方法

使用remove()方法删除元素时，如果元素不存在，会抛出异常。

例如：

```
>>> set4={'语文', '数学', '英语', '物理'}
>>> set4.remove('物理')  #删除元素物理
>>> set4.remove('化学')  #元素不存在，抛出异常
```

3.6 集合

（3）pop()方法

使用集合对象的pop()方法随机删除集合中任意一个元素并返回该元素。

```
>>> set4={'语文', '数学', '英语', '物理'}
>>> set4.pop()   #随机删除并返回一个无素
'英语'
```

（4）clear()方法

使用集合对象的clear()方法删除集合的所有元素。

```
>>> set4.clear()
>>> set4
set()   #空集合
```

3.6 集合

（2）discard()方法

使用集合对象的discard()方法删除集合元素，如果元素不存在则忽略此操作，不会抛出异常。

例如：

```
>>> set4={'语文', '数学', '英语', '物理'}
>>> set4.discard('物理')  #删除元素物理
```

3.6 集合

5. 集合运算

集合还支持典型的数学集合运算并、交、差等，可用相应的操作符和方法实现。

（1）并集

两个集合A、B的并集是A、B集合的所有元素构成的集合。使用操作符“|”执行并集操作，也可使用union()方法实现。例如：

```
>>> a_set=set([1,2,3,4,5])
>>> b_set=set([3,4,5,6,7])
>>> a_set|b_set   #使用运算符“|”实现并操作
>>> a_set.union(b_set)   #使用union()方法实现并操作
```

3.6 集合

（2）交集

两个集合A、B的交集是含有所有既属于A又属于B的元素的集合。使用“&”操作符执行交集操作，也可使用intersection()方法实现。

例如：

```
>>> a_set&b_set   #使用运算符“&”实现交操作
>>> a_set.intersection(b_set)   #使用intersection()方法实现交操作
```

3.6 集合

（3）差集

集合A与B的差集是所有属于A且不属于B的元素构成的集合。使用操作符“-“执行差集操作，也可使用difference()方法实现。

例如：

```
>>> a_set-b_set    #使用运算符“-”实现差操作
>>> b_set.difference(a_set)   #使用difference()方法实现差操作
>>> b_set.difference(b_set)   #一个集合和本身的差集为空集
```

3.6 集合

（4）对称差集

两个集合的对称差集是只属于其中一个集合而不属于另一个集合的元素，即两个集合的相异元素构成的集合。使用“^”操作符执行差集操作，也可使用symmetric_difference()方法实现。

例如：

```
>>> a_set^b_set   #使用运算符“^”实现对称差操作
>>> b_set.symmetric_difference(a_set)
```

3.6 集合

（5）集合的比较

- 判断集合相等与不相等

如果集合A、B具有相同的元素，则两集合相等，返回True，否则返回False。

例如：

```
>>> a_set={1,2,3,4,5}
>>> b_set={2,3,5,1,4}
>>> a_set==b_set#使用关系运算符判断是否相等
```

3.6 集合

（5）集合的比较

- 判断子集与真子集

如果集合A中的元素都是B的元素，且A不等于B，则A是B的真子集。

```
>>> a_set={1,2,3,4,5}
>>> b_set={1,2,3,4,5,6,7}
>>> a_set<b_set   #判断真子集
```

如果集合A中的元素都是B的元素，则A是B的子集。

```
>>> c_set=a_set
>>> a_set<=c_set   #判断子集
```

3.6 集合

（5）集合的比较

- 判断超集与真超集

如果集合B中的元素都是A的元素，且A不等于B，则A是B的真超集。

```
>>> b_set>a_set   #判断真超集
```

如果集合B中的元素都是A的元素，则A是B的超集。

```
>>> b_set>=a_set   #判断超集
```

3.6 集合

（5）集合的比较

- 判断超集与真超集

如果集合B中的元素都是A的元素，且A不等于B，则A是B的真超集。

```
>>> b_set>a_set   #判断真超集
```

如果集合B中的元素都是A的元素，则A是B的超集。

```
>>> b_set>=a_set   #判断超集
```

注意：关系运算符“==”，只需要判断两个集合中的元素是否一致，与顺序无关。关系运算符“>、>=、<、<=”作用于集合时，表示集合之间的包含关系，而不是比较集合中元素的大小关系。

3.6 集合

6.集合推导式

集合推导式与列表推导式类似，唯一的区别是它使用大括号“{}”，使用集合推导式可以快速创建特定集合。

例如：

```
>>> squared_set={x**2 for x in[1,2,3,4]}  #利用集合推导式创建特定集合
```

3.6 集合

【例3-8】编程统计商品的销售情况：（1）有人购买的商品（2）无人购买的商品。（3）每个人都购买的商品，输出统计后的商品列表。

分析：定义一个字典，存放商品编号和商品名称的键值对。用列表定义3个用户购买商品的清单，按要求进行统计并输出统计结果。

3.6 集合

【例3-9】小明想在学校中做一项问卷调查，为了实验的客观性，他先用计算机生成了N个1~1000之间的随机整数，对于其中重复的数字只保留一个，不同的数对应着不同学生的学号，然后再把这些数从小到大排序，按照排好的顺序去找同学做调查，请编程完成去重和排序工作。

分析：采用集合存储随机数自动去重，利用循环结构，没生成一个随机数添加到集合中即可。然后用集合的sorted()函数对集合进行升序排序。

3.7 序列解包

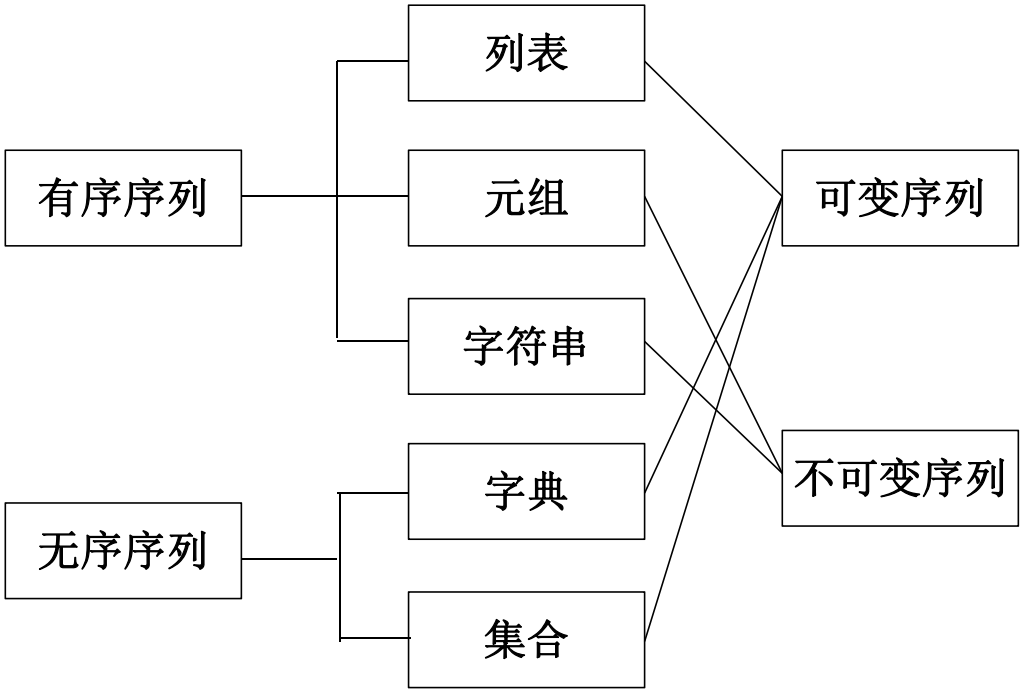
Python序列解包是将一个序列（或任何可迭代的对象）解包，并将得到的值存储在一系列变量中。简单的说，就是一次给多个变量赋值。序列解包是Python中非常重要和常用的一个功能，在赋值时更方便、更简洁，能有效的优化程序。

例如：

```
>>> a,b,c=1,2,3   #给多个变量赋值
>>> a,b=b,a   #交换两个变量的值
>>> a,b,c=1,2   #序列元素个数与变量个数不等，抛出异常
```

本章小结

1.组合数据类型简介
2.列表
3.元组
4.字符串
5.字典
6.集合
7.序列解包





第4章 Python控制结构



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



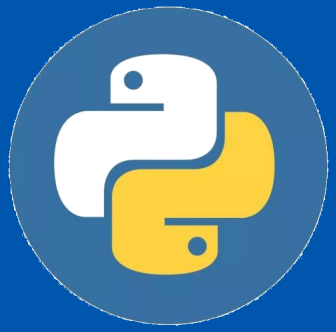
西北工业大学
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

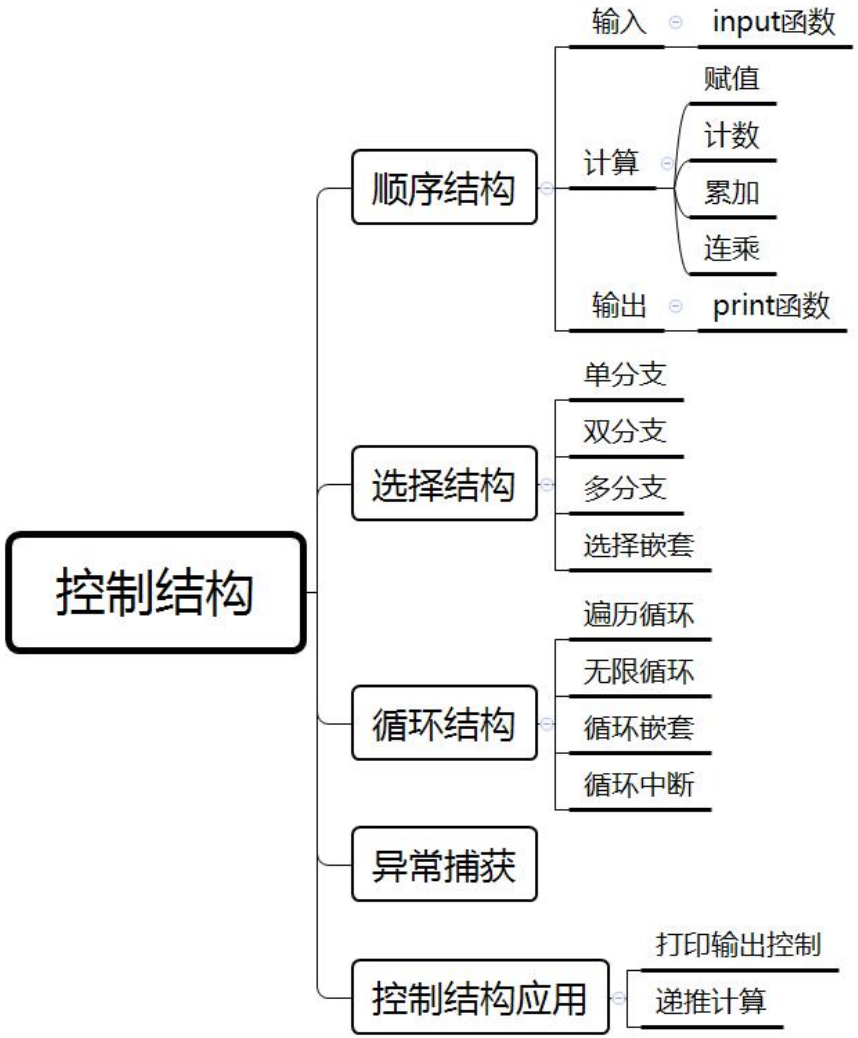
QQ 群：　Python程序设计
760581125



《Python程序设计》

第4章　Python控制结构

第4章 Python控制结构





第4章 Python控制结构

4.1 Python顺序结构

4.2 Python选择结构

4.3 Python循环结构

4.4 Python控制结构应用举例



4.1 Python顺序结构

- 程序与算法
- 控制结构概述
- 输入
- 计算
- 输出



程序与算法

程序：计算机能够识别和执行的指令集合。程序类似于任何自然语言写的文章，是对事物运动状态及其规律的描述，只不过是写给计算机来阅读并严格遵照执行的任务书。

程序=算法+数据结构

算法：对特定问题求解步骤的一种描述，计算机算法是指令的有限序列，其中每一条指令表示一个或多个操作。

算法特征：

- 确定性
- 可行性
- 有穷性
- 0个以上输入
- 1个以上输出

算法评价标准：

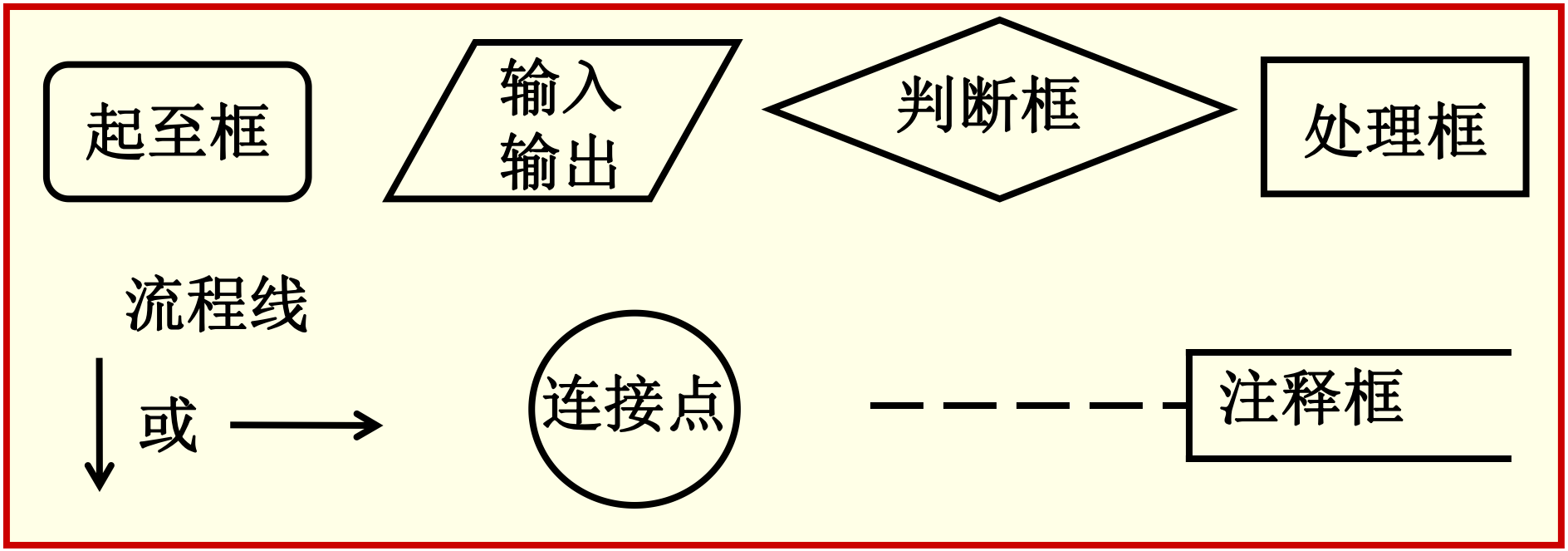
- 正确性
- 可读性
- 健壮性（鲁棒性）
- 高效率与低存储要求

程序与算法

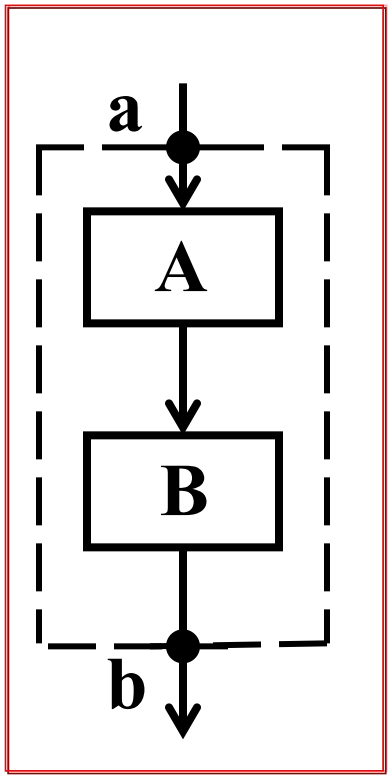
算法的表示方法：自然语言、流程图、N-S图、伪代码以及程序。

（1）自然语言：人们日常生活中的语言，如书面语言和口头语言，用自然语言通俗易懂，容易被人接受。

（2）流程图的基本构成：

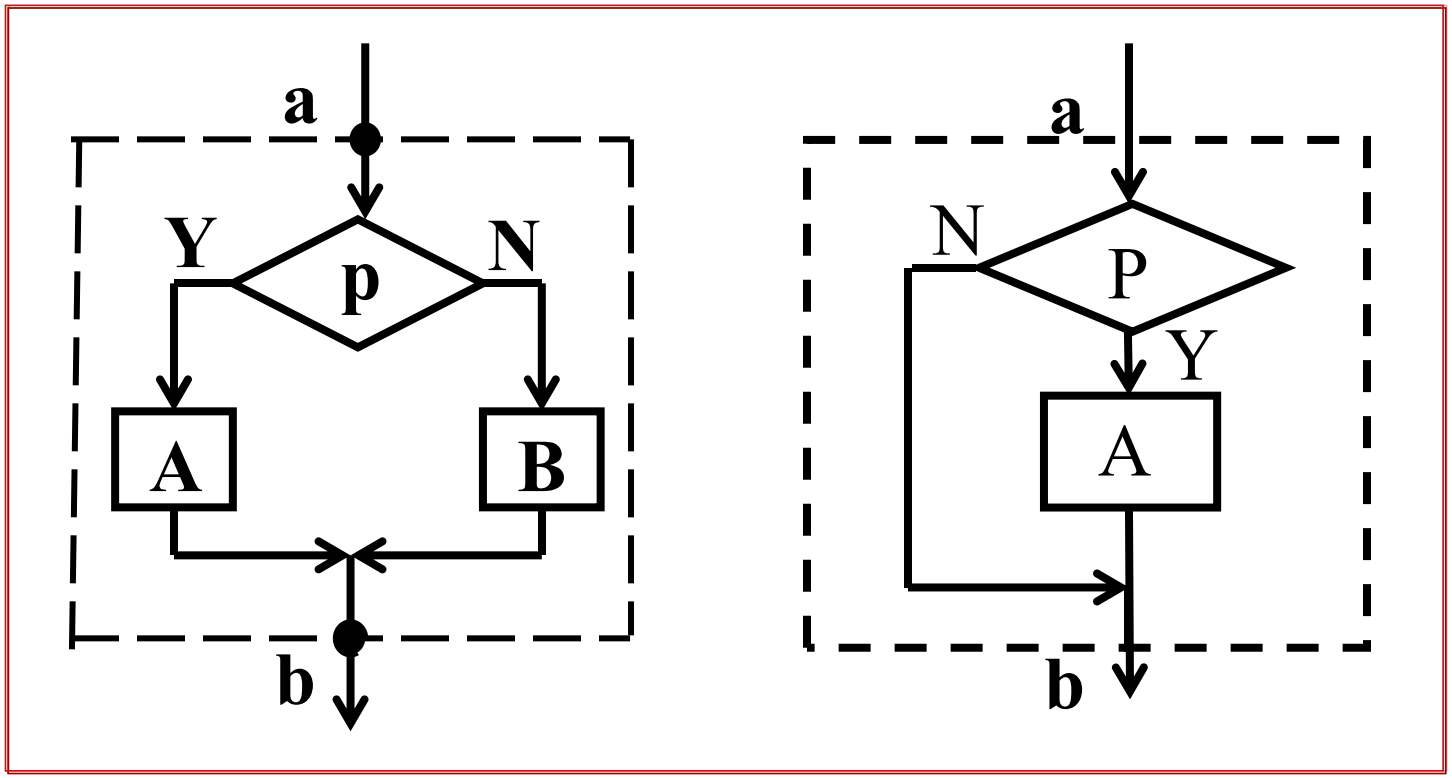


流程图的三种基本结构



顺序结构

先执行A,
再执行B.

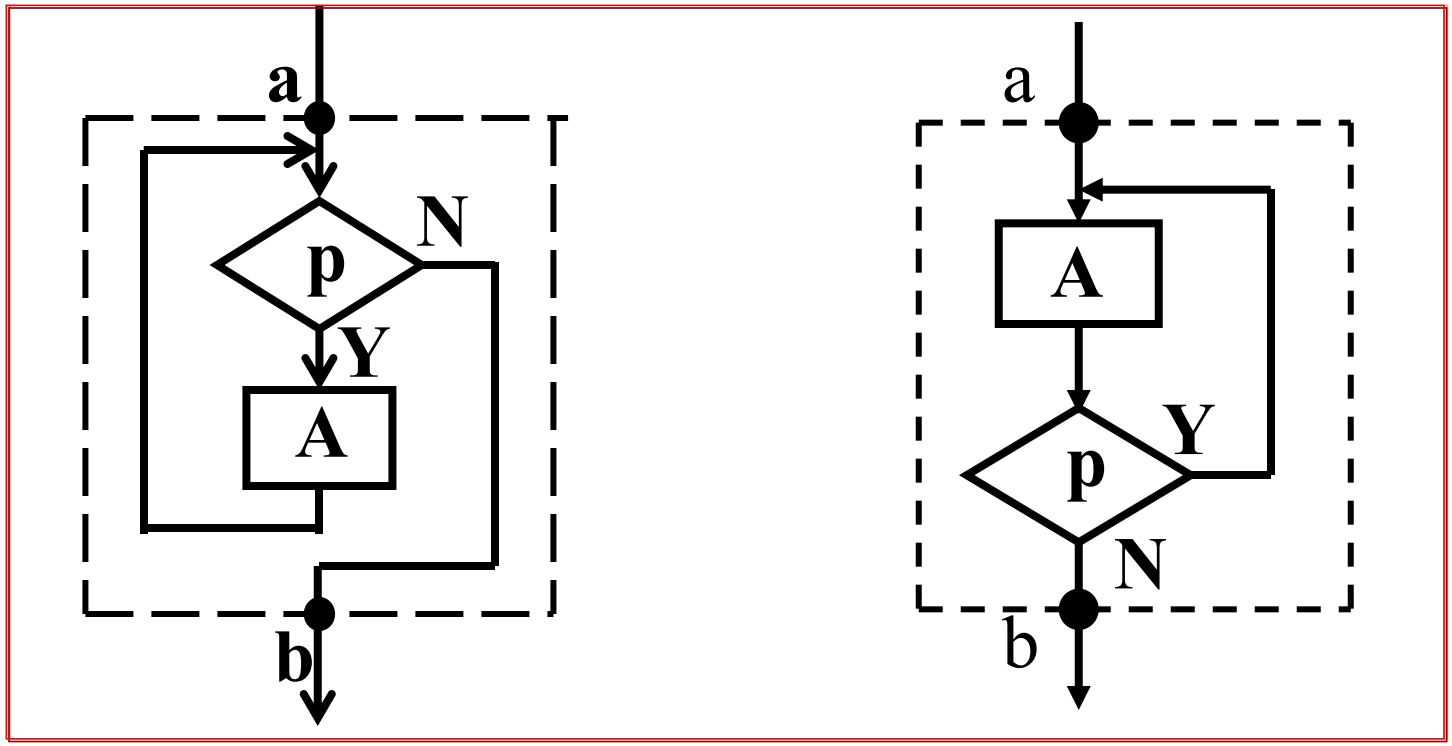


选择结构

若P为真,则执行A,
否则执行B.

若P为真,则执行A,
否则跳过A.

流程图的三种基本结构



当型循环结构

当P为真,反复执行A,
P为假时出循环。

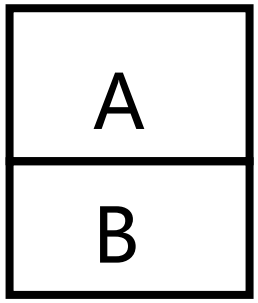
直到型循环

先执行A,再判断,
若P为真,反复执行A,
P为假时出循环。

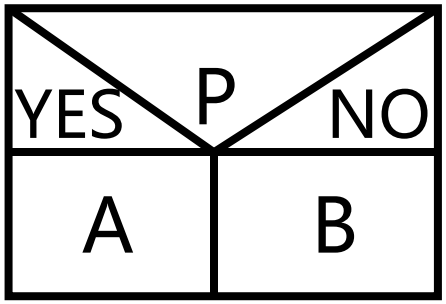
N-S图的三种基本结构

（3）N-S结构流程图

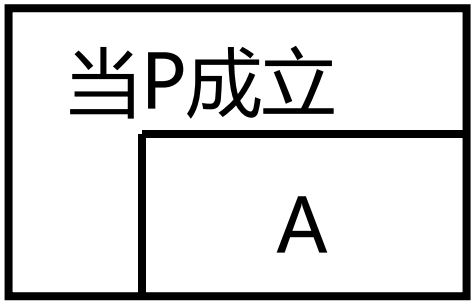
N-S流程图去掉了箭头，算法写在一个矩形框里。它也有三种结构：顺序、选择和循环



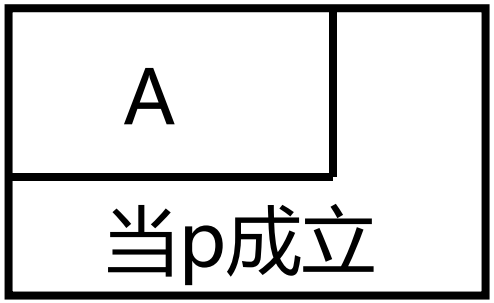
顺序结构



选择结构



当型循环



直到型循环

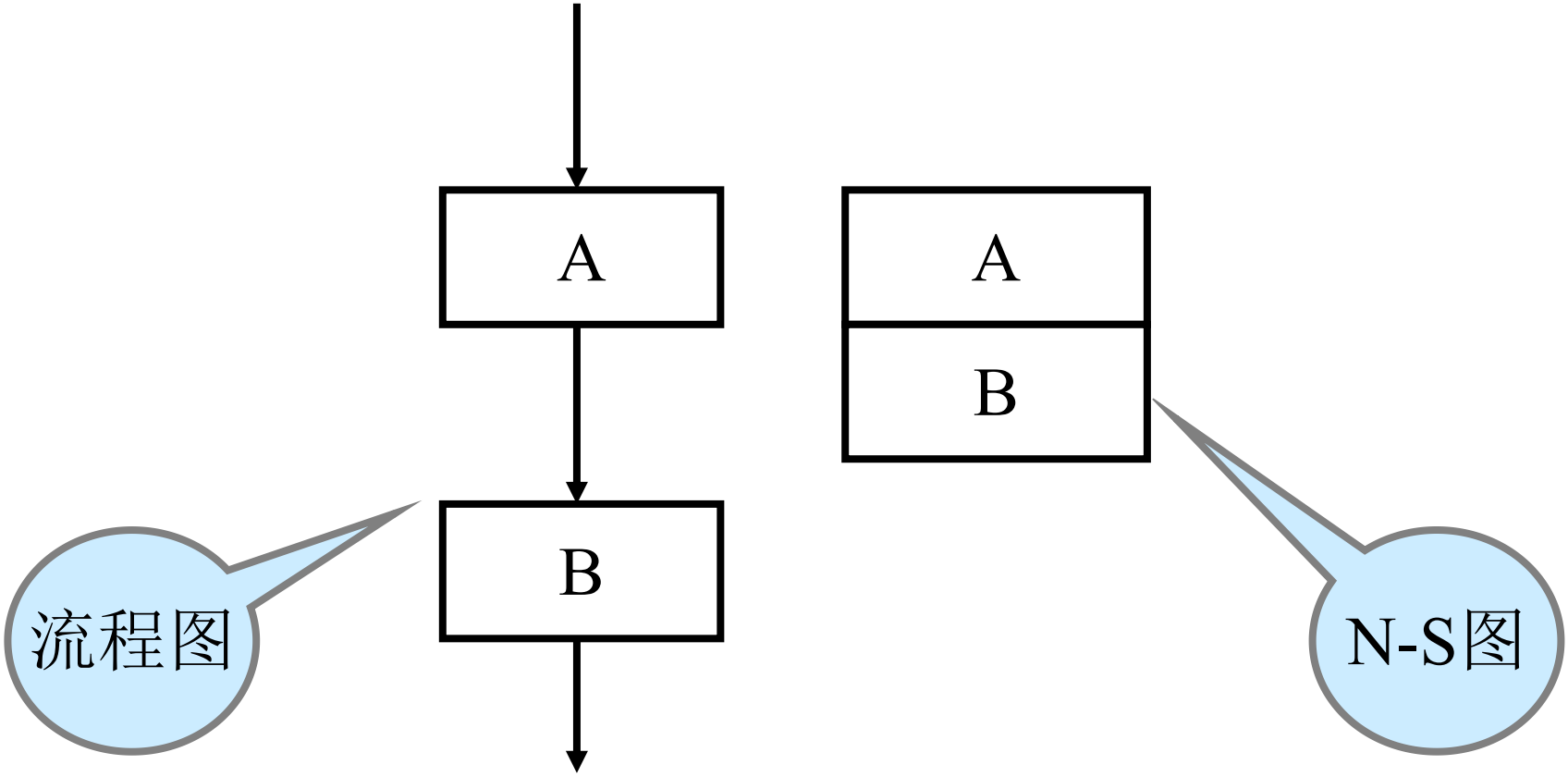


程序的基本控制结构

每一种编程语言都会提供一套算法描述机制，称为程序控制结构，其基本原理相同，具体语法形式不同而已。

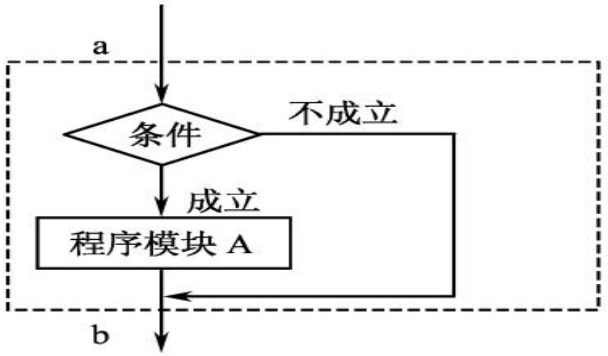
“顺序”、“选择”和“循环”构成计算机程序的三种基本控制结构，通过有限次的组合与嵌套来描述丰富多彩的事物运动规律，解决纷繁复杂的各种问题。

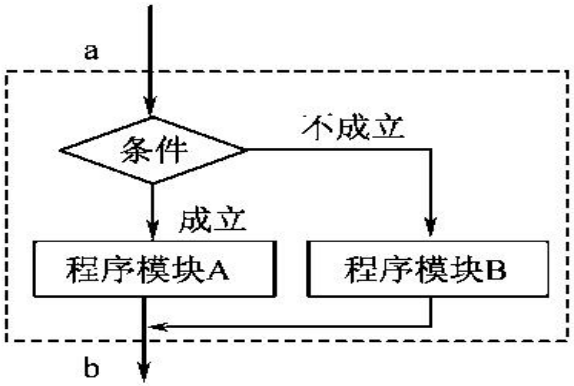
顺序结构

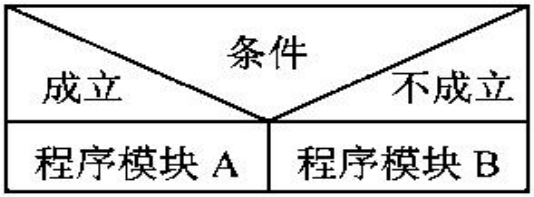


选择结构

■分支选择结构

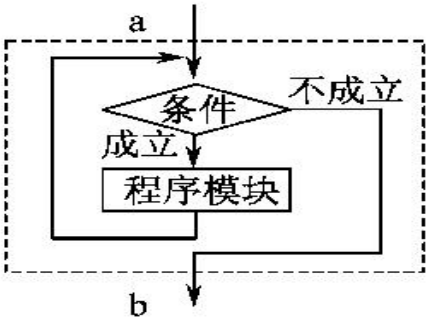

a
条件
不成立
成立
程序模块 A
b


a
条件
不成立
成立
程序模块A
程序模块B
b

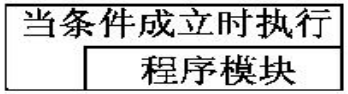

条件
成立
不成立
程序模块 A
程序模块 B

循环结构

■当型循环

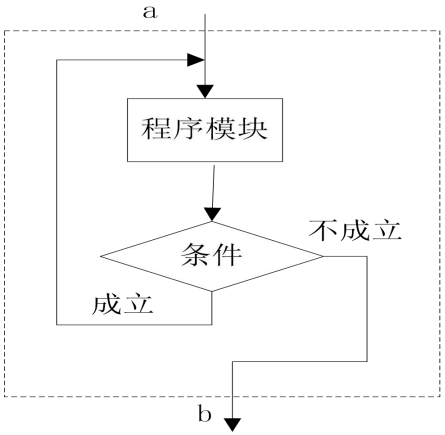


(a)

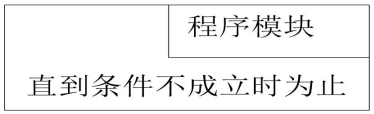


(b)

■直到型循环



(a)



(b)



三种基本结构的特点

- 共同特点：单入口、单出口
- 三种结构之间可以是顺序关系、平行关系，也可以互相嵌套，通过结构之间的复合形成复杂的关系。

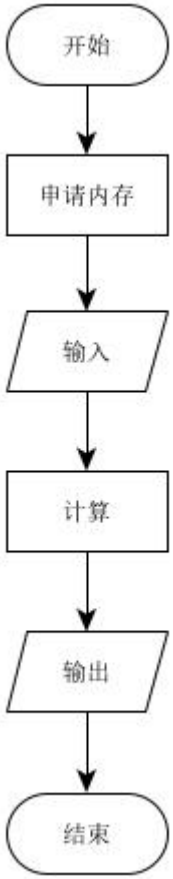
4.1 顺序结构

顺序结构的三种基本操作：输入、输出和计算

程序设计的框架：

- 申请内存
- 输入
- 计算
- 输出

基本流程如右图所示：



4.1 顺序结构

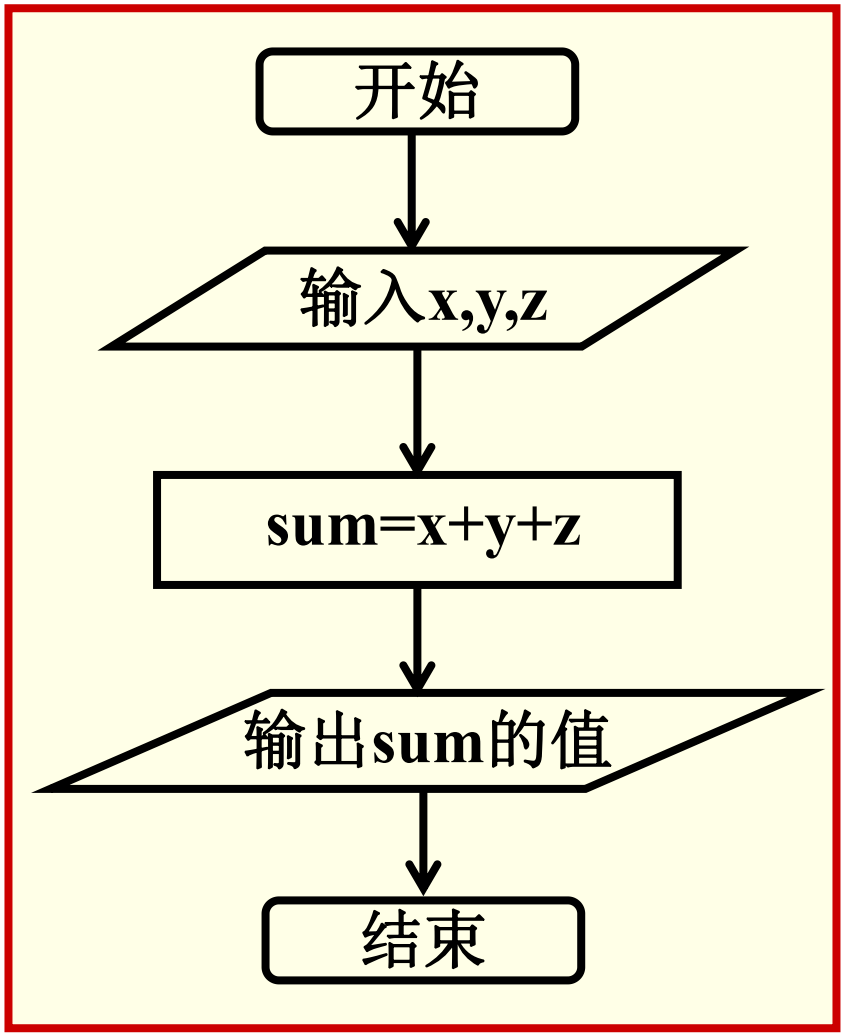
例1：求三个整数的和。

分析：

- 申请内存：定义4个变量x、y、z、sum
- 输入：赋值给x、y、z
- 计算：求和
- 输出：sum

输入x,y,z
sum=x+y+z
输出sum的值

N-S结构流程图



流程图



4.1.1 输入

标准输入：向计算机输入数据是人机交互最基本的操作，有多种方式，如鼠标、键盘、扫描仪、话筒、数据文件等等。Python中有一个内置函数input()，实现如何通过键盘向计算机输入数据。

Input函数的用法如下：

str=input([prompt])

其中：

- str为函数的返回值，字符串类型
- prompt为输入时系统给出的提示信息
- 如果要输入数字，需要将str转换成数值型数据



标准输入：input()

如果要输入数值类型数据，可以使用类型转换函数将字符串转换为数值。

```
>>> x=int(input(“please input your input:”))
please input your input:5
>>>print(type(x))
<class'int'>
```

说明：x接受的是字符串“5”，通过int（）函数将字符串转换为整型。



标准输入：input()

在Python2.x中，函数返回结果的类型由输入时使用的界定符来决定。

```
>>>x=input("Please enter your input: ")
Please enter your input: 5
>>>x=input("Please enter your input: ")
Please enter your input: '5'
>>>x=input("Please enter your input: ")
Please enter your input: [1,2,3]
>>>x=input("Please enter your input: ")
Please enter your input: (1,2,3)
```



标准输出:print()

在Python2.x中使用的是print语句输出

在Python3.x中使用的是print()函数输出

print()函数的一般形式为：

print（[输出项1，输出项2，....,输出项n][,sep=分隔符][,end=结束符]）

说明：输出项之间用逗号分隔，没有输出项时输出一个空行。sep表示输出时候各个输出项之间的分隔符，end表示输出时的结束符。



标准输出:print()

print函数从左求出至右各输出项的值，并将各输出项的值依次显示在屏幕的同一行上。

```
>>>x,y=2,3
>>>print(x,y)
2 3
>>>print(x,y,sep=':')
2:3
>>>print(x,y,sep=':',end='%)
2:3%
```



4.1.2 输出

字符串格式化%

格式控制字符串%（输出项1，输出项2，...输出项）

（1）常规字符：包括可显示的字符和用转义字符表示的字符。

（2）格式控制符：以%开头的一个或多个字符，以说明输出数据的类型、形式、长度、小数位数等。

例如：

print（”sum=%d”%x)



print() 格式化说明

1	d或i	十进制格式
2	o	八进制格式
3	x、X	十六进制格式
4	c	一个字符格式
5	s	字符串格式
6	f	实数格式
7	e或E	指数格式
8	g或G	e和f中较短一种格式



附加格式说明符

m：域宽，十进制整数，用以描述输出数据所占宽度。

n：附加域宽，十进制整数，用于指定实型数据小数部分的输出位数。

—：输出数据左对齐，默认为右对齐

+：输出正数时，也以+号开头

#：作为o，x的前缀时，输出结果前面加上前导符号0，,0x



格式化输出：

```
>>>year = 2017
>>>month = 1
>>>day = 28
>> print('%04d-%02d-%02d'%(year,month,day))
2017-01-28
 >>>value = 8.123
>> print('%06.2f'%value)
008.12
>>>print('%d'%10)
10
>>>print('%o'%10)
12
```

```
>>>print('%02x'%10)
0a
>>>print('%04X'%10)
000A
>>>print('%.2e'%1.2888)
1.29e+00
```



4.1.3 计算

对计算机来说，不论是数值计算，还是信息处理，归根结底都是在计算，因此计算操作语句是整个程序的主体，计算主要有算术运算和逻辑运算，都可归结为赋值语句，都是顺序结构程序。

赋值语句的一般形式：

变量名 = 表达式（常数、变量、函数、对象等）

赋值语句的主要作用是将表达式的值赋值给变量名，一般用于给变量赋值、复制对象或取对象的地址引用。



4.1.3 计算

单变量赋值举例

var=1.58　　#给单变量赋值

L1=[1,2,3,4,5]　　#给列表对象赋值

L2=L1　　#复制对象

在赋值时，Pyhon进行动态类型转换，变量的数据类型按照赋值的类型而动态变化，如：

```
>>>v1="3.6" ↵
>>>print(v1) ↵
3.6
>>>type(v1) ↵
str
```

```
>>>v2=8↵
>>>v1=v2↵
>>>print(v1) ↵
8
>>>type(v1)↵
int
```

```
>>>v3=1.58↵
>>>v1=v3↵
>>>print(v1) ↵
1.58
>>>type(v1)↵
float
```



4.1.3 计算

多变量赋值举例

多元变量赋值	列表赋值	通用序列赋值	链式变量赋值
>>>x,y=10,20 >>>print(x,y) 10 20 >>>(x,y)=(10,20) >>>print(x,y) 10 20 >>>x,y=[10,20] >>>z=(x,y) >>>print(z) (10, 20) >>>type(z) tuple	>>>[x,y]=[10,20] >>>print([x,y]) [10, 20] >>>[x,y]=10,20 >>>print(x,y) 10 20 >>>z=[x,y] >>>type(z) list	>>>x,y='pi' >>>print([x,y]) ['p', 'i'] >>>print(x,y) p i >>>z=x,y >>>print(z) ('p', 'i') >>>type(z) tuple	>>>x=y=z=10 >>>print(x,y,z) 10 10 10 扩展序列解包 >>>x,*y = 'Python' >>>print(x,y) P ['y', 't', 'h', 'o', 'n']



4.1.3 计算

常用计算赋值：计数

赋值语句	计算机操作名称	计算机操作	数学方程
n=n+1	计数赋值	Read:n;Compute:n+1;Write:n	无解
sn=sn+an	累加赋值	Read:sn,an;Compute:sn+an;Write:sn	an=0
pn=pn*an	连乘赋值	Read:pn,an;Compute:pn*an;Write:pn	an=1
x=sin(x)	函数赋值	Read:x;Compute:sin(x);Write:x	0

n=n+1

这个式子在数学方程里是无解的，但在计算机里却是计算的核心，它是由三个操作组成的，第一步，从内存n里读取数值；第二步，计算n+1的和；第三步，将计算结果写回到内存n。因此，它是**“读→计算→写”**的操作组合。

n=n+1又称为计数操作



4.1.3 计算

常用计算赋值：累加

例：求1～100的自然数之和。计算机用以下语句完成：

```
sn=0
for an in range(1,101):
    sn=sn+an
print(sn)
```

结果输出为：5050

可以看出，共本程序进行了100次累加，进行了100次**“读→计算→写”**的操作，但只用了一个式子sn=sn+an。也就是说，计算机程序只是一个操作的模式，不断的重复计算是计算机擅长的事情。



4.1.3 计算

常用计算赋值：连乘

例：求10的阶乘。计算机用以下语句完成：

```
pn=1
for an in range(1,11):
    pn=pn*an
print(pn)
```

结果输出为：3628800

可以看出，共本程序进行了10次连乘，进行了10次**“读→计算→写”**的操作，阶乘对于手工计算的工作量并不少，哪怕只是10的阶乘。但计算机只用了一个式子pn=pn*an。



4.1.3 计算

常用计算赋值：函数

【例4-6】如果x=π/6,则求sin(x)。计算机用以下语句完成：

```
import math
x=3.14*30/180
x=math.sin(x)
print(x)
```

结果输出为：0.4997701026431024

顺序程序设计举例

例1：从键盘输入一个3位整数，分离出它的个位、十位和百位并分别在屏幕输出。

程序运行结果如下：

请输入一个3位整数:235

百位=2，十位=3，个位=5

```
x=int(input(“请输入一个3位整数:”))
a=x//100
b=(x-a*100)//10
c=x%10
print(“百位=%d，十位=%d，个位=%d”%(a,b,c))
```



顺序程序设计举例

例2：已知三角形三条边的边长，求三角形的面积。

分析：三角形面积= $\sqrt{p(p-a)(p-b)(p-c)}$

其中，a、b、c是三角形三边的边长，p是三角形周长的一半。

自然语言描述：

1.申请内存：定义五个变量作为三角形三边长a、b、c，周长的一半p，面积s；

3.输入：输入三角形三边长；

4.计算：周长的一半p和面积s;

5.输出：面积



顺序程序设计举例

```
import math
a = float(input("请输入三角形的边长a："))
b = float(input("请输入三角形的边长b："))
c = float(input("请输入三角形的边长c："))
p = (a + b + c) / 2
s = math.sqrt(p*(p-a)*(p-b)*(p-c));
print(str.format("三角形的面积=%f"%s))
```



小结

- 理解程序和算法的概念。
- 掌握描述算法的图形工具：流程图。

  掌握使用流程图描述算法的方法。
- 了解程序流程的三种基本结构：顺序结构、选择结构、循环结构。
- 掌握常用的I/O函数，重点掌握input、print函数的控制格式。



作业

① 输入长方形的长和宽，计算面积并输出。

② 输入一个3位整数n，输出其逆序数m。例如输入n=128,则m=821。

③ 输入三个浮点数，求它们的平均值并保留一位小数输出。



第4章 Python控制结构



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



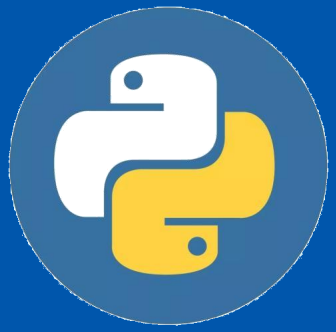
西北工业大学
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

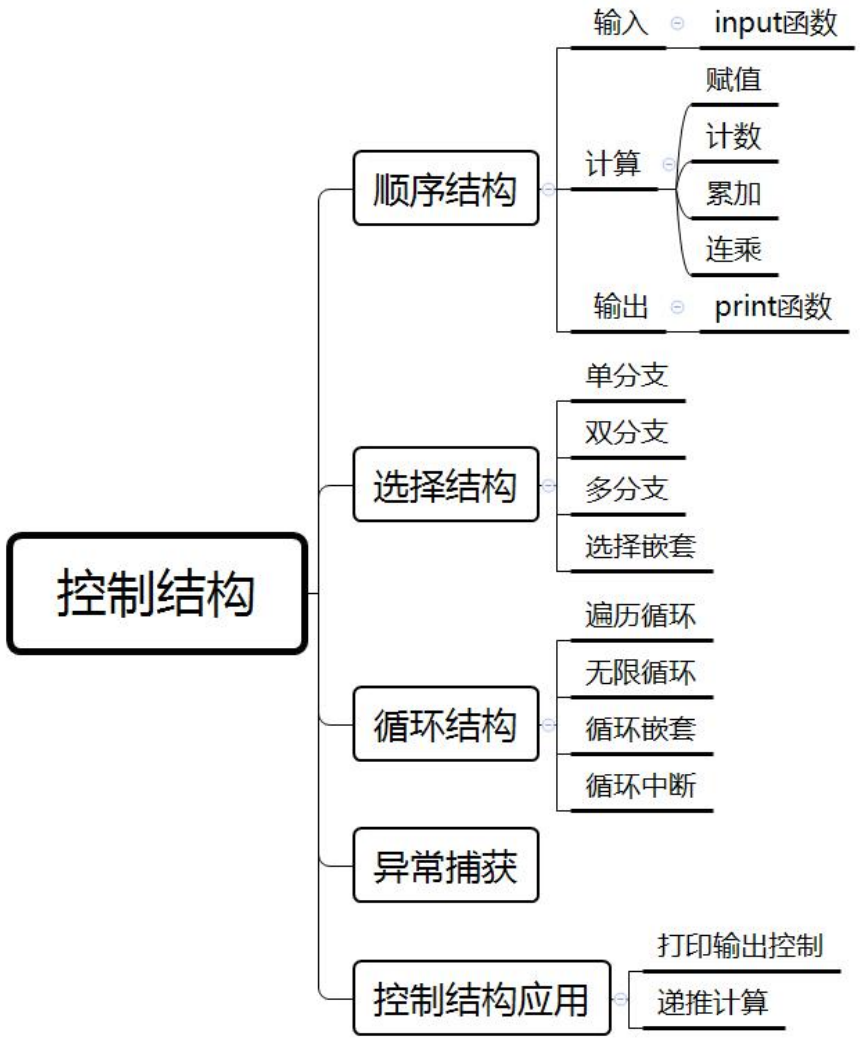
QQ 群：　Python程序设计
760581125



《Python程序设计》

第4章　Python控制结构

第4章 Python控制结构





第4章 Python控制结构

4.1 Python顺序结构

4.2 **Python选择结构**

4.3 Python循环结构

4.4 Python控制结构应用举例



4.2 Python选择结构

- if 单分支选择结构语句
- if...else 双分支选择结构语句
- if...elif...elif...else 多分支选择结构语句
- if 嵌套 选择嵌套结构语句

单分支选择结构

单分支选择结构用来描述只有一个条件，决定程序的执行与否，其自然语言的表述为：

如果<满足条件>

那么<执行相应任务>

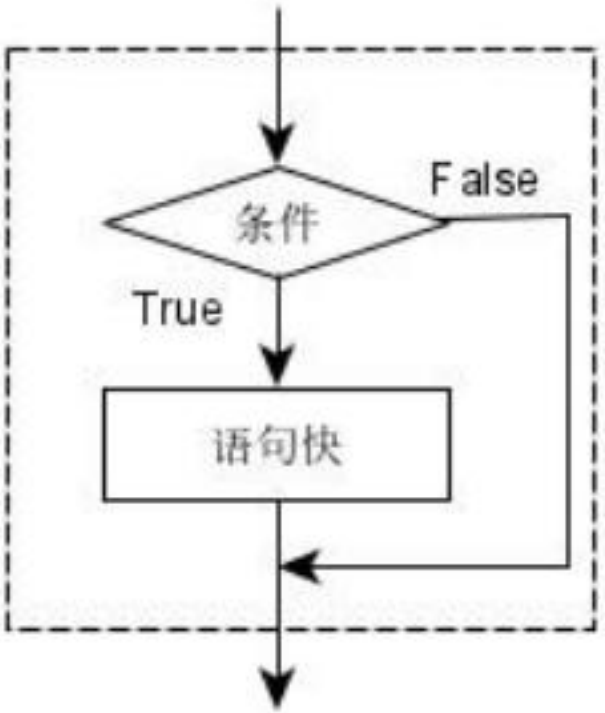
如果<不满足条件>

那么<跳过此处继续>

Python的语法格式为：

if<条件表达式>：

语句块



单分支选择结构

if<条件表达式>：
**　　语句块**



当程序执行到此语句时，首先检查“条件表达式”的值是否为True。如果是True，则执行冒号后的“语句块”，然后接着执行下面的语句。如果“条件表达式”的值为False，则不执行“语句块”中的任何语句，直接跳到下一条语句执行。

双分支选择结构

双分支选择结构也是用来描述只有一个条件的情况，但是决定程序执行两个方面，其自然语言的表述为：

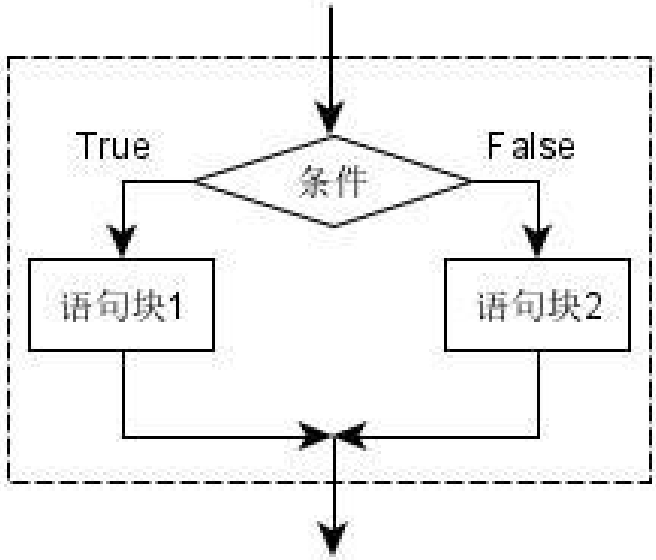
如果<满足条件>

那么<执行相应任务1>

否则<执行相应任务2>

Python的语法格式为：

```
if<条件表达式>:
    语句块1(条件为真时要执行的语句块)
else:
    语句块2(条件为假时要执行的语句块)
```





多分支选择结构

多分支选择结构是用来解决多条件判断多情况讨论问题而设计的，其自然语言的表述为：

如果<满足条件1>

那么<执行相应任务1>，

否则如果<满足2>**那么**<执行相应任务2>，

否则如果...**那么**...，

否则如果<满足条件n>**那么**<执行相应任务n>，

否则如果<前边条件都不满足>

那么<如果有安排，则执行其它任务，否则什么都不做，结束判断>



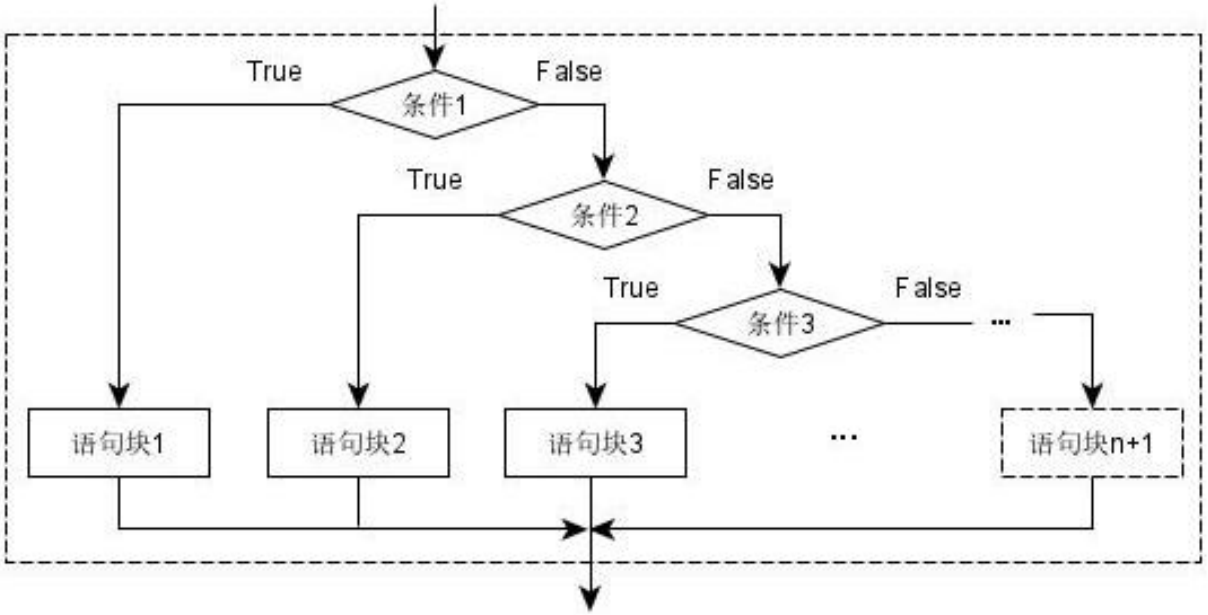
多分支选择结构

Python的语法格式为：

```
if <条件表达式1>：
    语句块1
elif <条件表达式2>：
    语句块2
…
elif <条件表达式n>：
    语句块n
[else：
    语句块n+1]
```

当程序执行到if … elif … else …语句时，首先判断条件1，如果其值为True，执行语句块1，然后结束整个选择结构；如果条件1的值为False，则判断条件2，如果其值为True，执行语句块2，然后结束整个选择结构；如果条件2的值为False，继续往下判断其他条件的值；如果所有条件的值都为False，则执行else之后的语句块n+1，如果没有else，则结束整个选择结构。

多分支选择结构



也就是说，从上到下，第几个条件表达式值为True，就执行相应的第几个语句块，然后结束整个选择结构，执行下面的语句。如果有不止一个条件表达式的值为True，只有最上面的条件所对应的语句块被执行。如果所有的条件均不成立，而又没有else语句，则不执行本结构中的任何语句块，直接执行选择结构下面的语句。



单分支选择结构举例

例：将小写字母转换为大写字母

```
c=input("Please input character:")
if c>='a' and c<='z':
        c=ord(c)-32
print("%c\n"%c)
```

思考：
若将大写字母转换为小写字母.

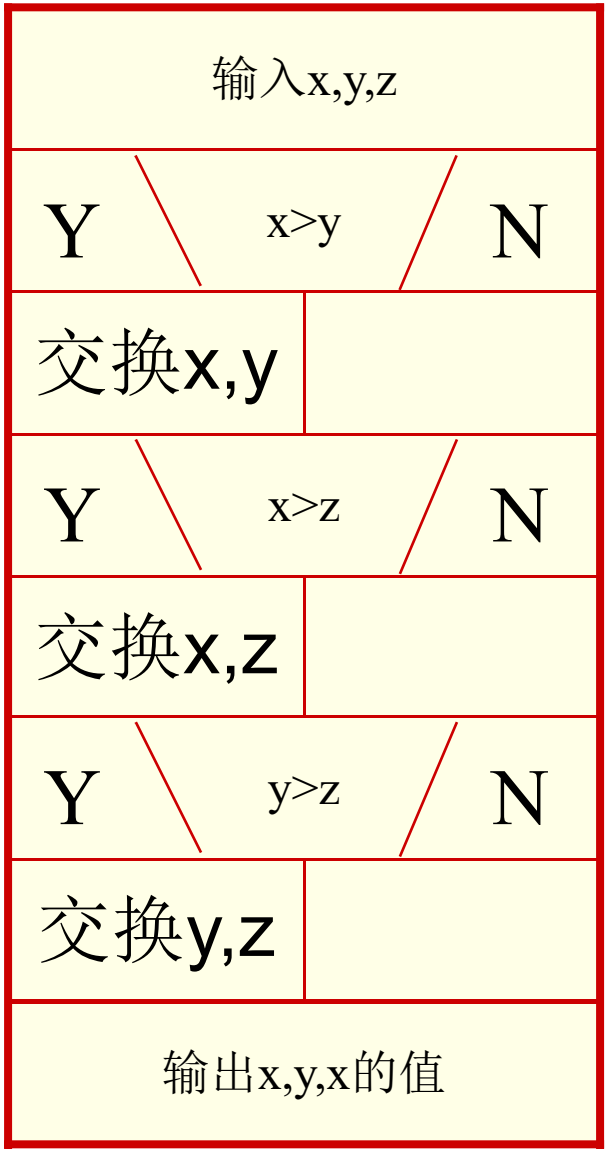
单分支选择结构举例

例: 输入三个整数x,y,z，把这三个数由小到大输出

```
x,y,z=eval(input('请输入x、y、z:'))
if x>y:
    x,y=y,x
if x>z:
    x,z=z,x
if y>z:
    y,z=z,y
print (x,y,z)
```

思考：

- 如果要将三个数从大到小输出，程序该如何编写？



双分支选择结构举例

例:求输入的两个整数的最小值

分析：对两个数进行比较，取小数。

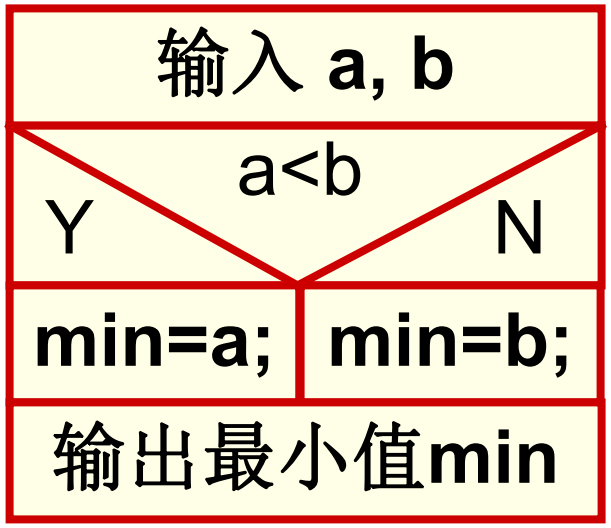
自然语言描述：

申请内存：定义变量x,y

输入：从外设输入两数赋值给x,y

计算：比较x,y,如果x>y，则交换，取x,否则，如果x<y，直接取x

输出：最小值x

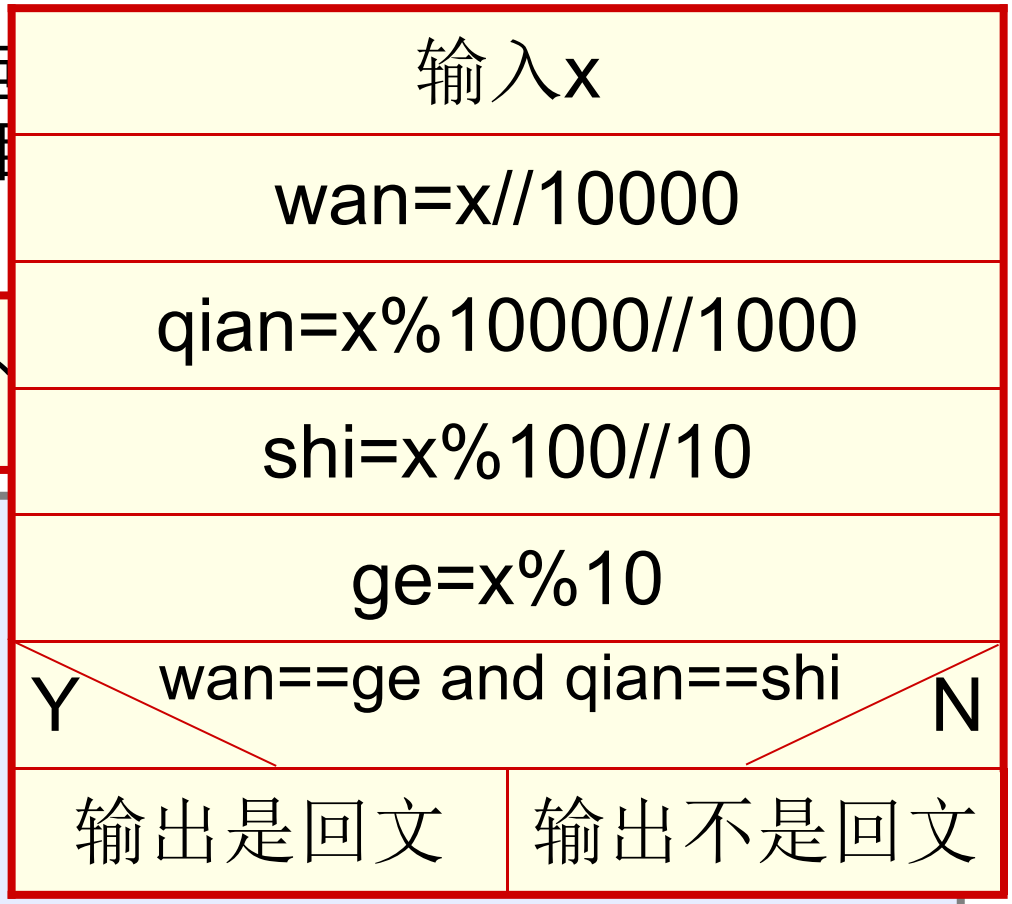



```
x,y=eval(input('请输入x、y:'))
if a<b:
        min=a
else:
        min=b
printf("The minimum is %d"%min)
```

双分支选择结构举例

例:一个5位数，判断它是不是回
12321是回文数，个位与万位相
千位相同。

分析：分解出每一位数（万位、
和个位），然后判断。

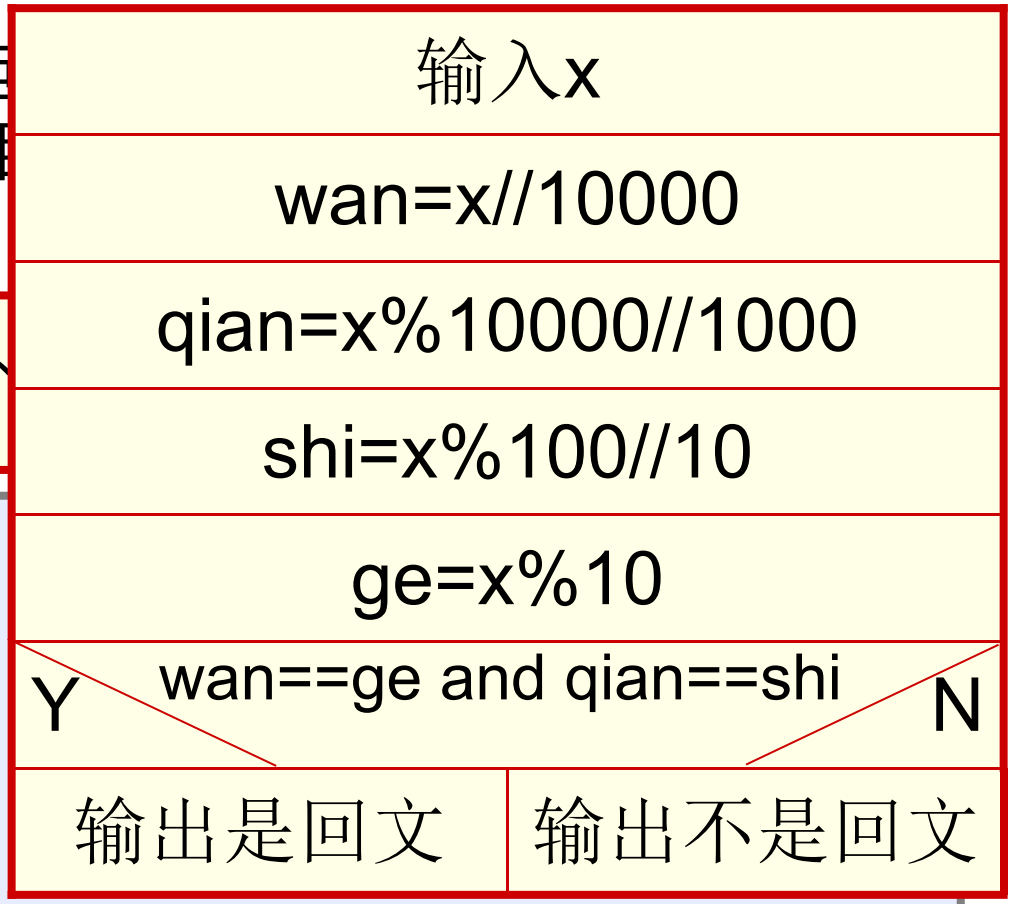



```
x=int(input('请输入x:'))
wan=x//10000;
qian=x%10000//1000;
shi=x%100//10;
ge=x%10;
if ge==wan and shi==qian:
    print("It is palindromic number!\n")
else:
    print("It is not palindromic number!\n")
```

双分支选择结构举例

例:一个5位数，判断它是不是回
12321是回文数，个位与万位相
千位相同。

分析：分解出每一位数（万位、
和个位），然后判断。




```
x=int(input('请输入x:'))
wan=x//10000;
qian=x%10000//1000;
shi=x%100//10;
ge=x%10;
if ge==wan and shi==qian:
    print("It is palindromic number!\n")
else:
    print("It is not palindromic number!\n")
```



4.2.4 选择结构嵌套

if语句的嵌套：如果if的内嵌语句中又使用了一个if语句，则构成if语句的嵌套。

```
if 表达式1:
    if 表达式2:
        语句1
    else:
        语句2
else:
    if 表达式3:
        语句3
    else:
        语句4
```

内嵌if

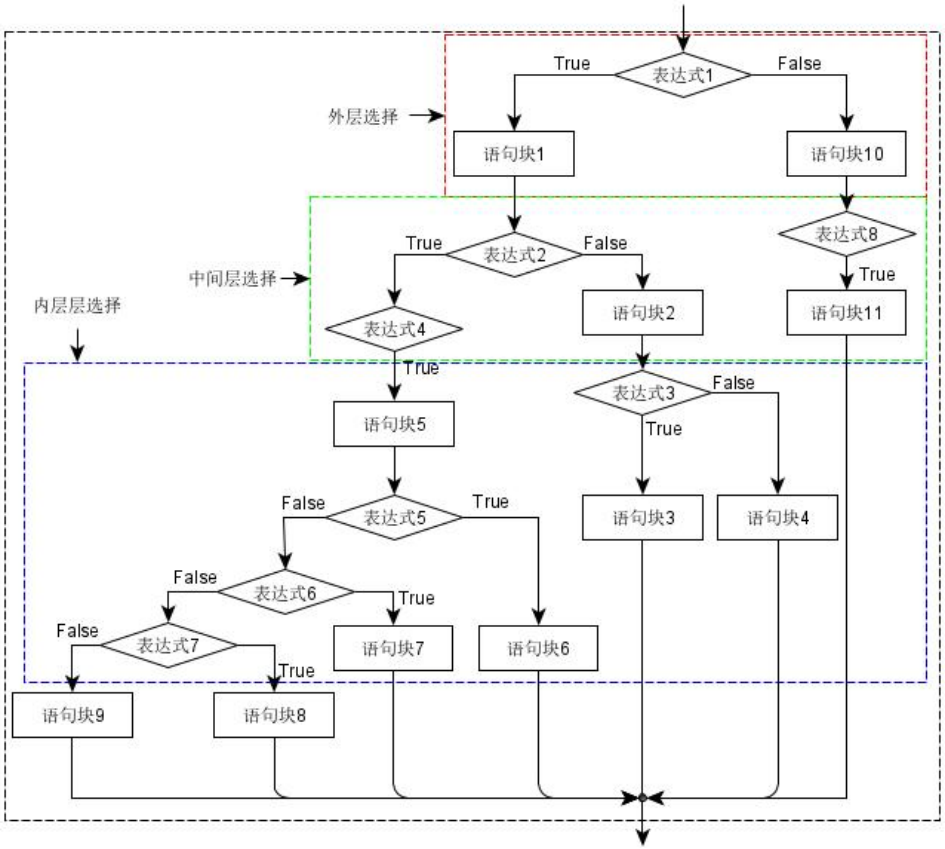
内嵌if

外包if

4.2.4 选择结构嵌套

if语句之中可以嵌套if单分支语句，也可以嵌套if双分支语句或者多分支语句，当然每一个判断中都可以嵌套三种结构，需要注意的是层次结构不能内外层交叉，需要缩进对齐。

```
if <表达式1>:                    //第1层判断
    语句块(1)
    if  <表达式2>:               //第2层判断
        语句块(2)
        if <表达式3>:            //第3层判断
            语句块(3)
        else:                    //第3层判断
            语句块(4)
    elif <表达式4>:              //第2层判断
        语句块(5)
        if  <表达式5>:           //第3层判断
            语句块(6)
        elif <表达式6>:          //第3层判断
            语句块(7)
        elif <表达式7>:          //第3层判断
            语句块(8)
        else:                    //第3层判断
            语句块(9)
else:                            //第1层判断
    语句块(10)
    if  <表达式8>:               //第2层判断
        语句块(11)
```



选择结构综合举例

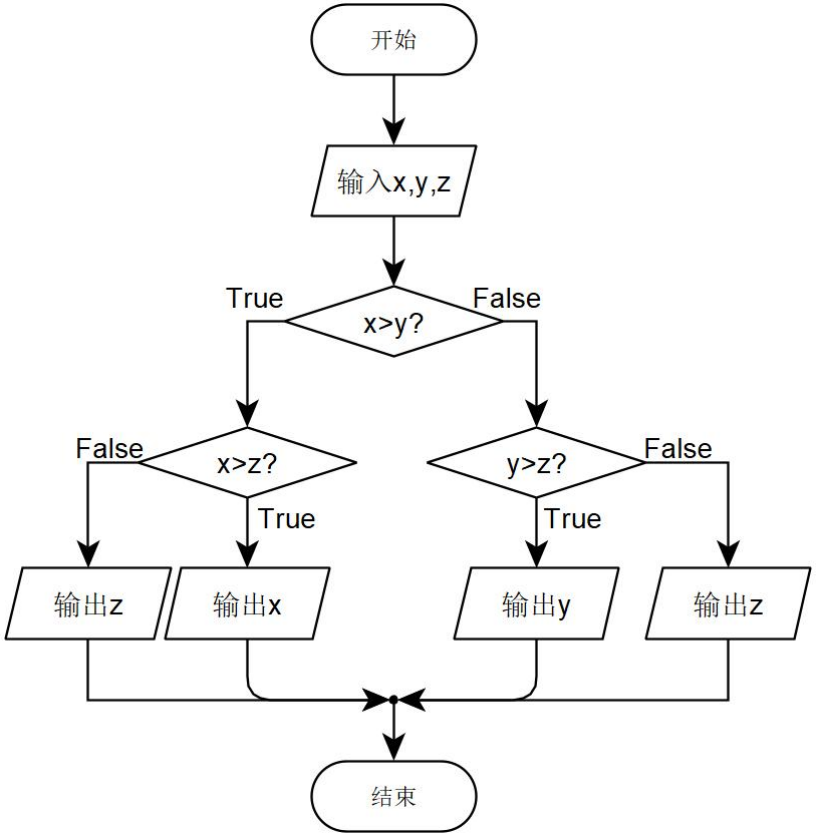
例:任给三个实数，求其中的最大值，并输出。

方法一分析：可以先从三个数中任选两个进行比较，留下较大值，然后将其与第三个数比较，得到最大值。

自然语言算法描述：

（1）三取二，二取一，三个中任取两个数比较，找出较大的一个；

（2）二取一，（1）中较大者与第三个数比较，得到最大值。





选择结构综合举例

```
x,y,z =map(float,input().split(","))
if x>y:
    if x>z:  #与第三个数比较
        print(x)
    else:   #x<=z
        print(z)
else:         #x<=y
    if y>z:  #与第三个数比较
        print(y)
    else:    #y<=z
        print(z)
```

选择结构综合举例

例:任给三个实数，求其中的最大值，并输出。

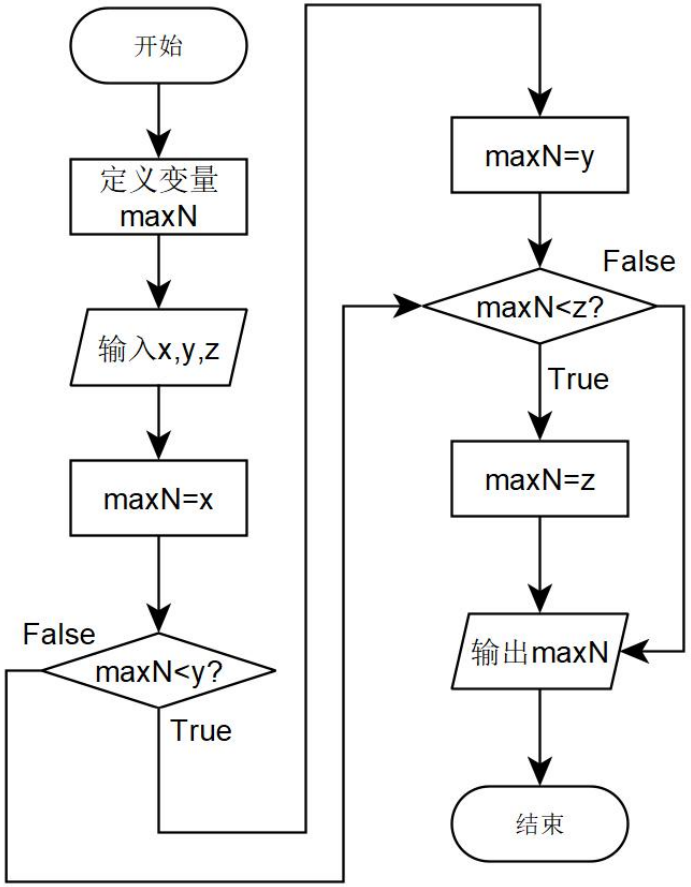
方法二分析：申请一个变量把第一个数赋值给它作为最大值，然后依次跟剩余数比较，碰到比该变量大的就赋值给它，经过一轮后该变量中存放三个数的最大值。

自然语言算法描述：

（1）定义最大值变量maxN，第1个数赋值给它；

（2）从第2个数开始，直到最后一个数结束，依次与maxN比较，如果大于maxN，则赋值给maxN；

（3）输出maxN，maxN为所有数中的最大值。





选择结构综合举例

```
maxN=0.0  #定义最大值变量
x,y,z =map(float,input().split(","))
maxN=x  #取第1个数给最大值变量
if maxN<y: #依次跟后边数比较
    maxN=y  #保留较大值
if maxN<z:    #依次跟后边数比较
    maxN=z  #保留较大值
print(maxN)
```



小结

- 深刻理解选择结构描述的物质运动规律特征。掌握使用N-S图或流程图描述选择结构算法的方法。
- 熟练掌握选择结构的Python描述语法：单分支、双分支、多分支以及选择嵌套。
- 熟悉常用的用选择结构描述的算法，如选最大最小值、成绩评定、输入字符分类等。



作业

① 购买地铁车票的规定如下：乘1-4站，3元/位；乘5-9站，4元/位；乘9站以上，5元/位。 输入人数、站数，输出应付款。

② 输入一个整数，判断它是否为水仙花数。所谓水仙花数，是指这样的一些三位整数：各位数字的立方和等于该数本身，例如$\mathbf{153} = \mathbf{1}^{\mathbf{3}} + \mathbf{5}^{\mathbf{3}} + \mathbf{3}^{\mathbf{3}}$，因此153是水仙花数。



第4章 Python控制结构



陕西省语音与图像信息处理重点实验室
Shaanxi Provincial Key Laboratory of Speech & Image Information Processing



空天地海一体化大数据应用技术国家工程实验室
Integrated Air-Space-Ground-Ocean Big Data Application Technology



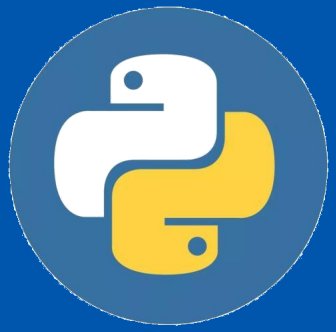
西北工業大學
Northwestern Polytechnical University

Python程序设计

Python Programming

主　讲：　邹晓春　电子信息学院

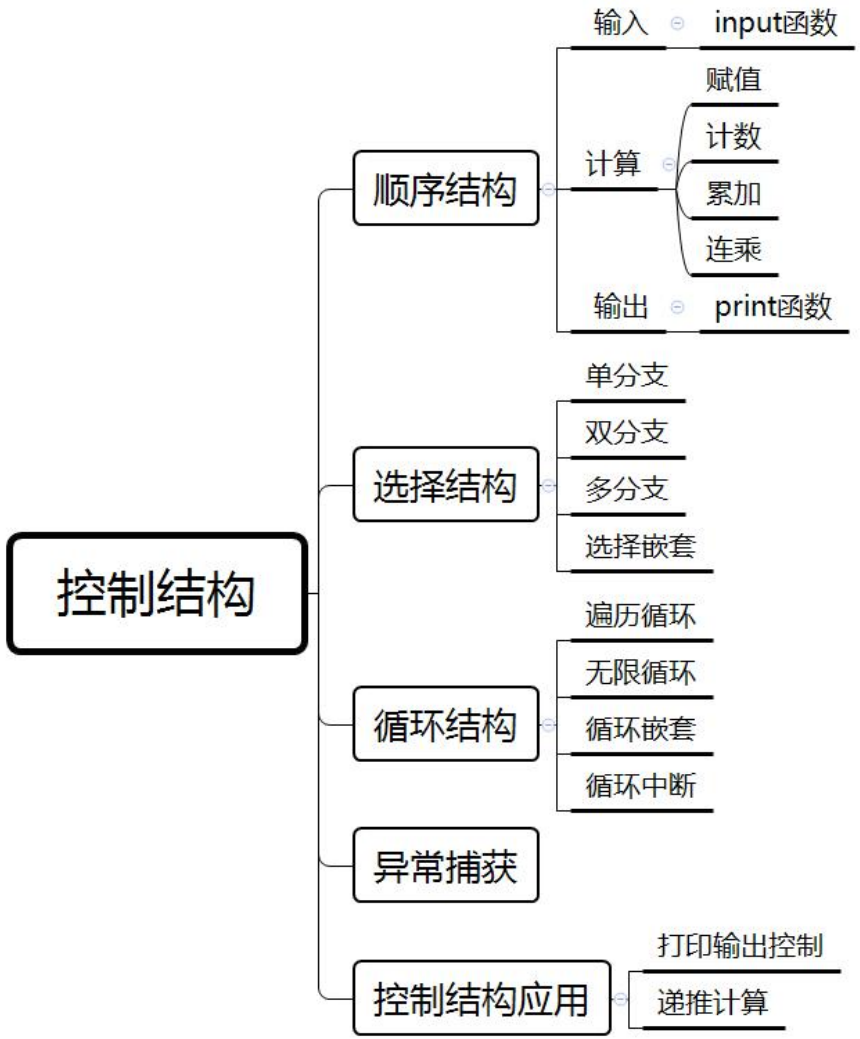
QQ　群：　Python程序设计
760581125



《Python程序设计》

第4章　Python控制结构

第4章 Python控制结构





第4章 Python控制结构

4.1 Python顺序结构

4.2 Python选择结构

4.3 **Python循环结构**

4.4 Python控制结构应用举例

引例

要求：打印由星型构成的等腰三角形，图案如下：



```
    *
   * *
  * * *
 * * * *
* * * * *
```



```
        *
       * *
      * * *
     * * * *
    * * * * *
   * * * * * *
  * * * * * * *
 * * * * * * * *
* * * * * * * * *
```

```
print("    *")
print("   * *")
print("  * * *")
print(" * * * *")
print("* * * * *")
```

```
n=eval(input())
for i in range(1,n+1):
    print(" "*(n-i),"* "*i)
```



循环结构程序设计

4.3.1 for循环

4.3.2 while循环

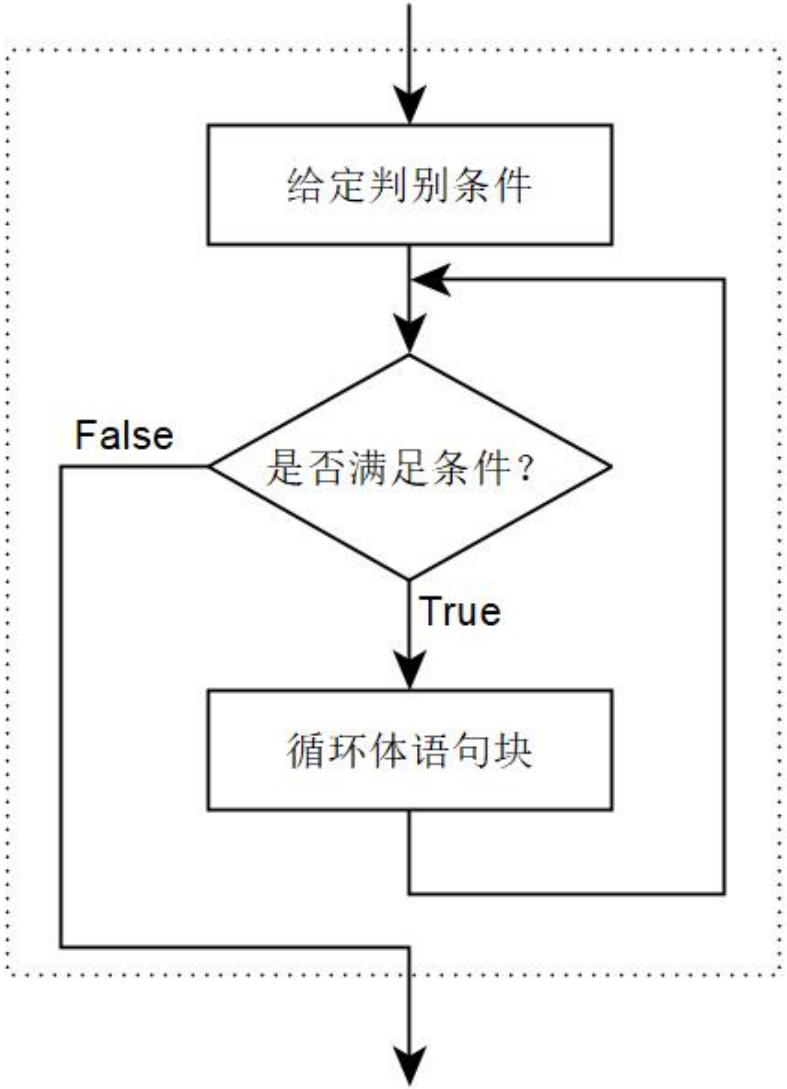
4.3.3 循环的嵌套

4.3.4 循环控制语句

4.3.5 循环结构程序举例

循环结构的基本类型

- **当型循环**
  for, while





循环结构程序设计

循环结构程序设计的应用场合：当某个条件满足时，需要多次反复进行同样工作。

例如:

1、求数列1，2，…，i, … 的前n项和

2、求实数e的x次方

3、判断一个整数是否是素数

……



4.3.1 for循环（遍历循环）

□ for循环一般形式：

for 目标变量 in 序列对象：

循环体语句

说明：

for循环的首行定义了目标变量和遍历的序列对象，后面是需要重复执行的语句块。语句块中的语句要向右缩进，且缩进量要一致。



4.3.1 for循环（遍历循环）

注意：

（1）for语句是通过遍历任意序列的元素来建立循环的。

（2）for语句也支持一个可选的else块，一般格式如下：

```
for 目标变量 in序列对象：
    语句块
else：
    语句
```

（3）else语句只在循环正常执行后才执行并结束，因此，可以在<语句块2>中放置评价循环执行情况的语句。



4.3.1 for循环（遍历循环）

循环语句四要素：

- 循环（控制）变量初始化
- 循环控制条件
- 循环体（关键是找规律）
- 循环变量的修改



4.3.1 for循环（遍历循环）

□ range 函数在for循环中的应用

range函数返回的是可迭代对象，range（）函数的一般格式为：

range（[start,]end[,step]）

range ()函数共有三个参数，start和step是可选的，start表示开始，默认值为0，end表示结束，step表示步长，默认值为1。函数的功能是从一个start参数的值开始，到end参数的值结束的数字序列。



4.3.1 for循环（遍历循环）

例4-1 range()

传递一个参数的range()函数：

```
>>> for i in range(5):
        print(i)
```

传递两个参数的range()函数：

```
>>> for i in range(2,4):
        print(i)
```

传递三个参数的range()函数：

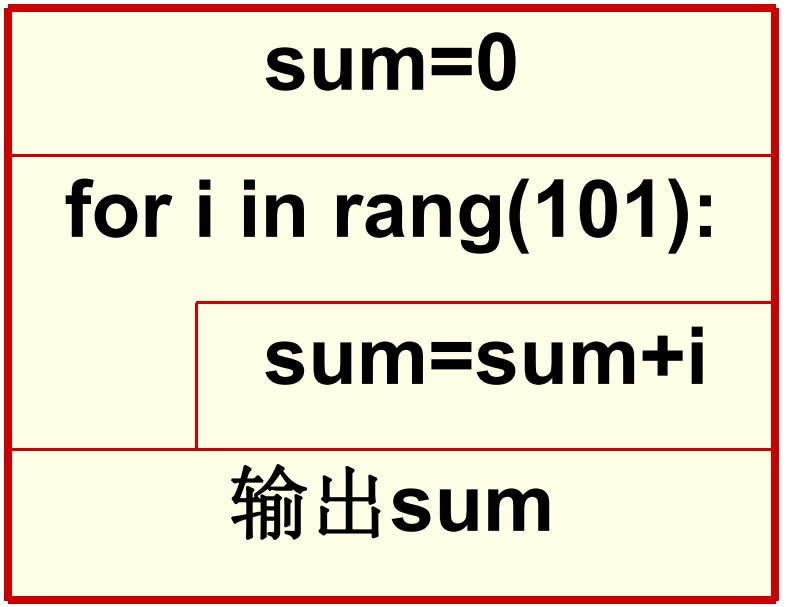
```
>>> for i in range(2,20,3):
        print(i)
```



4.3.1 for循环（遍历循环）

例4-2: 用for循环求$\sum_{i=0}^{n} i$

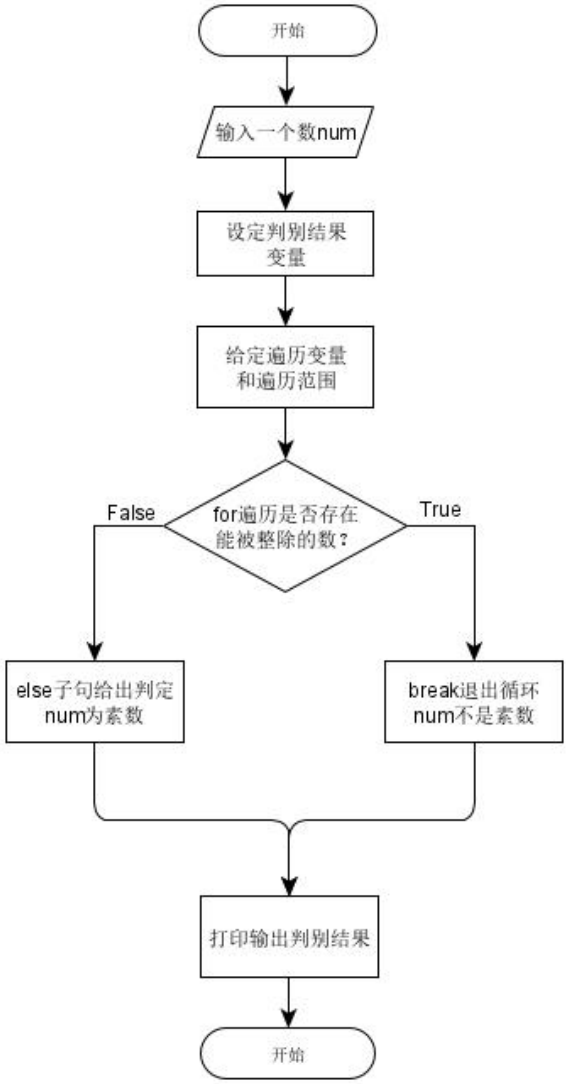
```
sum=0
for i in range(101):
        sum=sum+i
print("1+2+3+....+100", sum)
```



4.3.1 for循环（遍历循环）

例4-3: 判断num是否为素数。

```
num=int（input(“请输入要判断的正整数num”)）
flag=1
for i in range(2, num):
    if  num%i==0:
    flag=0
    break
if  flag==1:
    print(“%d 是素数”%num)
else:
    print(“%d不是素数”%num)
```





4.3.1 for循环（遍历循环）

例4-4：已知四位数3025具有特殊性质：它的前两位数字30与后两位数字25之和是55，而55的平方正好等于其本身3025。编程列举出具有该性质的四位

```
print("满足条件的四位数分别是：")
 for i in range(1000,10000):
                a=i//100
                b=i%100
                if (a+b)* *2==i:
                        print(i)
```



4.3.1 for循环（遍历循环）

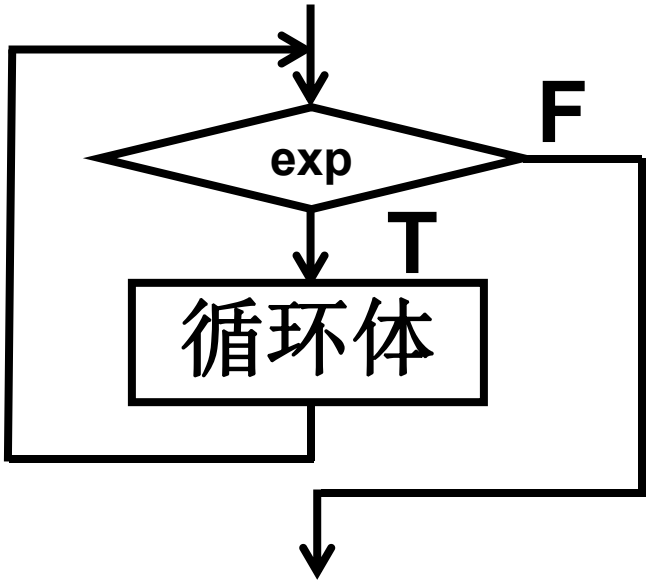
例4-5: 求出1到100能被7或11整除但不能被7和11同时整除的所有整数并将它们输出。每行10个。

```
print(“满足条件的数分别是：”)
count=0
for i in range(1,100):
        if i%7==0 and i%11!=0 or i%11==0 and
i%7!=0:
                print(i ,end=”    ”)
                count=coun+1
                if count%10==0:
                        print(“”)
```

4.3.2 while语句(无限循环)

- 特点：先判断，再执行
- while 语句的语法格式：

```
while exp:
    循环体
```



注意：

1. 循环体可能一次也不执行，可能无限次循环；
2. 循环体可为任意类型语句；
3. 退出while循环的情况：
   - exp不成立
   - 循环体内遇break, return, exit等。

例4-6:求$\sum_{n=1}^{100} n$

- 1）问题分析：

sum = 1+2+…+100

100个数

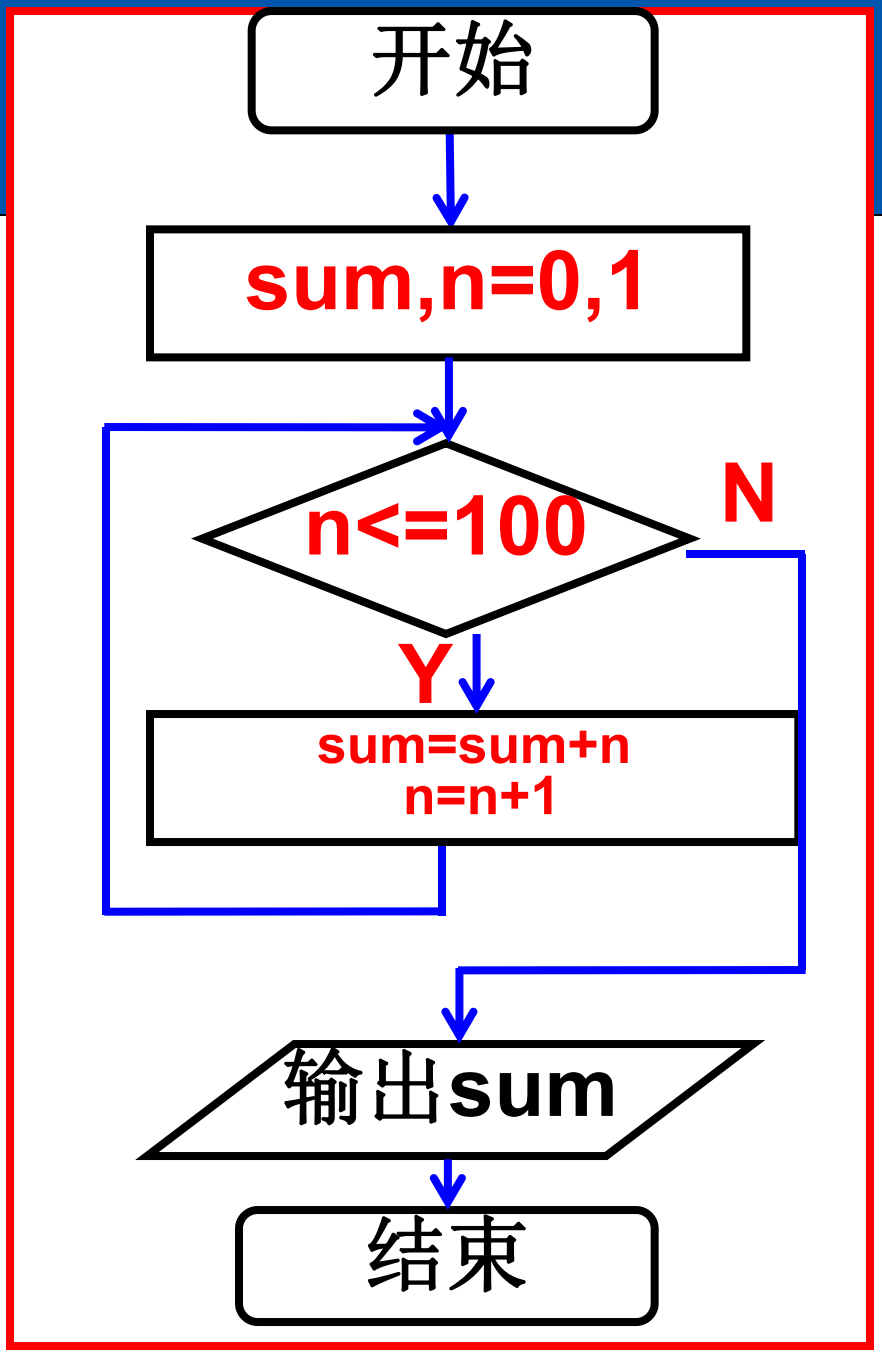
- 循环语句：

循环变量初始化：n＝1

循环控制条件：n<=100

循环体：sum+=n

循环变量的修改：n=n+1





例4-6:求$\sum_{n=1}^{100} n$

```
sum,n=0,1
while n<=100:
    sum=sum+n
    n=n+1
printf("1+2+3+....+100=",sum)
```

运行结果：1+2+3.....+100=5050



在while语句中使用else子句

Python可以在循环语句中使用else子句，即构成：while……else循环结构。

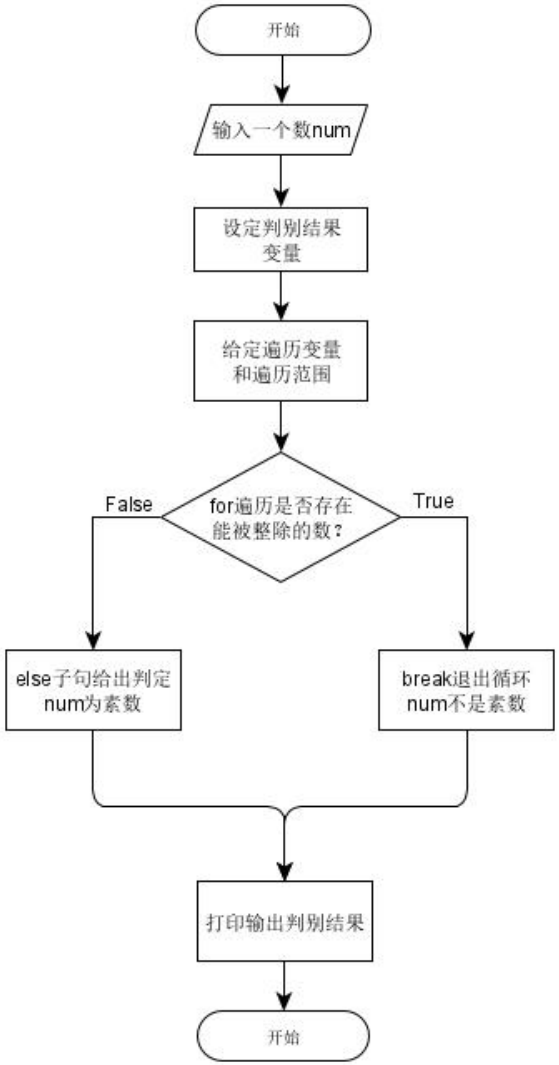
```
while exp:
    语句块1
else:
    语句块2
```

else语句只在循环正常执行后才执行并结束，因此，可以在<语句块2>中放置评价循环执行情况的语句。

while....else循环结构

例4-7：用while循环实现素数判断

```
x=eval(input("请输入一个大于1的自然数："))
judge=""
i=2
while i<x:
    if x%i==0:
        judge=str(x)+"不是素数。"
        break
    else:
        i+=1
else:
    judge=str(x)+"是素数。"
print(judge)
```





温馨提示：

对于while语句的用法，要注意以下几点：

★如果while后面表达式的值一开始就为假，则循环体一次也不执行。

★循环体中的语句可以任意类型的语句。

★遇到下列情况，退出while循环：

A 表达式不成立

B 循环体内遇到break、return语句。



温馨提示：

★while 语句之前必须预先设置判断的初始条件；

★防止“死循环”的发生，在循环体中必须有使循环趋向结束的操作，即 在循环体中要有能让逻辑表达式值改变的语句；

★正确使用缩进；当循环体中含多条语句时,需用缩进这些多条语句，而且缩进一定要符合逻辑关系；

★在循环体中，语句的先后位置必须符合逻辑，否则会影响运算结果。



4.3.3 循环嵌套

循环的嵌套，又称多层循环，应用于多个变量同时变化的情况。

说明：1. 外层循环可包含两个以上内循环
2. 嵌套的循环的执行流程

4.3.3 循环嵌套

注意：

- while、for循环语句可以并列,也可以相互嵌套,但要层次清楚,不能出现交叉
- 多重循环程序执行时，外层循环每执行一次，内层循环都需要循环执行多次。

```
for i in range (6):
        for j in range (10):
                ……
```

1. 外循环执行了6次
2. 内循环执行10次
3. 循环正常结束时，内循环执行了10×6=60次



4.3.3 循环嵌套

循环语句while和for可以相互嵌套。在使用循环嵌套时，应注意以下几个问题：

- 外层循环和内层循环控制变量不能同名，以免造成混乱。
- 循环嵌套的缩进在逻辑上一定要注意，以保证逻辑上的重要性。
- 循环嵌套不能交叉，即在一个循环体内必须完整地包含另一个循环，如图所示的循环嵌套都是合法的嵌套形式。

4.3.3 循环嵌套

合法的嵌套格式：

```
for <循环变量> in <遍历结构>:
    ...
    for <循环变量> in <遍历结构>:
        ...
```

```
for <循环变量> in <遍历结构>:
    ...
    while <循环变量>:
        ...
```

```
while <条件>:
    ...
    for <循环变量> in <遍历结构>:
        ...
```

```
while <条件>:
    ...
    while <条件>:
        ...
```



4.3.3 循环嵌套

循环嵌套在使用中的重点内容：

- 能够正确判断内层循环体的范围
- 能够正确判断内层循环体的执行次数
- 注意内层循环的初值设定
- 能正确处理内层循环与外层循环变量的关系



例4-8：输出九九表，输出格式如下：

```
1*1=1
1*2=2  2*2=4
1*3=3  2*3=6  3*3=9
1*4=4  2*4=8  3*4=12  4*4=16
```

```
for i in range (1,10,1):
    for j in range(1,i+1,1):
        print("%d*%d=%2d  "%(j,i,i*j),end=" ")
    print("")
```

*9=81



4.3.3 循环嵌套

例4-9：找出所有的三位数，要求它的各位数字的立方和正好等于这个三位数。例如：$153=1^3+5^3+3^3$就是这样的数。

```
for i in range(1,10):
    for j in range(0,10):
        for k in range(0,10):
            if i**3+j**3+k**3==i*
                print("%d%d%
```

程序运行结果：

```
153
370
371
407
```



4.3.3 循环嵌套

例4-10：求100~200之间的全部素数。

```
print("100~200之间的素数有：")
for m in range(100,200):
    flag=1
    for i in range(2,m):
        if  m%i==0:
            flag=0
            i=m
    if flag==1:
            print(m,end=" ")
```



4.3.4 循环控制语句

有时候我们需要在循环体中**提前**跳出循环，或者在某种条件满足时，不执行循环体中的某些语句而立即**从头**开始新的一轮循环，这时就要用到循环控制语句**break**、**continue**和**pass**语句。

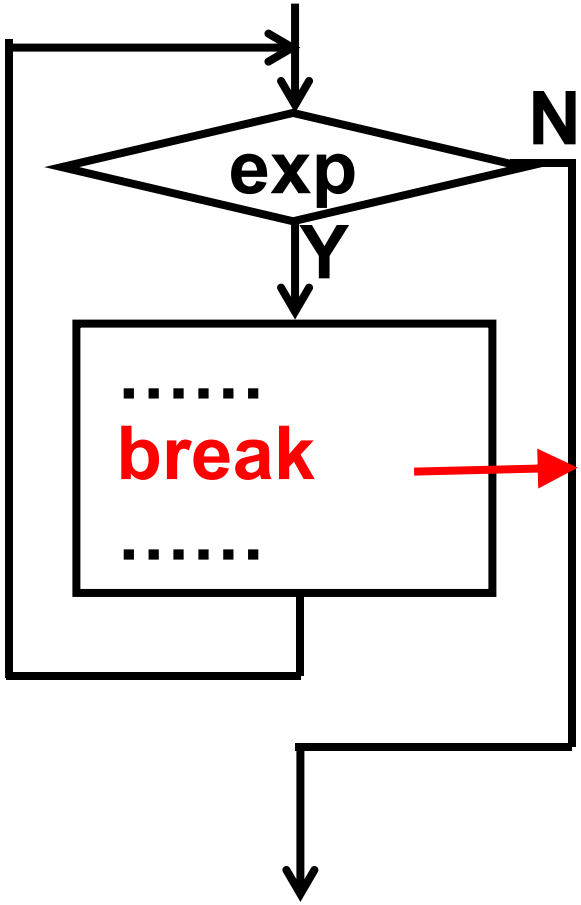
break语句

一般格式：

break

功能：

break语句用在循环体内，迫使所在循环立即终止，即跳出所在循环体，继续执行循环结构后面的语句。





例4-11：求两个整数m与n的最大公约数

- **分析：**找出m与n中较小的一个，则最大公约数必在1与较小整数的范围内。使用for语句，循环变量i从较小整数变化到1。一旦循环控制变量i同时能被m与n整除，则i就是最大公约数，然后使用break语句强制退出循环。

```
m,n=eval(input("请输入两个整数："))
if m<n:
    min=m
else:
    min=n
for i in range(min,1,-1):
  if m%i==0 and n%i==0:
      print("最大公约数是：",i)
      break
```

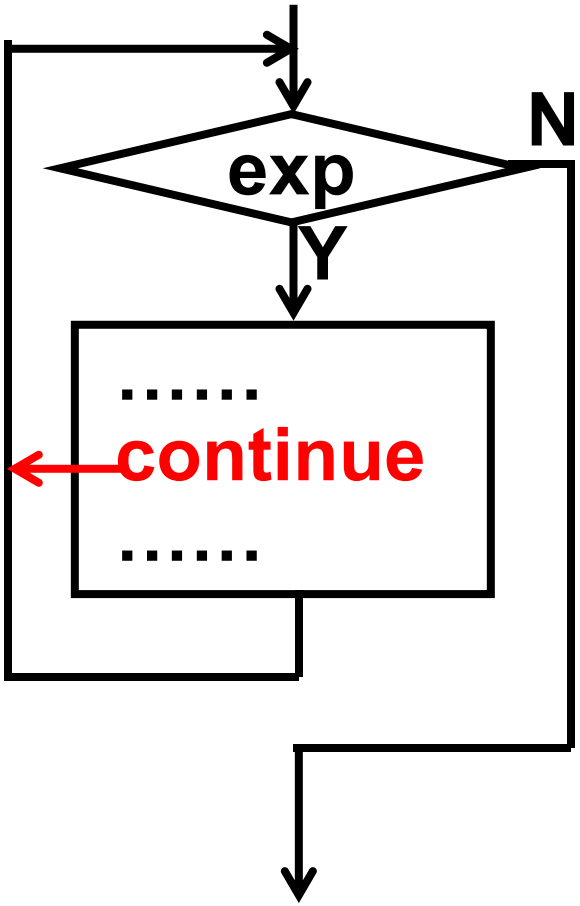
continue语句

一般格式：

continue

功能：

在循环语句中强行提前结束本次循环，而不是终止循环。





例4-12 求1～100之间的全部奇数之和。

程序如下：

```
x=y=0
while True:
    x+=1
    if  x%2==0:continue
    elif x>100:break
    else:y+=x
print("y=",y)
```

程序运行结果：

```
y= 2500
```



pass语句

pass语句是一个空语句，它不做任何操作，代表一个空操作，在特别的时候用来保证格式或是语义的完整性。例如下面的循环语句：

```
for i in range(5):
    pass
```

该语句的确会循环5次，但是除了循环本身之外，它什么也没做。



例4-13：pass语句应用：逐个输出“Python”字符串中的字符。

```
for letter in "Python":
  if letter == "o":
     pass
     print("This is pass block")
  print("Current Letter :", letter)
print("End!")
```

程序运行结果：

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : h
This is pass block
Current Letter : o
Current Letter : n
End!
```

4.3.5 循环结构程序举例

例4-14：利用下面的公式求$\frac{\pi}{4}$的近似值，要求累加到最后一项小于10^{-6}为止。

$$\frac{\pi}{4} \approx 1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\cdots\cdots$$

程序如下：

```
import math
s=1
n=1.0
t=1.0
pi=0
while math.fabs(t)>=1e-6:
    pi=pi+t
    n=n+2
    s=-s
    t=s/n
pi=pi*4
print("PI=%f"%pi)
```

程序运行结果：

PI=3.141591



例4-15：百钱买百鸡

公鸡5文钱一只，母鸡3文钱一只，小鸡3只一文钱，用100文钱买一百只鸡，其中公鸡、母鸡、小鸡都必须要有，问公鸡，母鸡，小鸡要买多少只刚好凑足100文钱?

分析：显然这是一个组合问题，也可以看作是解不定方程的问题，采用列举的方法实现。令i、j、k分别表示公鸡、母鸡和小鸡的数目。

例4-15：百钱买百鸡

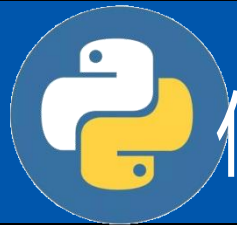
[方法一]令i、j、k的列举范围分别为：
i：1~20（公鸡最多能买20只）
j：1~33（母鸡最多能买33只）
k：1~100（小鸡最多能买100只）
可以采用三重循环逐个搜索。
程序如下：

```
for i in range(1,21):
```

程序运行结果：

```
公鸡：4只,母鸡：18只,小鸡：78只
公鸡：8只,母鸡：11只,小鸡：81只
公鸡：12只,母鸡：4只,小鸡：84只
```

在程序中，循环体被执行了
20×33×100=66000次。

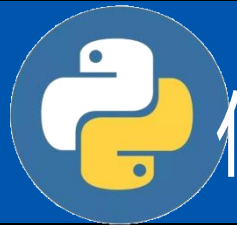


例4-15：百钱买百鸡

[方法二]令i、j、k的列举范围分别为（保证每种鸡至少买一只）：
i：1~18（公鸡最多能买18只）
j：1~31（母鸡最多能买31只）
k：100-i-k（当公鸡和小鸡数量确定后，小鸡的数量可计算得到）
可以采用两重循环逐个搜索。
程序如下：

```
for i in range(1,19):
    for j in range(1,32):
        k=100-i-j
        if i+j+k==100 and i*5+j*3+k/3==100:
            print("公鸡：%d只,母鸡：%d只,小鸡：%d只"%(i,j,k))
```

在程序中，循环体被执行了18×81=558次。



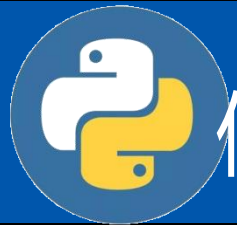
例4-15：百钱买百鸡

[方法三]从题意可得到下列方程组：

由方程组可得到式子7i+4j=100。由于i和j至少为1，因此可知i最大为13，j最大为23。方法二的两重循环可改进为以下程序：

```
  for i in range(1,14):
      for j in range(1,24):
            k=100-i-j
            if i+j+k==100 and i*5+j*3+k/3==100:
                  print("公鸡：%d只,母鸡：%d只,小鸡：%d只"%(i,j,k))
```

该程序的循环体被执行了14×24=336次。



例4-15：百钱买百鸡

[方法四] 由方法三中的式7i+4j=100可得：j=(100-7i)/4。观察7i+4j=100，4j同100都是4的倍数，因此i一定也是4的倍数。有了这些条件，程序实现时只需要对i进行逐个搜索即可，i的搜索范围为1~13。

采用单层循环进行逐个搜索。

程序如下：

```
for i in range(1,14):
    j=(100-7*i)/4
    k=100-i-j
    if i%4==0:
        print("公鸡：%d只,母鸡：%d只,小鸡：%d只"%(i,j,k))
```

该算法程序只循环了13次。



小结

- for循环
- while循环
- 循环的嵌套
- 循环控制语句

作业

- 思考题

编程实现求e^x,$\sin(x)$,$\cos(x)$，先写出算法步骤，再画出流程图，最后编程实现。

提示：$e^x = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!} + \cdots$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots + (-1)^k \frac{x^{2k}}{(2k)!} + \cdots$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots + (-1)^k \frac{x^{2k+1}}{(2k+1)!} + \cdots$$